

ESTRUCTURAS DE DATOS

APUNTES DE LA ASIGNATURA

CURSO 2025-26



©2026

Algunos derechos reservados.

Sergio Caveró Díaz (sergio.cavero@urjc.es)

Salvador Sánchez Alonso (salvador.sanchez@urjc.es)

Este documento se distribuye bajo la licencia Compartir Igual 4.0 (CC BY-SA 4.0)

Tabla de contenidos

Tema I – Introducción	5
1. La abstracción.....	5
Un ejemplo práctico: el coche automático.....	7
2. Tipos de datos	8
Especificación formal.....	9
3. Memoria dinámica.....	9
Punteros en Pascal: Concepto y declaración.....	10
Operaciones con punteros.....	11
Tema II – Tipos Abstractos de Datos (TAD).....	12
1. Definición y esencia de los TADs.....	12
2. Especificación formal	13
Ejemplo 1: Semáforo Binario	13
Ejemplo 2: Semáforo de 3 estados	14
Ejemplo 3: Contador simple.....	14
Ejemplo 4: Contador con Límite	14
3. Implementación en Pascal.....	15
4. Beneficios del uso de TADs	16
Resumen	17
Tema III – Análisis de la complejidad de los algoritmos	18
1. Conceptos Fundamentales	18
Complejidad Temporal	18
El caso más desfavorable (Worst-Case Analysis).....	19
Complejidad espacial.....	19
2. Notación O grande (Big-O)	19
3. Reglas para calcular complejidades	21
Regla de la suma	21
Regla del producto	21
Análisis de algoritmos recursivos	22
4. Implicaciones en el diseño de Estructuras de Datos	23
Optimización prematura vs. análisis informado.....	23
Tema IV – Pilas	25
1. El TAD Pila	25
2. Implementación de una pila mediante un array	26
3. Implementación con memoria dinámica.....	28
4. Aplicaciones prácticas.....	30
Resumen	31

Tema V – Colas	32
1. El TAD Cola	32
2. Implementación con arrays.....	33
Array lineal con eliminación simple	34
Array lineal con reorganización.....	34
Array circular	35
Comparativa de implementaciones con arrays.....	37
3. Implementación con memoria dinámica.....	37
4. Aplicaciones prácticas de las colas	39
5. Complejidad computacional de las operaciones	40
Tema VI – Listas	41
1. El TAD Lista	41
2. Lista enlazada simple	42
Inicialización.....	42
Inserciones	42
Eliminación de elementos	43
Pertenencia de un elemento	44
Otras operaciones.....	45
3. Lista circular	46
4. Listas doblemente enlazadas	48
5. Aplicaciones y estructuras derivadas.....	51
Tema VII - Conjuntos y bolsas	53
1. El TAD Conjunto	53
2. El TAD Bolsa	55
3. Implementación	56
Implementación con arrays	56
Implementación con listas enlazadas	57
Implementación con un árbol binario de búsqueda.....	57
Implementación con Tablas Hash	58
4. Aplicaciones y uso de conjuntos y bolsas.....	60
Resumen	61
Tema VIII - Árboles	62
1. Tipos de árboles	63
2. El TAD Árbol binario	63
Recorridos	65
3. El TAD Árbol Binario de búsqueda (BST).....	66
4. Árboles AVL.....	69
Ejemplo de rotaciones	70

Tema IX. Grafos.....	71
1. El TAD Grafo	72
Representación mediante matriz de adyacencia	73
Representación dinámica	73
2. Implementación de un digrafo valorado con matriz de adyacencia	75
3. Implementación de un grafo valorado con listas de adyacencia	77
4. Recorridos	80
Recorrido en anchura (BFS).....	80
Recorrido en profundidad (DFS)	81

Tema I – Introducción

El estudio de las Estructuras de Datos es fundamental en la Ingeniería del Software, pues sienta las bases para diseñar soluciones computacionales eficientes, modulares y elegantes. Para construir sobre cimientos sólidos, iniciaremos nuestro recorrido explorando tres pilares conceptuales esenciales. En primer lugar, abordaremos el concepto de abstracción, el proceso mental que nos permite modelar la complejidad del mundo real identificando y aislando las características esenciales de un problema, ignorando los detalles superfluos. A continuación, veremos cómo esta abstracción se materializa en el código a través de los tipos de datos y su especificación formal, donde definiremos el "contrato" que gobierna el comportamiento de nuestras estructuras. Finalmente, estudiaremos el mecanismo que dota de flexibilidad a estas construcciones: la gestión de memoria dinámica mediante punteros, la herramienta que nos permitirá crear estructuras de tamaño variable y relaciones complejas en tiempo de ejecución. Dominar estos tres elementos no es un mero ejercicio teórico; es la preparación indispensable para comprender, diseñar e implementar las estructuras de datos clásicas —como listas, pilas, colas y árboles— que constituyen el esqueleto de cualquier sistema software robusto y bien diseñado.

1. La abstracción

El diseño y la implementación de software eficiente y fácil de mantener comienza con un concepto fundamental: la abstracción. Podemos definir abstracción como el proceso mental mediante el cual el ser humano extrae las características esenciales de un objeto, un sistema o una idea, ignorando deliberadamente los detalles superfluos que no son relevantes para el propósito inmediato. Este mecanismo es primordial para poder modelar la complejidad del mundo real en un programa de ordenador sin vernos abrumados por una infinidad de pormenores.

Imaginemos, por ejemplo, un mapa de metro. Este mapa no intenta reproducir con exactitud topográfica las curvas, distancias o pendientes de las vías; en cambio, abstrae la información crucial: las líneas, las estaciones y sus interconexiones. Esta simplificación permite al usuario planificar un viaje de manera eficiente, una tarea mucho más difícil con un plano a escala real.



Ilustración 1. Plano de metro (vista parcial)

La abstracción es un proceso mental que aplicamos constantemente. Por ejemplo, cuando dibujamos. Imagina que tienes que representar un gato. ¿Cómo lo harías? La respuesta depende completamente de para qué necesitas el dibujo y qué información es esencial comunicar. La abstracción es precisamente esto: el arte de seleccionar lo fundamental y eliminar lo superfluo, dependiendo del contexto y objetivo. Así, podemos elegir un esquema minimalista con formas

geométrica básicas tales como triángulos y líneas simples, sin detalles anatómicos precisos, pasando por un dibujo más estilizado (formas simplificadas con detalles seleccionados que comunican carácter), hasta una representación detallada con mayor fidelidad anatómica, texturas e incluso sombras que buscarían un mayor realismo. Cada nivel de abstracción sirve, por tanto, a un propósito distinto, seleccionando qué información incluir y qué omitir según el contexto y el objetivo de la representación.

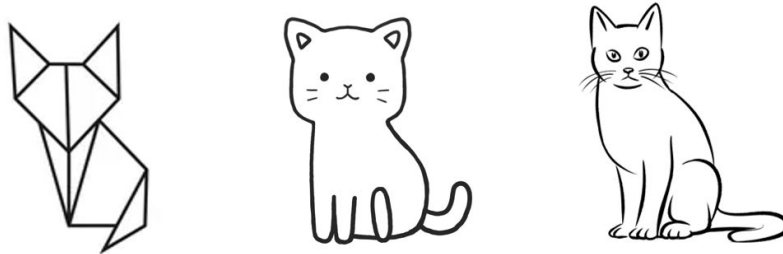


Ilustración 2. Diferentes niveles de abstracción en la representación de un gato

En la Ingeniería del Software, la abstracción permite concebir sistemas complejos manejando solo aquellos aspectos que nos interesan en un momento dado y ocultando la complejidad subyacente. En este contexto hay varios elementos de gran utilidad que presentamos a continuación.

El concepto de **caja negra** implica estudiar un sistema centrándose exclusivamente en su comportamiento externo observable, obviando por completo su funcionamiento interno. Es como si el sistema estuviera encerrado en una caja opaca que no podemos abrir. Este enfoque tiene sus raíces en la ingeniería eléctrica de los años 1940 (métodos de Cauer), donde se analizaban circuitos por la respuesta que producían a ciertas entradas, sin necesidad de conocer cada resistor o capacitor individual. El científico Ross Ashby generalizó posteriormente esta idea en el campo de la cibernética (1958), afirmando que todo sistema complejo puede ser estudiado como una máquina cuyo interior se ignora.



Ilustración 3. Caja negra

La **interfaz** es el conjunto de elementos (operaciones, métodos, funciones) mediante los cuales se puede interactuar con un objeto visto como una caja negra. Es el medio de comunicación entre componentes de software.

La interfaz de un ratón de ordenador define el modo en que se comunican el dispositivo y el usuario (o el sistema operativo). Físicamente, se compone del conector USB (o receptor inalámbrico), los botones primarios y secundarios, la rueda de desplazamiento y, en algunos modelos, botones laterales programables. A nivel lógico y de software, esta interfaz se traduce en un conjunto estandarizado de señales y protocolos que comunican eventos como "clic izquierdo", "movimiento delta-X/Y" o "scroll", sin revelar cómo se generan internamente. Gracias a esta interfaz bien definida y universal, cualquier sistema que la implemente—independientemente de su hardware interno, sistema operativo o marca—puede interpretar estas señales y hacer que el cursor responda de manera predecible. Esto permite la interoperabilidad total: el mismo ratón funciona en un PC con Windows, un Mac o una Raspberry Pi, porque todos "hablan" el lenguaje común de la interfaz definida para el ratón.

El **encapsulamiento** es el proceso que pone en práctica la ocultación de los detalles internos de una abstracción. En programación, es esencial para la reutilización y la robustez del código. Al ocultar *cómo* está hecho un módulo (su implementación) y solo exponer *qué* se puede hacer con él (su interfaz), logramos dos objetivos fundamentales:

- Reutilización: Podemos utilizar ese módulo en diferentes programas, confiando en que su comportamiento externo será constante.
- Protección: Impedimos que otros componentes del programa utilicen el objeto de modos no deseados o accedan directamente a su estado interno, lo que podría corromperlo. La interacción solo puede darse a través de las operaciones definidas, que actúan como guardianes.

El encapsulamiento se ejemplifica perfectamente con el ejemplo anterior del ratón de ordenador. Como usuario, solo necesito conocer su interfaz física (conector USB, los botones y la rueda de desplazamiento) para utilizarlo efectivamente. Su funcionamiento interno—el sensor óptico, los circuitos, el firmware—me está deliberadamente ocultado. Esto me permite usar el mismo ratón en cualquier ordenador con puerto USB, sin importar sus componentes internos. Si se rompe, puedo reemplazarlo por cualquier otro que respete la misma interfaz, y seguiré usándolo desde el primer día, demostrando cómo la ocultación de detalles (encapsulación) garantiza intercambiabilidad y reutilización.

Vinculado a todo lo anterior interfaz está el concepto de **contrato**, una metáfora poderosa introducida por Bertrand Meyer. Este contrato establece las obligaciones y los beneficios mutuos y públicamente declarados entre el diseñador de un componente y su usuario. Así:

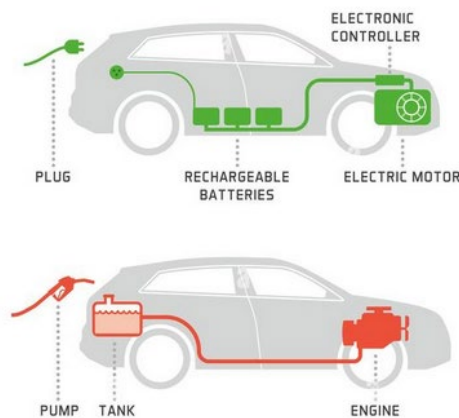
- El diseñador (o implementador) se compromete a *publicar* en la interfaz un conjunto de operaciones que garantizan un comportamiento específico. Oculta todo lo demás mediante encapsulación.
- El usuario (o cliente) se compromete a utilizar *únicamente* esas operaciones publicadas, confiando en que producirán los resultados esperados.

Este "acuerdo" tiene un beneficio enorme: permite al diseñador cambiar completamente la implementación interna de un componente (por ejemplo, hacerlo más rápido o eficiente) sin que el usuario tenga que modificar ni una sola línea de su código, siempre y cuando se respete el contrato (la interfaz) establecido.

Un ejemplo práctico: el coche automático

Para ilustrar estos conceptos, consideremos un coche automático...

- Abstracción: La idea de "un vehículo para transportarse".
- Interfaz/Contrato: Las funciones principales que se ofrecen al conductor: *girar*, *acelerar* y *frenar*. Los mecanismos para ejecutarlas son el volante y los pedales.
- Implementación (detalles ocultos): El motor (diésel, eléctrico, híbrido), el sistema de frenos (de disco, ABS), el tipo de tracción (delantera, trasera, integral) o la dirección (eléctrica, hidráulica).
- Encapsulamiento: El capó del coche actúa literalmente como una barrera que oculta estos detalles complejos. Cualquier conductor,



sin conocer ingeniería mecánica, puede operar cualquier coche gracias a esta abstracción bien definida.

Un buen ejercicio para entrenar el pensamiento abstracto puede ser analizar objetos cotidianos y descomponerlos en sus conceptos clave:

- Interfaz: ¿Con qué interactúo? (botón, manija, superficie).
- Contrato: ¿Qué me promete hacer? (emitir luz, calentar, operar números).
- Posibles implementaciones: ¿De qué maneras distintas se podría construir? (linterna de dinamo vs. de pilas, placa de inducción vs. de gas, bombo físico vs. generador de números aleatorios).
- Nivel de generalidad: ¿Es específico (una cola para un banco) o genérico (una estructura de datos "Cola" que sirve para cualquier escenario)?

Piensa en una linterna, una placa de cocina, un bombo de lotería, un contador, un conjunto de elementos, una cola de personas o una tabla de récords y trata de encontrar respuesta a todos los interrogantes anteriores.

2. Tipos de datos

Una vez comprendida la necesidad de abstraer, debemos encontrar la manera de representar estas abstracciones dentro de un lenguaje de programación. Aquí es donde entran en juego los tipos de datos.

Los lenguajes de programación nos ofrecen un amplio catálogo de tipos de datos, que podemos clasificar de la siguiente manera:

A) Tipos predefinidos. Son de uso muy común y pueden clasificarse en:

- **Simples:** Modelan valores indivisibles. Algunos ejemplos en Pascal son *integer* (números enteros), *real* (números en coma flotante), *boolean* (valores lógicos True/False) y *char* (un carácter).
- **Estructurados:** Agrupan varios valores relacionados, por ejemplo el tipo *string* (cadena de caracteres) que es un tipo estructurado predefinido.

B) Tipos definidos por el usuario: Dado que los tipos básicos predefinidos no dan normalmente cobertura a todas las necesidades de modelado de la información que necesitan nuestros programas, los lenguajes incluyen mecanismos para que el programador pueda crear sus propios tipos. Así, en Pascal tendremos:

- **Simples:** *subrango* (permiten definir un rango de valores, ej: 1..10), *enumerado* (permiten definir una lista de identificadores, ej: (rojo, verde, azul)), o *puntero* (permiten referenciar a otra variable, como veremos más adelante).
- **Estructurados:** *array* (vector o matriz de elementos), *registro* (estructura que agrupa campos generalmente de distintos tipos), *conjunto* (set) o *archivo*.

Un tipo de dato puede y debe verse como una abstracción en sí mismo. Cada instancia de un tipo está sujeta a un "contrato" que define:

- Un **nombre** que lo identifica (*boolean*, *integer*).
- Un conjunto de **valores posibles** que puede tomar (su estado interno: para *boolean* son *true* y *false*; para *integer* es un rango muy amplio).
- Un conjunto de **operaciones** válidas que se pueden realizar con él (para *boolean*: *and*, *or*, *not*; para *integer*: *+*, *-*, ***, *div*, etc.).
- Las **reglas de interacción** con otros tipos (ej: un *array de enteros* utiliza el tipo *integer* para sus elementos).

Por ejemplo el tipo Boolean está definido mediante los siguientes datos:

- Nombre: Boolean
- Valores posibles: True, False.
- Operaciones: AND, OR, NOT.
- Uso de otros tipos: Normalmente, solo opera consigo mismo.

Especificación formal

La especificación de un tipo de datos es la definición formal de su contrato. Podemos definirlo como todo aquello que el cliente (quien lo va a usar) necesita saber para utilizarlo correctamente, sin revelar *cómo* se implementa. Una especificación consta de dos partes:

- **Signatura:** Define la "forma" de las operaciones.
 - Nombre del tipo (ej: TipoCola).
 - Nombres de las operaciones (ej: Encolar, Desencolar, EstaVacía).
 - Parámetros de entrada y tipo de resultado de cada operación.
- **Axioma:** Define la "semántica" o comportamiento de las operaciones. Describe cómo interactúan entre sí, a menudo mediante reglas o pre/postcondiciones. Por ejemplo, un axioma para una cola podría ser: "Desencolar aplicado a una cola resultante de Encolar(*X*, *C*) debe devolver *X* y una cola equivalente a *C*".

Esta separación entre *qué hace* (especificación) y *cómo lo hace* (implementación) es el núcleo de la programación abstracta y permite diseñar programas más flexibles y fáciles de mantener, un objetivo fundamental dentro de la ingeniería del software.

3. Memoria dinámica

Hasta ahora, hemos trabajado principalmente con **estructuras de datos estáticas**, como los arrays. Estas estructuras, aunque útiles, presentan rigideces importantes:

- **Tamaño fijo:** Su capacidad debe conocerse en tiempo de compilación, algo que no siempre es posible. No se puede redimensionar un array estándar durante la ejecución del programa.
- **Reorganización costosa:** Modificar el orden de los elementos (como mover el último al principio) puede requerir desplazar muchos elementos, una operación ineficiente.

Para superar estas limitaciones y poder crear estructuras que se adapten a necesidades desconocidas antes de la ejecución, necesitamos gestionar la memoria de forma dinámica. La herramienta fundamental para esto, en lenguaje Pascal, es el puntero.

Pero antes de entrar en detalles, y para entender los punteros, creemos que es útil visualizar cómo se organiza la memoria de un programa en ejecución (proceso). Esta memoria generalmente, se divide en cuatro segmentos:

- **Segmento de Código (Code):** Contiene las instrucciones del programa. Asignación automática.
- **Segmento de Datos Globales (Global/BSS):** Almacena variables globales y `static`. Asignación automática.
- **Pila (Stack):** Gestiona la memoria local de las funciones. Cada llamada a función reserva un "marco de pila" para sus parámetros y variables locales. Es de tamaño limitado y su gestión es automática (LIFO: Last-In, First-Out).

- **Montículo (Heap):** Un bloque grande de memoria para asignación manual. Aquí es donde se reserva la memoria dinámica. El programador es responsable de pedir (*new*) y liberar (*dispose*) espacio aquí.

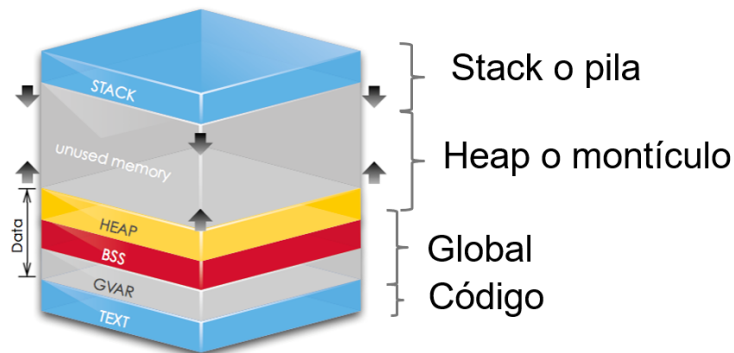


Ilustración 4. Esquema de bloques de memoria de un programa en ejecución

Un ejemplo de código que utiliza la pila sería el siguiente, donde en la declaración de la variable global “precio” se almacenan datos en el segmento global, en la llamada a la función *Calculo Iva* y en el retorno de la misma se utiliza la pila y finalmente en la propia llamada (dentro del *writeln* en el programa principal) se crea una pila para la llamada donde se “deja pendiente” la ejecución del resto del programa principal y se activa la ejecución de la función:

```
PROGRAM Memoria;
VAR
    precio: real; // Se almacena en el segmento global

FUNCTION CalculoIVA(p: real): real; // p y el valor de retorno usan la pila
BEGIN
    CalculoIVA := p * 0.21;
END;

BEGIN
    precio := 200.25;
    writeln('El IVA es: ', CalculoIVA(precio)); // Una pila para la llamada
END.
```

Punteros en Pascal: Concepto y declaración

Un puntero es una variable cuyo valor no es un dato “normal” (como un número o un carácter), sino una dirección de memoria. Es decir, en lugar de almacenar un dato entero, real o booleano, almacena la dirección de memoria de otro dato por lo que se dice que “apunta” a la ubicación donde realmente se almacena el dato de interés.

Se puede declarar de dos formas:

a) Tipo anónimo (directo): `VAR miPuntero: ^integer;`

b) Tipo definido por el usuario (recomendado para claridad):

```
TYPE
    TPunteroEntero = ^integer; // Define un tipo "puntero a entero"
VAR
    pEntero: TPunteroEntero; // Declara una variable de ese tipo
```

donde el símbolo ^ (acento circunflejo) se lee como “puntero a”. El tipo base (en este caso *integer*) indica qué tipo de dato se espera encontrar en la dirección apuntada.

Los punteros son una herramienta fundamental en la construcción de estructuras de datos avanzadas (listas enlazadas, árboles, grafos). Permiten:

- Crear estructuras de tamaño variable en tiempo de ejecución.
- Construir relaciones complejas entre datos (enlaces, nodos).
- Gestionar la memoria de forma eficiente.

Es esencial a la hora de manejarlos el distinguir siempre entre **puntero** (la variable que guarda una dirección) y **puntero^** (el valor almacenado en esa dirección).

Operaciones con punteros

A continuación se describen las operaciones más importantes con punteros en Pascal:

El **operador** @ (referencia) devuelve la dirección de memoria de una variable:

```
VAR
    precioPan: integer;
    pEntero: ^integer;

BEGIN
    precioPan := 85;
    pEntero := @precioPan; // pEntero ahora guarda la dirección de 'precioPan'
```

El **operador** ^ (desreferenciación) accede al valor almacenado en la dirección a la que apunta un puntero:

```
pEntero^ := 100; // Modifica el valor en la dirección apuntada
writeln(precioPan); // Imprime 100, porque precioPan y pEntero^ son sinónimos
```

Los punteros permiten llevar a cabo la gestión de memoria dinámica mediante las instrucciones **new** y **dispose**

La instrucción **new** reserva (asigna) un nuevo bloque de memoria en el *montículo* con el tamaño suficiente para almacenar un valor del tipo base del puntero. Como resultado de su ejecución la variable puntero pasada como argumento queda apuntando a ese nuevo bloque de memoria. Téngase en cuenta que el contenido de este bloque es inicialmente indefinido ("basura").

```
new(pEntero); // Reserva espacio en memoria para un entero en el montículo y
              // pone pEntero a apuntar allí
pEntero^ := 50; // Asignamos un valor a ese espacio
```

Por el contrario, la instrucción **dispose** libera (devuelve al sistema) el bloque de memoria del *heap* que está siendo apuntado por puntero. Es crítico llamar a **dispose** para cada bloque reservado con **new** cuando ya no se necesite, pues no hacerlo causa un efecto indeseado denominado fugas de memoria (*memory leaks*), que consiste en que el montículo se va llenando de memoria inaccesible pero que no se puede reutilizar al estar marcada como en uso, lo cual puede llevar al agotamiento de la memoria y al fallo del programa o del sistema.

Por último, **NIL** es una constante especial que representa un puntero que no apunta a ninguna dirección de memoria válida. Es una práctica excelente asignar **NIL** a un puntero tras liberar su memoria con **dispose**, para evitar su uso accidental (*dangling pointer*).

```
dispose(pEntero);
pEntero := NIL; // Buen hábito: lo marcamos como "no válido"
```

Tema II – Tipos Abstractos de Datos (TAD)

En el capítulo anterior establecimos que la abstracción es el proceso fundamental mediante el cual extraemos las características esenciales de un problema, ignorando los detalles superfluos. Ahora damos un paso crucial hacia la materialización de este concepto en el diseño de software: la definición formal de Tipos Abstractos de Datos (TAD). Si la abstracción forma parte del *pensamiento*, el TAD es su *expresión formal* dentro de la ingeniería de software.

Un TAD representa la cristalización de los principios de abstracción, encapsulamiento e interfaz que ya hemos estudiado. Es la herramienta que nos permite definir nuevos tipos de datos que no existen nativamente en el lenguaje de programación, pero que son esenciales para modelar información que necesitamos para resolver problemas específicos. A través de los TADs, transformamos ideas abstractas —como una cola de espera, un contador o un número complejo— en componentes software reutilizables, robustos y fáciles de mantener.

Este capítulo nos guiará a través de la definición, especificación e implementación de TADs, conectando continuamente estos conceptos con los fundamentos establecidos previamente sobre abstracción, caja negra y contrato.

1. Definición y esencia de los TADs

Un Tipo Abstracto de Dato (TAD) es una descripción de alto nivel de una colección de datos junto con el conjunto completo de operaciones que pueden realizarse sobre ellos. Define la estructura lógica y el comportamiento esperado mediante un contrato bien especificado, pero deliberadamente omite cualquier detalle sobre cómo se implementa internamente.

La esencia del TAD radica en su independencia de la implementación. Podemos concebir un TAD como una "caja negra" perfecta: sabemos exactamente qué funciones ofrece (su interfaz) y qué comportamiento debemos esperar de ellas (su contrato), pero nos es completamente opaco cómo produce internamente esos resultados. Esta separación entre *qué hace* y *cómo lo hace* es precisamente la materialización práctica del principio de abstracción.

Consideremos un TAD Cola. Su especificación nos dice que permite las siguientes operaciones:

- `enqueue(elemento)`: Añade un elemento al final de la cola
- `dequeue()`: Elimina y devuelve el elemento del frente de la cola
- `isEmpty()`: Consulta si la cola no contiene elementos
- `front()`: Consulta cuál es el primer elemento sin eliminarlo
- `back()`: Consulta cuál es el último elemento

Internamente, esta cola podría implementarse de múltiples formas: Usando un array con índices para marcar el frente y el final, empleando una lista enlazada con nodos que apuntan al siguiente elemento, usando dos pilas de manera creativa o empleando un buffer circular para optimizar el uso de memoria.

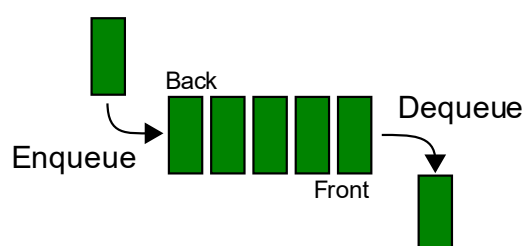


Ilustración 5. El TAD Cola

Lo más reseñable es que, a pesar de que existe tal amplitud en las posibilidades de implementación interna, desde la perspectiva del usuario del TAD todas estas implementaciones son intercambiables. El comportamiento externo es idéntico porque todas respetan el mismo contrato, lo cual ilustra perfectamente el poder del TAD: el programador que utiliza la cola puede concentrarse en *resolver su problema* (gestión de procesos, simulaciones, etc.) sin preocuparse por los detalles de implementación internos a la misma.

Las operaciones de un TAD pueden clasificarse según su propósito, lo que ayuda a diseñar interfaces completas y coherentes:

- Operaciones de Creación (Constructores): Permiten inicializar y obtener nuevos objetos del tipo que corresponde al TAD. En programación orientada a objetos se denominan constructores. Ejemplo: `crearColaVacía()`.
- Operaciones de Modificación (Mutadores): Alteran el estado interno del TAD, cambiando los valores almacenados o las relaciones entre ellos. Ejemplos: `enqueue()`, `dequeue()`.
- Operaciones de Consulta (Observadores): Extraen información del TAD sin modificarlo, lo cual proporciona una "ventana" al estado interno. Ejemplos: `front()`, `isEmpty()`.
- Operaciones propias del TAD: Funciones específicas que tienen sentido solo para ese tipo abstracto particular y que por tanto no son aplicables a cualquier tipo. Ejemplos: el determinante de una matriz, la conjugación de un número complejo, o el cálculo de la distancia entre dos puntos en el espacio.

Esta clasificación guía no solo el diseño del TAD, sino también su documentación y uso, ayudando a los programadores a comprender rápidamente cómo interactuar con el nuevo tipo.

2. Especificación formal

La especificación de un TAD es el proceso mediante el cual definimos formalmente su contrato. Consiste en separar claramente el comportamiento esperado (qué debe hacer) de cualquier implementación concreta (cómo lo hace). Estamos, en esencia, definiendo cómo se comportará un nuevo tipo de datos, estableciendo las reglas del juego antes de escribir la primera línea de código.

Existen diferentes enfoques para especificar TADs, desde el uso de lenguaje natural (accesible pero potencialmente ambiguo) hasta especificaciones algebraicas (rigurosas y formales) o lenguajes de especificación especializados. En este nivel introductorio, nos centraremos en especificaciones en lenguaje natural estructurado, que constituyen un excelente punto de partida.

Una especificación completa en lenguaje natural suele contener los siguientes elementos:

- **Nombre del TAD:** Identificador claro y descriptivo.
- **Valores posibles:** El conjunto de estados válidos que puede tomar una instancia del TAD.
- **Operaciones:** Lista completa de operaciones disponibles, incluyendo para cada una:
 - o Nombre y propósito
 - o Parámetros de entrada y tipo de retorno
 - o Precondiciones (condiciones que deben cumplirse antes de ejecutar la operación)
 - o Postcondiciones (condiciones garantizadas después de ejecutar la operación)
- **Uso de otros tipos:** Dependencias con tipos de datos existentes.

Veamos algunos ejemplos prácticos de especificación.

Ejemplo 1: Semáforo Binario

- **Nombre:** Semáforo
- **Valores posibles:** rojo, verde
- **Operaciones:**
 - o `abrir()`: Procedimiento que cambia el estado interno a verde



- o `cerrar()` : Procedimiento que cambia el estado interno a rojo
 - o `estaAbierto()` : Función que devuelve `true` si el estado es verde, `false` si es rojo
- **Uso de otros tipos:** Se podría implementar usando un `boolean`, un `char`, o incluso un entero, pero esa decisión pertenece a la implementación, no a la especificación.

Ejemplo 2: Semáforo de 3 estados

- **Nombre:** `SemáforoCompleto`
- **Valores posibles:** rojo, ámbar, verde
- **Operaciones:**
 - o `abrir()` : Procedimiento que cambia el estado interno a verde
 - o `cerrar()` : Procedimiento que cambia el estado interno a ámbar, espera 3 segundos, luego cambia a rojo
 - o `estaAbierto()` : Función que devuelve `true` si el estado es verde, `false` en caso contrario
- **Uso de otros tipos:** Como en el ejemplo anterior, se podría implementar usando un `boolean`, un `char`, o incluso un entero, decisión que pertenece a la implementación.



Observa cómo el comportamiento de `cerrar()` se ha hecho más complejo, pero la interfaz pública (los nombres de las operaciones) puede mantenerse idéntica. Esto demuestra cómo podemos evolucionar la implementación sin romper el contrato con los usuarios existentes.

Ejemplo 3: Contador simple

- **Nombre:** `Contador`
- **Valores posibles:** Enteros no negativos (0, 1, 2, ...)
- **Operaciones:**
 - o `inicializar()` : Procedimiento que establece el valor interno a 0
 - o `incrementar()` : Procedimiento que suma 1 al valor interno
 - o `obtenerValor()` : Función que devuelve el valor interno actual
 - o `decrementar()` : Procedimiento que resta 1 al valor interno. Esta operación tiene una precondición: El contador no debe estar en 0
- **Uso de otros tipos:** Enteros para el valor interno.



Ejemplo 4: Contador con Límite

- **Nombre:** `ContadorLimitado`
- **Valores posibles:** Enteros desde 0 hasta un límite máximo L
- **Operaciones:**
 - o `inicializar()` : Procedimiento que establece el valor interno a 0
 - o `establecerLimite(L)` : Procedimiento que define el límite máximo
 - o `incrementar(n)` : Procedimiento que suma n unidades al valor interno. Debe contemplar que, si al incrementar se supera el límite L, el contador vuelve a 0.
 - o `obtenerValor()` : Función que devuelve el valor interno actual
 - o `decrementar()` : Procedimiento que resta 1 al valor interno, teniendo en cuenta la precondición siguiente: El contador no debe estar en 0
- **Uso de otros tipos:** Enteros para el valor y el límite.



Estos ejemplos muestran cómo una especificación clara define completamente el comportamiento esperado, dejando abiertas múltiples posibilidades de implementación. También ilustran cómo podemos extender TADs existentes (de `Contador` a `ContadorLimitado`) manteniendo la compatibilidad cuando ello es deseable.

3. Implementación en Pascal

La implementación de los TADs en un lenguaje de programación conlleva la adaptación a las especificidades del lenguaje a emplear. De los varios conceptos importantes necesarios para implementar un TAD tal y como los hemos descrito, algunos Pascal no los soporta nativamente, requiriendo disciplina por parte del programador:

- **Privacidad (encapsulación):** Pascal distingue entre lo público (accesible desde fuera) y lo privado (detalles internos ocultos). En el nivel más restrictivo, el usuario externo no puede acceder directamente a la representación interna, solo a través de las operaciones públicas, pero Pascal no permite especificar, como Java, distintos niveles de privacidad.
- **Genericidad:** Capacidad de definir TADs parametrizados por tipos. Por ejemplo, una "Cola" genérica que pueda contener cualquier tipo de elemento (enteros, registros, etc.), especificando el tipo concreto al crear una instancia. Pascal estándar no soporta este concepto directamente.
- **Sobrecarga:** Posibilidad de tener múltiples subprogramas con el mismo nombre pero diferentes parámetros. Por ejemplo, `incrementar()` sin parámetros y `incrementar(n)` con un parámetro. Pascal no permite sobrecarga de funciones/procedimientos.

Dado que Pascal carece de soporte nativo para estos conceptos avanzados, la disciplina del programador será esencial para respetarlos. Así, debemos utilizar el TAD exclusivamente a través de su interfaz pública, nunca accediendo directamente a sus datos internos, aunque el lenguaje técnicamente lo permita.

Para implementar TADs en Pascal utilizamos unidades (UNITs), que es el mecanismo del lenguaje para llevar a cabo la modularización y encapsulación de TADs. Una unidad en Pascal consta de dos partes claramente separadas:

- Interfaz Pública (Sección `INTERFACE`): Declara todo lo que será visible y accesible desde programas externos. Aquí definimos los tipos públicos, constantes, variables, y las cabeceras de los subprogramas que componen la interfaz del TAD.
- Implementación Privada (Sección `IMPLEMENTATION`): Contiene el código real de los subprogramas declarados en la interfaz, además de posibles tipos, constantes y variables auxiliares que son privados (no accesibles desde fuera).

El nombre de la unidad y el nombre del archivo que la contiene deben coincidir. Por ejemplo, la unidad `uContador` se guardaría en `uContador.pas`.

Veamos a continuación un ejemplo para un TAD sencillo: el contador. Su interfaz pública estaría almacenada en un archivo denominado `uContador.pas` y su contenido sería el siguiente:

```
unit ucontador;
interface
type
    contador = ^tcontador; // tipo opaco: el usuario solo ve un puntero
    tcontador = record      // definición interna (invisible externamente)
        valor: integer;
    end;

// operaciones públicas del tad
procedure crear(var c: contador);
procedure incrementar(var c: contador);
function obtenervalor(c: contador): integer;
procedure destruir(var c: contador);
```

En cuanto a su implementación, que sería privada y por tanto no accesible a los usuarios del TAD, esta iría en el mismo archivo (`uContador.pas`) e inmediatamente a continuación de la interfaz. Sería la siguiente:

```

implementation

procedure crear(var c: contador);
begin
    new(c);           // reserva memoria dinámica
    c^.valor := 0;     // inicializa el valor interno
end;

procedure incrementar(var c: contador);
begin
    c^.valor := c^.valor + 1;
end;

function obtenervalor(c: contador): integer;
begin
    obtenervalor := c^.valor;
end;

procedure destruir(var c: contador);
begin
    dispose(c);       // libera la memoria dinámica
    c := nil;          // buen hábito: evitar punteros colgantes
end;

end.

```

Finalmente, el uso del TAD desde un programa Pascal sería algo como esto:

```

program pruebacontador;
uses ucontador; // incluye la unidad con el tad
var
    micon contador;
begin
    // usamos solo las operaciones públicas
    crear(micon);
    incrementar(micon);
    incrementar(micon);
    writeln('valor actual: ', obtenervalor(micon)); // muestra 2
    destruir(micon);
end.

```

4. Beneficios del uso de TADs

La adopción de TADs en el desarrollo de software produce beneficios sustanciales que impactan en todas las fases del ciclo de vida del software.

La primera es la **separación de responsabilidades**. El uso de TADs promueve una clara separación entre los siguientes roles:

- El programador del TAD, que se concentra en implementar correctamente las operaciones especificadas, optimizándolas para maximizar su eficiencia, robustez, etc.
- El programador cliente, que utiliza el TAD a través de su interfaz pública, concentrándose en la lógica de negocio de su aplicación sin preocuparse por los detalles internos del TAD.

Otro beneficio tangible es la **facilidad de mantenimiento**. Esta propiedad mejora porque cuando la implementación de un TAD necesita cambios (por optimización, corrección de errores, o nuevas características), el código cliente no necesita modificarse siempre que se mantenga inalterada la interfaz pública. Esto reduce mucho el riesgo de introducir errores al modificar sistemas complejos.

Por otra parte, se potencia la reutilización y la fiabilidad de los componentes software, algo muy deseable desde la perspectiva de la ingeniería del software. La existencia de TADs bien diseñados y probados que se convierten en **componentes reutilizables** y que pueden integrarse en múltiples proyectos potencia la construcción de sistemas complejos a partir de componentes ya existentes (en su mayoría, al menos). Dado que su interfaz es estable y su comportamiento está especificado formalmente, podemos confiar en ellos como "cajas negras" certificadas.

Por último, decir que el código que utiliza TADs resulta **más legible** y expresivo. En lugar de operaciones de bajo nivel sobre estructuras de datos primitivas, vemos operaciones de alto nivel que reflejan directamente el dominio del problema. Compara los dos códigos siguientes:

Sin TAD (bajo nivel)	Con TAD (alto nivel)
<pre>arrayCola[final] := elemento; final := final + 1;</pre>	<pre>encolar(miCola, elemento);</pre>

Como se aprecia, la versión con TAD es inmediatamente comprensible, mientras que la versión sin TAD requiere entender la representación interna específica.

Y finalmente, podemos citar el beneficio que supone la **abstracción progresiva**, ya que en el fondo la construcción de programas complejos se reduce a interconectar módulos TADs existentes y escribir una cantidad mínima de código "pegamento" específico del problema. Esta aproximación modular es escalable y gestionable incluso para sistemas de gran envergadura.

Resumen

Los Tipos Abstractos de Datos (TADs) representan la materialización práctica de los principios de abstracción estudiados en el capítulo anterior. A través de ellos, transformamos conceptos del dominio del problema en componentes software reutilizables, encapsulados y bien definidos.

La especificación formal (el "contrato") establece el comportamiento esperado, mientras que la implementación concreta (usando unidades en Pascal, clases en POO, etc.) proporciona la realización eficiente de ese comportamiento. Esta separación es poderosa: nos permite razonar sobre corrección a nivel abstracto, cambiar implementaciones para optimizar rendimiento, y construir sistemas complejos mediante la composición de componentes probados y fiables.

Al dominar el diseño e implementación de TADs, adquirimos una habilidad fundamental para cualquier ingeniero de software: la capacidad de crear abstracciones efectivas que capturen lo esencial de un problema, ignorando lo accesorio. Esta habilidad trascenderá el uso de Pascal o cualquier lenguaje específico, constituyendo una piedra angular del pensamiento computacional y el diseño de sistemas robustos y fáciles de mantener.

En los próximos capítulos, aplicaremos estos conceptos para construir estructuras de datos clásicas —pilas, colas, listas, árboles— cada una implementada como un TAD con su interfaz bien definida y múltiples posibles implementaciones. Los TADs serán nuestro marco unificador para entender, comparar y utilizar estas estructuras fundamentales.

Tema III – Análisis de la complejidad de los algoritmos

En los capítulos anteriores hemos establecido los fundamentos conceptuales para diseñar y especificar Tipos Abstractos de Datos. Ahora, al comenzar a implementar algoritmos y estructuras de datos concretas, surge una pregunta fundamental: ¿cómo evaluamos y comparamos diferentes soluciones al mismo problema? La respuesta nos lleva al estudio de la complejidad algorítmica.

La complejidad de un algoritmo mide su eficiencia para resolver un problema determinado, considerando dos recursos críticos: el tiempo de ejecución y el espacio de memoria requerido. Estas mediciones no son absolutas (no se expresan en segundos o megabytes concretos), sino relativas al tamaño del problema y, lo que es más importante, independientes de factores externos como la velocidad del procesador, la plataforma de ejecución o el lenguaje de programación utilizado.

Consideremos un ejemplo práctico: si necesitamos buscar un elemento en una colección de datos, podríamos optar por una búsqueda lineal (revisando elemento por elemento) o una búsqueda binaria (dividiendo repetidamente el espacio de búsqueda). Intuitivamente, sabemos que la búsqueda binaria es "más rápida", pero ¿cómo cuantificamos esta diferencia? ¿Cómo determinamos cuándo un algoritmo es lo suficientemente eficiente para un problema de cierto tamaño? El análisis de complejidad nos proporciona las herramientas matemáticas para responder estas preguntas de manera rigurosa y objetiva.

Este capítulo nos introducirá en las técnicas para analizar y comparar algoritmos, centrándonos especialmente en la notación O grande (Big-O), el lenguaje universal para describir cómo escala el tiempo de ejecución de un algoritmo a medida que crece el tamaño de la entrada.

1. Conceptos Fundamentales

Como ya hemos dicho, la complejidad de un algoritmo mide su eficiencia para resolver un problema, lo cual puede ser analizado desde dos perspectivas diferentes: cuánto tiempo tarda en ejecutarse y cuánto espacio de memoria necesita para ello.

Complejidad Temporal

La complejidad temporal de un algoritmo se refiere a cómo crece el tiempo de ejecución en función del tamaño de los datos de entrada, denotado típicamente como n . En lugar de medir tiempo real en segundos (que variaría según el hardware y otros factores), contamos el número de operaciones básicas que realiza el algoritmo. Estas operaciones básicas incluyen asignaciones, comparaciones, sumas, multiplicaciones, accesos a memoria, etc., asumiendo que cada una requiere un tiempo aproximadamente constante.

Supongamos que tenemos un array ordenado de 10 elementos y queremos determinar si contiene un valor específico. Una búsqueda lineal, en el peor caso (el elemento buscado está al final o no está presente), nos obliga a examinar los 10 elementos. Decimos pues que realiza n operaciones de comparación, donde $n=10$. Por el contrario, una búsqueda binaria compara el elemento buscado con el elemento central, descarta la mitad de los elementos, y repite estos pasos hasta encontrar el elemento o darse cuenta de que no se halla en el array. En nuestro ejemplo, necesitamos como máximo 4 comparaciones pues matemáticamente, el número máximo de comparaciones que hace este algoritmo para $n=10$ es aproximadamente $\log_2(n)$, donde en este caso $\log_2(10) \approx 4$.

La diferencia es muy importante cuando tratamos con colecciones grandes: para $n=1.000.000$, la búsqueda lineal podría requerir un millón de comparaciones, mientras que la búsqueda binaria solo

requiere aproximadamente 20. Esta diferencia cuantitativa ilustra la importancia de analizar la complejidad de los algoritmos como paso previo a seleccionar el idóneo en cada caso.

El caso más desfavorable (Worst-Case Analysis)

Cuando analizamos algoritmos, consideramos siempre el peor escenario posible. Esto puede parecer pesimista, pero proporciona una garantía: no importa qué entrada específica recibamos, el algoritmo nunca será peor que aquello que nuestro análisis predice. Esta aproximación es crucial para sistemas donde el rendimiento predecible es esencial (sistemas de tiempo real, aplicaciones críticas, etc.).

No obstante, existen otros tipos de análisis:

- Caso promedio: Considera la distribución típica de entradas.
- Mejor caso: Considera la entrada más favorable.

Sin embargo, el análisis del peor caso es el más común en la práctica porque proporciona una cota superior garantizada, evita tener que hacer suposiciones sobre la distribución de las entradas y desde luego, es más fácil de calcular que el caso promedio en la mayoría de algoritmos.

Complejidad espacial

La complejidad espacial mide la cantidad de memoria adicional que necesita un algoritmo, más allá de la memoria ocupada por la entrada misma. Se expresa también en función de n (tamaño de la entrada) e incluye la memoria para variables locales, las estructuras de datos auxiliares y la pila de llamadas en algoritmos recursivos.

Aunque en este texto nos centraremos principalmente en la complejidad temporal, es importante recordar que tiempo y espacio son recursos intercambiables en el diseño de algoritmos. A veces podemos hacer un algoritmo más rápido usando más memoria (*caching*, tablas de búsqueda), o podemos ahorrar memoria a costa de un tiempo de ejecución mayor. Este equilibrio se conoce como *trade-off* (solución de compromiso) entre tiempo y espacio.

2. Notación O grande (Big-O)

La notación O grande (Big-O) es la notación utilizada comunmente para describir cómo crece el tiempo de ejecución de un algoritmo a medida que crece el tamaño de la entrada. Formalmente, decimos que una función $f(n)$ es $O(g(n))$ si existen constantes positivas c y n_0 tales que:

$$f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0$$

En términos más intuitivos, $O(g(n))$ establece una cota superior asintótica para la función que cuenta las operaciones de nuestro algoritmo. Nos dice: "Más allá de cierto tamaño de problema, el tiempo de ejecución crecerá como mucho tan rápido como $g(n)$, multiplicado por alguna constante".

Cuando utilizamos esta notación ignoramos las constantes pero, ¿Por qué? En notación Big-O, simplificamos $O(3n^2 + 2n + 100)$ a $O(n^2)$. Esto es así porque para valores grandes de n , el término n^2 domina completamente el comportamiento. Las constantes (3, 2, 100) y los términos de menor orden ($2n$) se vuelven insignificantes comparados con n^2 cuando n es suficientemente grande.

Las complejidades algorítmicas se organizan en una jerarquía clara, desde las más eficientes hasta las menos escalables:

- **O(1) - Complejidad Constante:** El tiempo de ejecución no depende del tamaño de la entrada. Es el caso ideal. Ejemplos: acceder a un elemento de un array por índice, realizar una operación aritmética simple.

- **$O(\log n)$ - Complejidad Logarítmica:** El tiempo crece logarítmicamente con n . Extremadamente eficiente para problemas grandes. Ejemplos: búsqueda binaria, operaciones en árboles binarios balanceados.
- **$O(n)$ - Complejidad Lineal:** El tiempo es proporcional al tamaño de la entrada. Considerada "aceptable" para muchos problemas prácticos. Ejemplos: recorrer un array o lista, búsqueda lineal.
- **$O(n \log n)$ - Complejidad Cuasi-lineal:** Ligeramente peor que lineal pero mucho mejor que cuadrática. Aparece en algoritmos óptimos de ordenación como Merge Sort y Quick Sort.

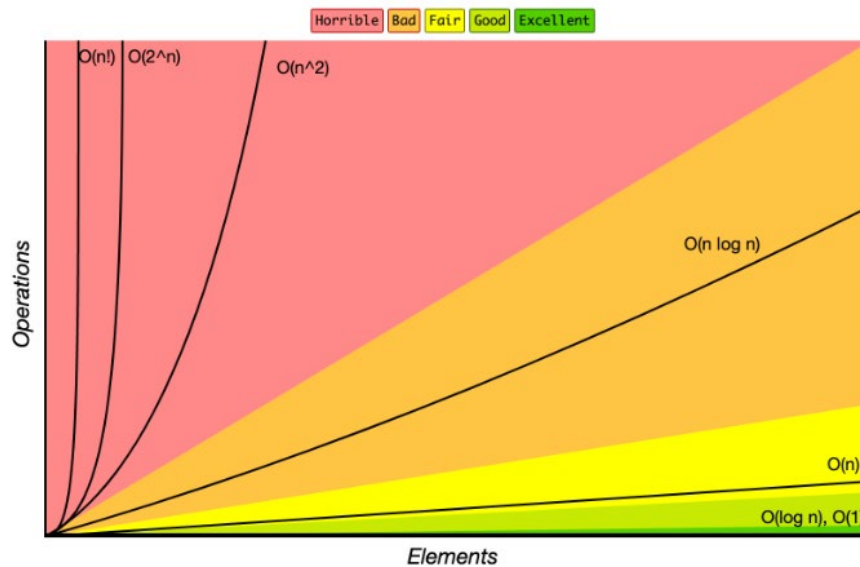


Ilustración 6. Jerarquía de complejidades algorítmicas en notación O grande

- **$O(n^2)$ - Complejidad Cuadrática:** El tiempo es proporcional al cuadrado del tamaño. Se vuelve problemático para entradas moderadamente grandes. Ejemplos: algoritmos de ordenación simples (Bubble Sort, Insertion Sort), procesamiento de matrices con bucles anidados.
- **$O(2^n)$ - Complejidad Exponencial:** El tiempo se duplica con cada incremento en el tamaño de entrada. Prácticamente intratable excepto para entradas muy pequeñas. Ejemplos: algunos algoritmos de fuerza bruta para problemas NP-completos.
- **$O(n!)$ - Complejidad Factorial:** Aún peor que exponencial. Solo manejable para entradas muy limitadas. Ejemplo: algoritmo de fuerza bruta para el problema del viajante.

Para apreciar las diferencias, consideremos cuántas operaciones requiere cada complejidad para diferentes tamaños de problema:

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$	$O(n!)$
10	1	~3	10	~30	100	1,024	3,628,800
100	1	~7	100	~700	10,000	1.27×10^{30}	9.33×10^{157}
1000	1	~10	1,000	~10,000	1,000,000	1.07×10^{301}	4.02×10^{2567}

Como se observa, las diferencias pasan a ser astronómicas para tamaños de n moderadamente grandes. Un algoritmo $O(n!)$ para $n=100$ es completamente impracticable, mientras que un algoritmo $O(n \log n)$ para el mismo n es perfectamente manejable. Resumiendo, podemos decir que, en la práctica, los algoritmos con complejidad $O(n^3)$ o peor rara vez son útiles para problemas de tamaño significativo, mientras que los algoritmos $O(n \log n)$ o mejores son los que buscamos para problemas de tamaño considerable.

3. Reglas para calcular complejidades

Se consideran **operaciones simples** cuya complejidad es $O(1)$ las asignaciones, las expresiones aritméticas, los accesos a elementos de arrays, la lectura/escritura de datos simples, y las llamadas a funciones que ejecutan en tiempo constante.

En el cálculo de complejidad hay que ignorar las constantes multiplicativas. Así, consideraremos que $O(5n) = O(n)$, $O(n/3) = O(n)$, $O(1000) = O(1)$, ya que las constantes no afectan el comportamiento asintótico.

Hay dos reglas más que debemos conocer: la de la suma y la del producto.

Regla de la suma

Cuando nos encontremos ante una secuencia de instrucciones, habremos de seguir la **Regla de la Suma** que se enuncia del modo siguiente:

“Si un algoritmo ejecuta el bloque A seguido del bloque B, la complejidad total es $O(\max(f(n), g(n)))$, donde $f(n)$ y $g(n)$ son las complejidades de A y B respectivamente. Solo importa el término dominante (el de mayor crecimiento).”

Si nos encontramos ante **estructuras selectivas** (if/else) tomaremos la máxima complejidad entre todas las ramas posibles.

Regla del producto

Si estamos analizando bucles, aplicaremos la **Regla del Producto**:

“Si un bucle de complejidad $O(f(n))$ contiene en su interior código de complejidad $O(g(n))$, la complejidad total es $O(f(n) \times g(n))$.”

Ejemplo:

```
for i := 1 to n do
  instrucción_simple { O(1) }
```

La complejidad de este bucle será: $n \times O(1) = O(n)$

Otro ejemplo:

```
for i := 1 to 100 do { 100 es constante, no depende de n }
  instrucción_simple { O(1) }
```

La complejidad de este bucle es: $100 \times O(1) = O(1)$

Un último ejemplo, con bucles anidados, sería:

```
for i := 1 to n do { O(n) }
  for j := 1 to n do { O(n) }
    instrucción_simple { O(1) }
```

Aquí la complejidad será: $O(n) \times O(n) \times O(1) = O(n^2)$ ya que los bucles son independientes. Si fuesen dependientes, como en el siguiente caso:

```
for i := 1 to n do { O(n) }
  for j := 1 to i do { O(i) que en promedio es O(n/2) }
    instrucción_simple { O(1) }
```

entonces la complejidad total debería tener en cuenta que el bucle interno ejecuta $1 + 2 + 3 + \dots + n = n(n+1)/2$ veces, por lo que su complejidad es: $O(n(n+1)/2) = O(n^2/2 + n/2) = O(n^2)$

Análisis de algoritmos recursivos

Para algoritmos recursivos, la complejidad se determina mediante ecuaciones de recurrencia. Consideremos dos ejemplos. El primero analiza el cálculo del Factorial recursivo:

```
function Factorial(n: integer): integer;
begin
  if n = 0 then
    Factorial := 1           { caso base: O(1) }
  else
    Factorial := n * Factorial(n-1); { llamada recursiva }
  end;
```

La ecuación de recurrencia para este algoritmo sería: $T(n) = T(n-1) + O(1)$

Para determinar la complejidad, desarrollamos y tenemos:

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= [T(n-2) + 1] + 1 = T(n-2) + 2 \\ &= [T(n-3) + 1] + 2 = T(n-3) + 3 \\ &\dots \\ &= T(0) + n = 1 + n \end{aligned}$$

Es decir, la complejidad del algoritmo es: $O(n)$

El segundo ejemplo analizaría el algoritmo de Búsqueda Binaria Recursiva:

```
function BusquedaBinaria(arr: array; inicio, fin, objetivo: integer): boolean;
var
  medio: integer;
begin
  if inicio > fin then
    BusquedaBinaria := false           { O(1) }
  else begin
    medio := (inicio + fin) div 2;      { O(1) }
    if arr[medio] = objetivo then
      BusquedaBinaria := true          { O(1) }
    else if arr[medio] > objetivo then
      BusquedaBinaria := BusquedaBinaria(arr, inicio, medio-1, objetivo)
    else
      BusquedaBinaria := BusquedaBinaria(arr, medio+1, fin, objetivo);
  end;
end;
```

Aquí la ecuación de recurrencia es: $T(n) = T(n/2) + O(1)$ y puesto que, en cada llamada, el problema se reduce a la mitad su complejidad es $O(\log n)$:

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &= [T(n/4) + 1] + 1 = T(n/4) + 2 \\ &= [T(n/8) + 1] + 2 = T(n/8) + 3 \\ &\dots \\ &= T(1) + \log_2(n) \approx 1 + \log_2(n) \end{aligned}$$

4. Implicaciones en el diseño de estructuras de datos

El análisis de complejidad no es solo un ejercicio teórico; tiene consecuencias directas en cómo diseñamos y seleccionamos las estructuras de datos que emplearemos en nuestros programas. Cada operación fundamental (inserción, búsqueda, eliminación) tiene una complejidad característica dependiendo de la estructura elegida:

Operación	Array	Lista Enlazada	Árbol Binario Balanceado	Tabla Hash (promedio)
Búsqueda	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$
Inserción	$O(n)^*$	$O(1)^{**}$	$O(\log n)$	$O(1)$
Eliminación	$O(n)^*$	$O(1)^{**}$	$O(\log n)$	$O(1)$

*En arrays: $O(n)$ si hay que desplazar elementos; $O(1)$ si es al final y hay espacio.

**En listas enlazadas: $O(1)$ si tenemos referencia al nodo; $O(n)$ si debemos buscarlo primero.

Esta tabla explica por qué no existe una estructura de datos perfecta para todos los casos: según las operaciones que más frecuentemente realicemos, elegiremos una estructura diferente. Por ejemplo, si necesitamos muchas búsquedas: tabla hash o árbol balanceado. Pero si necesitamos inserción/eliminación frecuentes en posiciones aleatorias utilizaremos una lista enlazada, mientras que si necesitamos acceso aleatorio por índice lo óptimo es utilizar un array.

Optimización prematura vs. análisis informado

Un principio importante en ingeniería del software es evitar la optimización prematura. No debemos gastar tiempo optimizando algoritmos sin antes:

- Medir el rendimiento real del sistema
- Identificar los cuellos de botella (usando profilers)
- Confirmar que la optimización mejorará significativamente el rendimiento

Sin embargo, el análisis de complejidad no es optimización prematura, sino diseño informado, ya que nos permite seleccionar algoritmos apropiados desde el inicio, predecir cómo se comportará nuestro sistema con datos más grandes y evitar diseños que serán insostenibles al escalar.

Como ejemplo, pensemos en la selección del mejor algoritmo de ordenación para un cierto caso. Supongamos que debemos ordenar una colección de datos y para ello tenemos varias opciones:

- Método de la burbuja: $O(n^2)$ - Simple pero ineficiente para $n > 1000$
- Método de inserción: $O(n^2)$ - Eficiente para arreglos casi ordenados o pequeños ($n < 50$)
- Quick Sort: $O(n \log n)$ promedio, $O(n^2)$ peor caso - Muy eficiente en la práctica
- Merge Sort: $O(n \log n)$ garantizado - Estable pero requiere memoria adicional

Mediante una decisión informada podremos establecer criterios de selección como:

- Si $n \leq 50$: Insertion Sort (constantes pequeñas, código simple)
- Si $50 < n \leq 10,000$: Quick Sort (rápido en promedio)
- Si $n > 10,000$ y necesitamos garantías: Merge Sort
- Y si los datos ya están casi ordenados podemos optar por Inserción.

En resumen, esta decisión se basa en comprender no solo las complejidades asintóticas, sino también las constantes ocultas y las características específicas del problema. Esto demuestra que el análisis de complejidad es mucho más que una técnica matemática; es una herramienta fundamental de diseño que todo ingeniero de software debe dominar, ya que nos permite predecir el comportamiento de nuestros algoritmos ante entradas grandes, comparar objetivamente diferentes soluciones al mismo problema y seleccionar las estructuras de datos más apropiadas para cada contexto.

No obstante, debemos tener en cuenta que la complejidad algorítmica es solo una dimensión de la calidad del software. Un algoritmo $O(n \log n)$ con código ilegible y lleno de bugs es peor que un algoritmo $O(n^2)$ correcto y bien documentado. La complejidad debe equilibrarse con otros factores como claridad, robustez y mantenibilidad. El ingeniero de software competente deberá saber encontrar el equilibrio óptimo entre eficiencia y otras cualidades del software.

Tema IV – Pilas

Las estructuras de datos que estudiaremos a partir de este capítulo son la materialización práctica de los conceptos teóricos fundamentales establecidos anteriormente: abstracción, especificación de TADs, análisis de complejidad y gestión de memoria dinámica. La pila (*stack*) representa una de las estructuras de datos lineales más elementales y, simultáneamente, más ubicuas en la computación.

Una pila es una estructura de datos lineal y dinámica que implementa el principio LIFO (*Last In, First Out*), que en español se traduce como "último en entrar, primero en salir". Esta analogía con una pila de objetos físicos (como una pila de platos o una pila de libros) es perfectamente ilustrativa: solo podemos añadir o quitar elementos por la parte superior, y el último elemento añadido será siempre el primero en ser retirado.

La importancia de las pilas trasciende su simplicidad conceptual. Constituyen un componente fundamental en múltiples aspectos de los sistemas computacionales: desde la gestión de llamadas a subprogramas (el *stack* de ejecución) hasta algoritmos de procesamiento de lenguajes (evaluación de expresiones, análisis sintáctico) y características de interfaz de usuario (historial de navegación, deshacer/rehacer). Este capítulo nos guiará a través de la especificación, implementación y aplicaciones prácticas de esta estructura fundamental.

1. El TAD Pila

Independientemente de cómo se implemente internamente, un Tipo Abstracto de Datos Pila debe proporcionar un conjunto mínimo de operaciones que respeten el contrato LIFO. La especificación en lenguaje natural sería:

- **Nombre:** Pila
- **Valores posibles:** Vacía o conteniendo una secuencia de elementos del tipo base, donde solo el elemento más reciente (la cima) es directamente accesible.
- **Operaciones básicas:**
 - o `initialize(p)` o `crear()`: Inicializa una pila vacía.
 - o `push(p, x)`: Inserta el elemento `x` en la cima de la pila `p`.
 - o `pop(p)`: Elimina el elemento situado en la cima de la pila `p`.
 - o `peek(p)` o `top(p)`: Consulta (sin eliminar) el elemento situado en la cima de la pila `p`.
 - o `isEmpty(p)`: Determina si la pila `p` no contiene elementos.
- **Uso de otros tipos:** Depende del tipo base de elementos que la pila contendrá (enteros, caracteres, registros, etc.) y de las estructuras auxiliares utilizadas en la implementación (arrays, punteros).

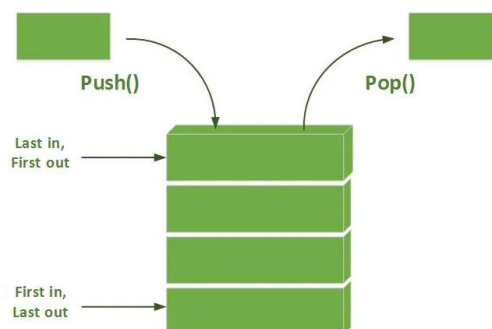


Ilustración 7. Operaciones básicas de un TAD pila

Algunas operaciones incluyen precondiciones que garantizan su correcto funcionamiento:

- **push(p, x)**: Añade x a la cima de p . *Precondición ideal*: La pila no debe estar llena (en implementaciones con capacidad limitada).
- **pop(p)**: Elimina el elemento de la cima de p . *Precondición obligatoria*: La pila no debe estar vacía ($!isEmpty(p)$). Intentar hacer `pop` en una pila vacía se conoce como *underflow* y es un error.
- **peek(p)**: Devuelve el elemento de la cima sin modificarlo. *Precondición obligatoria*: La pila no debe estar vacía ($!isEmpty(p)$).
- **isEmpty(p)**: Devuelve `true` si p no contiene elementos, `false` en caso contrario. No tiene precondiciones.

La gestión de estas precondiciones es responsabilidad tanto del implementador (que debe verificar condiciones cuando sea posible) como del usuario (que debe usar el TAD correctamente). En lenguajes modernos, violaciones de precondiciones suelen manejarse con excepciones, aunque Pascal carece de este mecanismo.

Además de las operaciones básicas, es común extender el TAD Pila con funcionalidades adicionales que, sin ser estrictamente necesarias, aumentan su utilidad práctica:

- **isFull(p)**: Determina si la pila ha alcanzado su capacidad máxima (en implementaciones con arrays de tamaño fijo).
- **clear(p)**: Vacía completamente la pila, dejándola en el mismo estado que después de `initialize`.
- **size(p)** o **count(p)**: Devuelve el número de elementos actualmente en la pila.
- **npop(p, n)**: Elimina n elementos de la cima de una sola vez (útil en ciertos algoritmos).

Estas operaciones pueden implementarse en términos de las operaciones básicas o, para mayor eficiencia, mediante acceso directo a la representación interna cuando se justifique.

2. Implementación de una pila mediante un array

La implementación de una pila utilizando un array como almacenamiento subyacente es conceptualmente sencilla y eficiente en términos de velocidad de acceso. Para ello necesitamos dos componentes fundamentales: un array de tamaño fijo para almacenar los elementos y un índice entero (típicamente llamado *cima* o *top*) que indica la posición del elemento superior en el array. . El tipo de los elementos del array corresponderá al tipo base de la pila, y en cuanto al índice, lo habitual es que cuando la pila está vacía, este índice tenga el valor 0.

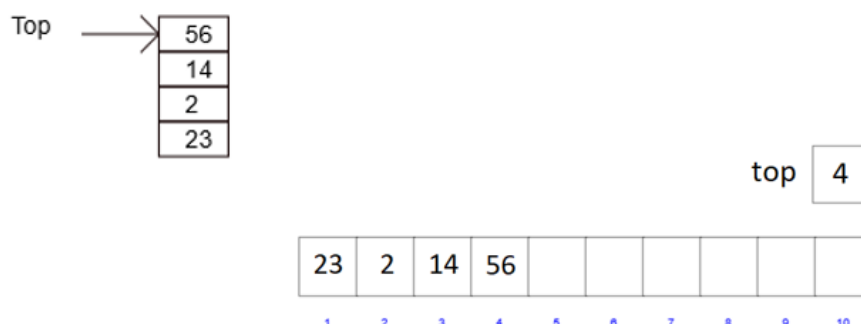


Ilustración 8. Implementación de un TAD pila mediante arrays

Tomaremos una serie de decisiones de diseño importantes, como por ejemplo definir una constante `MAX_ELEMENTOS` que establece la capacidad máxima de la pila. Además, al "eliminar" un elemento

(pop), no borraremos físicamente el valor del array; simplemente decrementamos el índice `cima`. El valor antiguo permanece en memoria pero se considera no válido.

En cuanto a la complejidad, esta implementación hace que las operaciones `push` y `pop` tengan complejidad temporal $O(1)$ (tiempo constante), por lo que son muy eficientes.

Si utilizamos como tipo base los enteros, la implementación en Pascal sería algo así:

```
unit uPilaEnteros;

interface

const
    MAX_ELEMENTOS = 100;
type
    tPilaEnteros = record
        cima: integer;
        stack: array[1..MAX_ELEMENTOS] of integer;
    end;
procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;
function isFull(p: tPilaEnteros): boolean; { Operación extendida }
procedure clear(var p: tPilaEnteros);      { Operación extendida }

implementation

procedure initialize(var p: tPilaEnteros);
begin
    p.cima := 0; { Pila vacía }
end;

function isEmpty(p: tPilaEnteros): boolean;
begin
    isEmpty := (p.cima = 0);
end;

function isFull(p: tPilaEnteros): boolean;
begin
    isFull := (p.cima = MAX_ELEMENTOS);
end;

procedure push(var p: tPilaEnteros; x: integer);
begin
    if not isFull(p) then
    begin
        p.cima := p.cima + 1;
        p.stack[p.cima] := x;
    end
    else
    { Manejo de error: pila llena (overflow) }
        writeln('Error: Pila llena - no se puede hacer push');
    end;

procedure pop(var p: tPilaEnteros);
begin
    if not isEmpty(p) then
        p.cima := p.cima - 1
    else
        writeln('Error: Pila vacía - no se puede hacer pop');
    end;

function peek(p: tPilaEnteros): integer;
```

```

begin
  if not isEmpty(p) then
    peek := p.stack[p.cima]
  else
    begin
      writeln('Error: Pila vacía - no se puede hacer peek');
      peek := 0; { Valor por defecto en caso de error }
    end;
  end;

  procedure clear(var p: tPilaEnteros);
  begin
    p.cima := 0; { Vacía la pila eficientemente }
  end;

end.

```

La implementación con arrays presenta ventajas e inconvenientes claros. En cuanto a las ventajas, podemos citar las siguientes:

- **Simplicidad:** El código es fácil de entender y depurar.
- **Eficiencia:** Acceso directo a memoria contigua, buen comportamiento de caché.
- **Predictibilidad:** Uso de memoria conocido y constante.

Por otra parte, tiene un buen número de desventajas:

- **Tamaño fijo:** Se debe conocer o estimar un tamaño máximo `MAX_ELEMENTOS` en tiempo de compilación. Si se subestima `MAX_ELEMENTOS`, la pila se llenará; si se sobreestima, se desperdicia memoria.
- **Gestión de errores rudimentaria:** Pascal no tiene excepciones, por lo que debemos manejar los errores (overflow/underflow) de manera rudimentaria y poco robusta poniendo mensajes por consola.
- **Nota sobre precondiciones:** En la implementación mostrada, verificamos las precondiciones (`isFull` antes de `push`, `isEmpty` antes de `pop/peek`) e imprimimos mensajes de error. En un contexto real, podríamos utilizar códigos de error, variables globales de estado, o (en lenguajes que lo soporten) lanzar excepciones.

3. Implementación con memoria dinámica

La implementación dinámica de una pila utiliza una estructura de **nodos enlazados**, donde cada nodo contiene el dato (del tipo base de la pila) y un puntero al nodo anterior (el que está "debajo" en la pila). La pila en sí se representa mediante un puntero al nodo de la cima (o `nil` si la pila está vacía). Esta representación elimina la limitación de tamaño fijo: la pila puede crecer hasta agotar la memoria disponible del sistema.

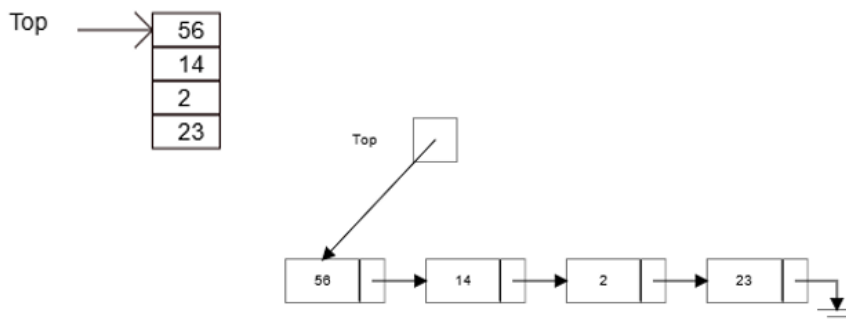


Ilustración 9. Implementación de un TAD pila mediante memoria dinámica

Las características clave de esta implementación son que no hay límite máximo predefinido (excepto la memoria disponible). Cada operación `push` requiere asignación dinámica (`new`) y cada operación `pop` requiere liberación de memoria (`dispose`). Las operaciones siguen siendo $O(1)$ en complejidad temporal. Con respecto a la implementación mediante arrays, aquí es necesaria más memoria por elemento, pues necesita espacio para el puntero, pero es algo poco relevante.

Su implementación en Pascal sería algo así:

```
unit uPilaEnterosDinamica;

interface

type
  { Tipo nodo: representa un elemento de la pila }
  tNodo = record
    info: integer;
    anterior: ^tNodo; { Puntero al nodo que está "debajo" }
  end;

  { La pila es un puntero al nodo de la cima }
  tPilaEnteros = ^tNodo;

procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;

implementation

procedure initialize(var p: tPilaEnteros);
begin
  p := nil; { Pila vacía }
end;

function isEmpty(p: tPilaEnteros): boolean;
begin
  isEmpty := (p = nil);
end;

procedure push(var p: tPilaEnteros; x: integer);
var
  nuevo: ^tNodo;
begin
  new(nuevo);
  nuevo^.info := x;
  nuevo^.anterior := p;
  p := nuevo;
end;

procedure pop(var p: tPilaEnteros);
var
  aux: ^tNodo;
begin
  if not isEmpty(p) then
  begin
    aux := p;
    p := p^.anterior;
    dispose(aux);
  end
  else
    writeln('Error: Pila vacía - no se puede hacer pop');
  end;
end;

function peek(p: tPilaEnteros): integer;
```

```

begin
  if not isEmpty(p) then
    peek := p^.info
  else
    begin
      writeln('Error: Pila vacía - no se puede hacer peek');
      peek := 0; { Valor por defecto }
    end;
  end;
end;

end.

```

4. Aplicaciones prácticas

Las pilas encuentran aplicación en numerosos dominios de la computación, demostrando que una estructura simple puede resolver problemas complejos cuando se aplica adecuadamente.

El uso más fundamental de las pilas en los sistemas computacionales es la pila de ejecución (*call stack*). Cada vez que se llama a una función o procedimiento se "apila" un marco de activación (*activation record*) que contiene: parámetros de la función, variables locales, dirección de retorno (dónde continuar después de la llamada) y otros datos de control. Cuando la función termina, su marco se "desapila" y se restaura el contexto de la función llamadora. Este mecanismo permite la recursividad (una función que se llama a sí misma) y gestiona automáticamente el ciclo de vida de las variables locales.

Las pilas son esenciales para evaluar expresiones aritméticas, especialmente en notación postfija (también llamada notación polaca inversa). El algoritmo es sencillo:

1. Recorrer la expresión de izquierda a derecha.
2. Si encontramos un operando (número), lo apilamos.
3. Si encontramos un operador (+, -, *, /), desapilamos los dos operandos superiores, aplicamos la operación, y apilamos el resultado.
4. Al final, el resultado estará en la cima de la pila.

Ejemplo: Evaluar "3 4 + 5 * " (equivalente a $(3+4) * 5$)

Paso 1: Apilar 3 → Pila: [3]

Paso 2: Apilar 4 → Pila: [3, 4]

Paso 3: Encontrar + → Desapilar 4 y 3, calcular $3+4=7$, apilar 7 → Pila: [7]

Paso 4: Apilar 5 → Pila: [7, 5]

Paso 5: Encontrar * → Desapilar 5 y 7, calcular $7*5=35$, apilar 35 → Pila: [35]

Resultado: 35

Otra aplicación, que resulta un problema clásico resuelto con pilas, es verificar si una secuencia de símbolos de agrupación (paréntesis, corchetes, llaves) está correctamente balanceada. El algoritmo sería el siguiente:

1. Recorrer la expresión de izquierda a derecha.
2. Si encontramos un símbolo de apertura ((, [, {), lo apilamos.
3. Si encontramos un símbolo de cierre (),], }):
 4. Si la pila está vacía → Error (cierre sin apertura)
 5. Desapilamos y verificamos que coincida con el símbolo de apertura
6. Al final, si la pila no está vacía → Error (apertura sin cierre)

Este algoritmo se extiende naturalmente a la validación de documentos XML/HTML (balanceo de etiquetas) y a la verificación de sintaxis en lenguajes de programación, por mostrar dos ejemplos del mundo real muy utilizados.

Otras aplicaciones serían los navegadores web (pila adelante/atrás) o los editores de texto (pila deshacer/rehacer) utilizan pilas para gestionar el historial de acciones, así, en el navegador al navegar a una nueva página, se apila la anterior: "Atrás" desapila, "Adelante" requiere una segunda pila. En el caso del editor, cada acción de edición se apila: "Deshacer" desapila la última acción, "Rehacer" la vuelve a apilar en una pila secundaria.

En problemas de búsqueda como laberintos, el problema de las 8 reinas, o recorridos de grafos, las pilas permiten implementar la técnica conocida como *backtracking* (vuelta atrás) de manera natural. Cada decisión se "apila", y si llegamos a un callejón sin salida, "desapilamos" para probar alternativas. En un laberinto, por ejemplo, en cada intersección apilamos las direcciones no exploradas y cuando llegamos a un callejón sin salida, desapilamos para volver a la última intersección con alternativas pendientes.

Resumen

La pila, a pesar de su simplicidad conceptual, es una de las estructuras de datos más importantes en la computación. Su implementación directa (con arrays o listas enlazadas) sirve como excelente introducción a conceptos más avanzados de estructuras de datos, mientras que sus aplicaciones prácticas demuestran la utilidad de los principios abstractos estudiados en capítulos anteriores.

La pila sigue un principio LIFO: Último en entrar, primero en salir - simple pero potente. Sus operaciones básicas son: `push`, `pop`, `peek`, `isEmpty`, `initialize` - contrato mínimo.

Hay dos implementaciones principales: Con arrays, una implementación sencilla, eficiente, pero con limitaciones derivadas de su tamaño fijo; o con memoria dinámica, más flexible, con tamaño ilimitado pero también con una mayor necesidad de memoria por elemento.

En cuanto a su complejidad, todas las operaciones básicas son $O(1)$ en ambas implementaciones.

Es una estructura que tiene un buen número de aplicaciones ubicuas: Call stack, evaluación de expresiones, validación de sintaxis, *backtracking*, historial de un navegador, etc.

El estudio de las pilas no solo nos proporciona una herramienta práctica, sino que también refuerza los conceptos fundamentales de la asignatura: la importancia de una especificación clara (TAD), la separación entre interfaz e implementación (abstracción), y el análisis de eficiencia (complejidad algorítmica). En los próximos capítulos, estudiaremos otras estructuras lineales (colas, listas) y no lineales (árboles, grafos), cada una con sus propias características y aplicaciones. Sin embargo, muchos de los principios aprendidos aquí -especialmente el uso de memoria dinámica y el diseño de interfaces limpias- se aplicarán de manera similar en esas estructuras más complejas. La pila es, en muchos sentidos, la puerta de entrada al mundo rico y diverso de las estructuras de datos.

Tema V – Colas

Tras haber estudiado las pilas y su comportamiento, nos adentramos ahora en el estudio de otra estructura lineal fundamental: la cola (*queue*). Si las pilas se comportan como una pila de platos donde solo podemos operar por la parte superior, las colas se asemejan a una fila de personas esperando su turno: el primero en llegar es el primero en ser atendido. Este principio se conoce como FIFO (*First In, First Out*) y constituye uno de los paradigmas más intuitivos y ampliamente utilizados en la organización de procesos.

Las colas tienen aplicación en innumerables contextos de los sistemas computacionales y del mundo real: desde la gestión de procesos en un sistema operativo hasta la organización de trabajos de impresión, pasando por la atención al cliente en centros de llamadas o la gestión de paquetes en redes de computadoras. La importancia de esta estructura radica en su capacidad para modelar de manera natural procesos donde el orden de llegada determina el orden de atención, garantizando equidad y previsibilidad.

Este capítulo explorará en profundidad el tipo abstracto de datos Cola, analizando sus diferentes implementaciones, complejidades computacionales y aplicaciones prácticas. Veremos cómo, a pesar de su simplicidad conceptual, la implementación eficiente de una cola presenta desafíos interesantes que nos obligarán a aplicar creativamente los conocimientos adquiridos sobre arrays, punteros y gestión de memoria dinámica.

1. El TAD Cola

Un Tipo Abstracto de Datos Cola define una estructura lineal que sigue estrictamente el principio FIFO. Su especificación formal en lenguaje natural sería:

- **Nombre:** Cola
- **Valores posibles:** Vacía o conteniendo una secuencia ordenada de elementos del tipo base, donde solo los elementos en los extremos (primero y último) son directamente accesibles.
- **Operaciones básicas:**
 - `initialize_queue(c)`: Inicializa una cola vacía.
 - `enqueue(c, x)`: Añade el elemento `x` al final de la cola `c`.
 - `dequeue(c)`: Elimina el elemento situado al frente de la cola `c`.
 - `first(c)`: Consulta (sin eliminar) el elemento situado al frente de la cola `c`.
 - `last(c)`: Consulta (sin eliminar) el elemento situado al final de la cola `c`.
 - `is_empty(c)`: Determina si la cola `c` no contiene elementos.
- **Uso de otros tipos:** Depende del tipo base de elementos que contendrá la cola y de las estructuras auxiliares utilizadas en la implementación.

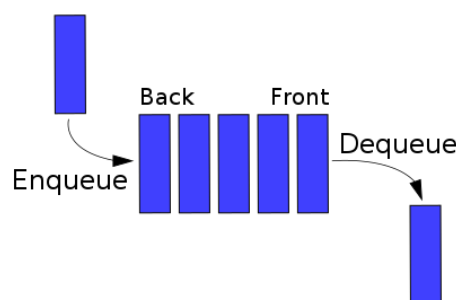


Ilustración 10. Operaciones básicas de un TAD Cola

Algunas operaciones conllevan precondiciones que garantizan su correcto funcionamiento:

- **enqueue (c, x)** : Añade x al final de c. *Precondición*: La cola no debe estar llena (solo para implementaciones con capacidad limitada).
- **dequeue (c)** : Elimina el elemento del frente de c. *Precondición*: La cola no debe estar vacía pues intentar hacer dequeue en una cola vacía es un error (*underflow*).
- **first (c)** : Devuelve el elemento del frente sin modificarlo. *Precondición*: La cola no debe estar vacía.
- **last (c)** : Devuelve el elemento del final sin modificarlo. *Precondición*: La cola no debe estar vacía.

La nomenclatura de estas operaciones varía según los lenguajes y contextos: enqueue/dequeue son comunes en contextos académicos, mientras que en algunas APIs se usan add/remove o offer/poll.

Además de las operaciones básicas, a menudo resulta útil extender el TAD Cola con funcionalidades adicionales tales como:

- **clear (c)** : Vacía completamente la cola.
- **size (c)** o **num_elems (c)** : Devuelve el número de elementos actualmente en la cola.
- **toString (c)** : Genera una representación textual de la cola.
- **copy (c1, c2)** : Crea una copia independiente de la cola c1 en c2.
- **isFull (c)** : Determina si la cola ha alcanzado su capacidad máxima (en implementaciones con límite).

Estas operaciones pueden implementarse eficientemente o, en algunos casos, requerirán recorrer toda la cola, afectando su complejidad temporal.

2. Implementación con arrays

Mientras que la implementación de pilas con arrays es relativamente sencilla (solo necesitamos un índice para la cima), las colas presentan un desafío mayor: necesitamos gestionar dos extremos (frente y final) que se mueven independientemente. Esta dualidad genera varios problemas de diseño, dando lugar a múltiples estrategias de implementación, cada una con sus ventajas e inconvenientes.

A continuación veremos algunas alternativas de implementación.

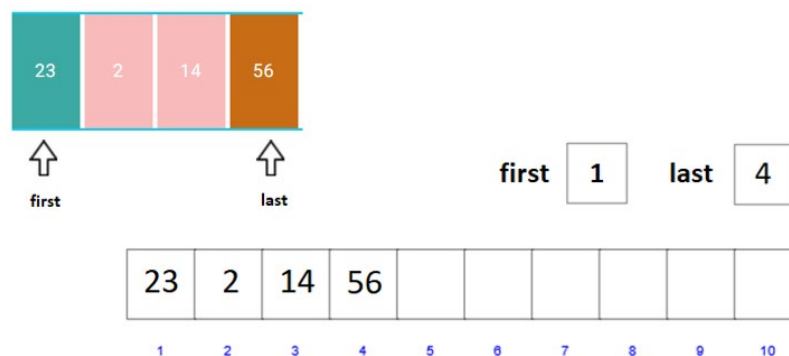


Ilustración 11. Esquema de un TAD cola implementado con arrays

Array lineal con eliminación simple

Mantenemos dos índices: `first` (frente) y `last` (final). La inserción siempre ocurre al final, incrementando `last`. La eliminación simplemente incrementa `first`.

Ejemplo visual:

Estado inicial: `first=1`, `last=0` (cola vacía)

[| | | |] (array de tamaño 5)

Después de `enqueue(10)`: `first=1`, `last=1`

[10| | |]

Después de `enqueue(20)`, `enqueue(30)`: `first=1`, `last=3`

[10|20|30| |]

Después de `dequeue()`: `first=2`, `last=3`

[10|20|30| |] (el 10 sigue en memoria pero ya no es parte de la cola)

Este enfoque tiene la ventaja de que su implementación es extremadamente sencilla, y las operaciones `enqueue` y `dequeue` son $O(1)$. Sin embargo, se desaprovecha espacio pues a medida que hacemos `dequeue`, el espacio al inicio del array queda inutilizable. Aunque el array pueda tener espacio libre al inicio, cuando `last` llega al final, consideramos la cola llena lo que en la práctica significa que los elementos "eliminados" siguen ocupando memoria.

En definitiva, esta implementación es insostenible para usos prolongados, ya que desperdicia progresivamente la capacidad del array.

Array lineal con reorganización

Para evitar el problema del agotamiento ficticio del espacio del modelo anterior, ahora cada vez que eliminemos un elemento, desplazaremos todos los elementos restantes una posición hacia el inicio, manteniendo `first` siempre en la posición 1.

Ejemplo visual:

Estado inicial:

[10|20|30|40|], `first=1`, `last=4`

Después de `dequeue()` y reorganizar:

[20|30|40| |], `first=1`, `last=3`

Esto obviamente complica la implementación del procedimiento de eliminación que no se limita únicamente a mover el índice `last` sino que incorpora reorganización:

```
procedure dequeue(var c: tColaArray);
var i: integer;
begin
  if not empty_queue(c) then begin
    { Desplazar todos los elementos hacia la izquierda }
    for i := 1 to c.last-1 do
      c.array[i] := c.array[i+1];
    c.last := c.last - 1;
  end;
end;
```

Este enfoque sí utiliza eficientemente todo el espacio del array, a diferencia del modelo anterior, dado que `first` siempre es 1 lo que además simplifica algunas operaciones. Sin embargo, `dequeue` es $O(n)$ ya que debemos desplazar $n-1$ elementos tras cada eliminación, lo que resulta un coste inaceptable para colas grandes. Así, esta implementación solo es adecuada para colas muy pequeñas o para colas en las que las eliminaciones son infrecuentes.

Array circular

Otra posibilidad para implementar una cola mediante la utilización como contenedor de elementos de un array, es tratar el array como un “anillo” donde el último elemento está lógicamente enlazado con el primero, usando aritmética modular para calcular las posiciones.

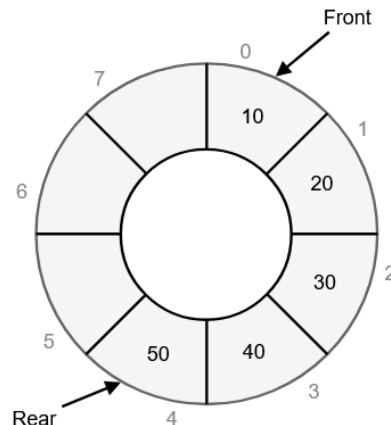


Ilustración 12. Implementación de una Cola mediante array circular

En esta implementación, el cálculo de posiciones debe utilizar aritmética modular, tanto en la inserción como en la eliminación. Así:

```
last := (last mod TAM) + 1; { Para insertar (enqueue) }
first := (first mod TAM) + 1; { Para eliminar (dequeue) }
```

Ejemplo visual (TAM=5):

Estado: `first=1`, `last=3`, `count=2`

[10|20| | |] (10 en posición 1, 20 en posición 2)

Después de `enqueue(30)`: `first=1`, `last=4`, `count=3`

[10|20|30| |]

Después de `dequeue()`: `first=2`, `last=4`, `count=2`

[|20|30| |] (la posición 1 está ahora disponible para reuso)

Después de `enqueue(40)`, `enqueue(50)`, `enqueue(60)`: `first=2`, `last=2`, `count=5`

[60|20|30|40|50] (cola llena, 60 ocupó la posición 5 y "volvió al inicio")

La ventaja de este modelo es que utiliza eficientemente todo el espacio, sin perjudicar el rendimiento ya que las operaciones `enqueue` y `dequeue` son $O(1)$ pues no requiere reorganización. Por otra parte, la implementación es algo más compleja que el modelo lineal, pero, sobre todo, sigue adoleciendo del problema fundamental de todas las implementaciones con array: el tamaño máximo es fijo, con todas las desventajas ya detalladas anteriormente.

Una posible implementación en Pascal sería la siguiente:

```
unit uColaCircular;

interface

    const
        TAM = 100;

    type
        tColaCircular = record
            elementos: array[1..TAM] of integer;
            first: integer; { Índice del primer elemento }
            last: integer;  { Índice donde se insertará el próximo }
            count: integer; { Número actual de elementos }
        end;

    procedure initialize_queue(var c: tColaCircular);
    function is_empty(c: tColaCircular): boolean;
    function is_full(c: tColaCircular): boolean;
    procedure enqueue(var c: tColaCircular; x: integer);
    procedure dequeue(var c: tColaCircular);
    function first(c: tColaCircular): integer;

implementation

    procedure initialize_queue(var c: tColaCircular);
    begin
        c.first := 1;
        c.last := 1;
        c.count := 0;
    end;

    function is_empty(c: tColaCircular): boolean;
    begin
        is_empty := (c.count = 0);
    end;

    function is_full(c: tColaCircular): boolean;
    begin
        is_full := (c.count = TAM);
    end;

    procedure enqueue(var c: tColaCircular; x: integer);
    begin
        if not is_full(c) then begin
            c.elementos[c.last] := x;
            c.last := (c.last mod TAM) + 1;
            c.count := c.count + 1;
        end;
    end;

    procedure dequeue(var c: tColaCircular);
    begin
        if not is_empty(c) then begin
            c.first := (c.first mod TAM) + 1;
            c.count := c.count - 1;
        end;
    end;

    function first(c: tColaCircular): integer;
    begin
        if not is_empty(c) then
            first := c.elementos[c.first];
    end;
```

Comparativa de implementaciones con arrays

Como resumen de lo visto hasta el momento, el siguiente cuadro analiza la complejidad de las operaciones de cada modelo de implementación:

Característica	Eliminación simple	Con reorganización	Array circular
Complejidad enqueue	O(1)	O(1)	O(1)
Complejidad dequeue	O(1)	O(n)	O(1)
Uso de memoria	Ineficiente (desperdicia)	Eficiente	Eficiente
Complejidad implementación	Muy simple	Simple	Moderada
Tamaño máximo	Fijo, mal aprovechado	Fijo, bien aprovechado	Fijo, bien aprovechado
Casos de uso	Solo demostración	Colas muy pequeñas	Producción (tamaño conocido)

La tabla muestra cómo el array circular es la única implementación con arrays que ofrece operaciones O(1) y uso eficiente de memoria, por lo que es la preferida cuando se conoce un tamaño máximo razonable. Si el tamaño máximo no se conoce, ninguna de las implementaciones basadas en un array sería aceptable.

3. Implementación con memoria dinámica

Si bien existen otras posibilidades, la implementación dinámica más sencilla de una cola utiliza una lista enlazada simple de elementos, donde cada nodo contiene el dato a almacenar y un puntero al siguiente nodo. Esta implementación mantiene dos punteros: `first` (al primer nodo) y `last` (al último nodo) y, lógicamente, una cola vacía tiene ambos punteros a `nil`.

Las ventajas de la implementación sobre arrays es que desaparece la limitación de tamaño máximo, lógicamente excepto el límite de la memoria disponible en el sistema sobre el que se ejecute. Tampoco hay desperdicio de espacio ni se necesitan reorganizaciones. Las operaciones `enqueue` y `dequeue` son eficientes, como veremos, pues su complejidad es O(1).

También presenta algunas desventajas, como la necesidad de mayor memoria por elemento (ya que se necesita espacio para el puntero), o que es necesaria una gestión explícita de memoria en lenguajes sin recolector automático de basura (uso de operaciones `new/dispose`).

Pascal

```
unit uColaEnteros;

interface

type
  PNode = ^TNode;
  TNode = record
    info: integer;
    siguiente: PNode;
  end;

  TCola = record
    primero: PNode; { Frente de la cola }
    ultimo: PNode;  { Final de la cola }
  end;

procedure initialize_queue(var c: TCola);
function empty_queue(c: TCola): boolean;
procedure enqueue(var c: TCola; x: integer);
procedure dequeue(var c: TCola);
```

```

function first(c: TCola): integer;
function last(c: TCola): integer;

implementation

procedure initialize_queue(var c: TCola);
begin
    c.primeros := nil;
    c.ultimo := nil;
end;

function empty_queue(c: TCola): boolean;
begin
    empty_queue := (c.primeros = nil);
end;

procedure enqueue(var c: TCola; x: integer);
var
    nuevo: PNode;
begin
    new(nuevo);
    nuevo^.info := x;
    nuevo^.siguiente := nil;
    if empty_queue(c) then c.primeros := nuevo
    else c.ultimo^.siguiente := nuevo;
    c.ultimo := nuevo;
end;

procedure dequeue(var c: TCola);
var
    aux: PNode;
begin
    if not empty_queue(c) then begin
        aux := c.primeros;
        c.primeros := c.primeros^.siguiente;
        if c.primeros = nil then
            c.ultimo := nil;
        dispose(aux);
    end
    else
        writeln('Error: Cola vacía - no se puede hacer dequeue');
    end;

function first(c: TCola): integer;
begin
    if not empty_queue(c) then
        first := c.primeros^.info
    else begin
        writeln('Error: Cola vacía - no se puede consultar first');
        first := 0; { Valor por defecto }
    end;
end;

function last(c: TCola): integer;
begin
    if not empty_queue(c) then
        last := c.ultimo^.info
    else begin
        writeln('Error: Cola vacía - no se puede consultar last');
        last := 0; { Valor por defecto }
    end;
end;

end.

```

Podemos ampliar nuestro TAD Cola con algunas operaciones extra tales como:

```
procedure clear(var c: TCola);  
function size(c: TCola): integer;  
function toString(c: TCola): string;
```

cuya implementación sería:

```
procedure clear(var c: TCola); { Vacía completamente la cola }  
begin  
    while not empty_queue(c) do  
        dequeue(c);  
    end;  
  
function size(c: TCola): integer; { Cuenta el número de elementos (O(n)) }  
var  
    actual: PNode;  
    contador: integer;  
begin  
    actual := c.primeros;  
    contador := 0;  
    while actual <> nil do begin  
        contador := contador + 1;  
        actual := actual^.siguiente;  
    end;  
    size := contador;  
end;  
  
{ Versión O(1) de size: requiere modificar la estructura TCola  
para incluir un campo "cantidad" que se actualizaría en enqueue/dequeue }  
  
function toString(c: TCola): string; { Representación textual de la cola }  
var  
    actual: PNode;  
    resultado: string;  
begin  
    resultado := '[';  
    actual := c.primeros;  
    while actual <> nil do begin  
        resultado := resultado + IntToStr(actual^.info);  
        actual := actual^.siguiente;  
        if actual <> nil then  
            resultado := resultado + ', ';  
        end;  
    end;  
    resultado := resultado + ']';  
    toString := resultado;  
end;
```

4. Aplicaciones prácticas de las colas

Una de las aplicaciones más comunes de las colas es la planificación de procesos en los sistemas operativos, donde distintos algoritmos, como FIFO Simple (First-Come, First-Served) y Round Robin, organizan la ejecución de tareas, asegurando equidad y eficiencia en el uso de la CPU. De manera similar, los sistemas de impresión emplean colas para gestionar trabajos mediante spooling, encolando cada documento enviado por múltiples usuarios y procesándolos en orden FIFO, lo que previene conflictos y garantiza un acceso ordenado al recurso compartido.

En el ámbito de las redes, los routers y switches utilizan colas para administrar el tráfico de paquetes, encolando los que llegan por un puerto para ser enviados por otro y aplicando políticas como FIFO para priorizar la transmisión durante congestiones. Los centros de atención al cliente, a través de sistemas de distribución automática de llamadas, también se basan en colas para

gestionar las llamadas entrantes, asignar agentes a las llamadas más antiguas, informar a los usuarios sobre su tiempo de espera e implementar prioridades para clientes especiales.

Otro uso muy común es la simulación de eventos discretos (simulaciones de tráfico de vehículos, líneas de producción o servicios de atención), donde las colas permiten modelar eficazmente situaciones como el comportamiento de los vehículos en un semáforo, los clientes en las cajas de un supermercado, los aviones en espera de despegue o aterrizaje, o los paquetes que aguardan en líneas de ensamblaje, demostrando así la versatilidad y utilidad de esta estructura en la representación de sistemas donde el orden de llegada determina el orden de atención.

5. Complejidad computacional de las operaciones

Para ambas implementaciones principales (array circular y lista enlazada), las complejidades son:

Operación	Array circular	Lista enlazada	Comentario
<code>initialize_queue</code>	$O(1)$	$O(1)$	
<code>empty_queue</code>	$O(1)$	$O(1)$	
<code>enqueue</code>	$O(1)$	$O(1)$	Amortizado $O(1)$ en lista
<code>dequeue</code>	$O(1)$	$O(1)$	
<code>first</code>	$O(1)$	$O(1)$	
<code>last</code>	$O(1)$	$O(1)$	Requiere puntero al último
<code>size</code>	$O(1)$	$O(n)$ o $O(1)^*$	*Con contador adicional
<code>clear</code>	$O(1)$	$O(n)$	Liberar todos los nodos
<code>toString</code>	$O(n)$	$O(n)$	

A la vista del cuadro anterior, hacemos un par de observaciones importantes relativas a la operación extra `clear`. En el array circular `clear` es $O(1)$ porque solo necesita resetear índices, no borrar elementos. En la lista enlazada, sin embargo, `clear` es $O(n)$ porque debe liberar cada nodo individualmente. Resaltar que la implementación con contador para `size` en lista enlazada es un buen ejemplo de optimización espacio-tiempo.

En cuanto a los factores a considerar a la hora de elegir una u otra implementación, debemos destacar los siguientes:

- **Tamaño:** Si el tamaño máximo es conocido y razonable elegimos un array circular, si es impredecible o muy variable la implementación idónea es la lista enlazada.
- **Frecuencia de operaciones:** Para uso intensivo elegiremos un array circular, pero si vamos a hacer un uso esporádico entonces mejor la lista enlazada (flexibilidad).
- **Requisitos de memoria:** Será muy raro pero si la memoria es muy limitada puede ser mejor el array circular (ya que el overhead es fijo), mientras que si la memoria es abundante optaremos por la lista enlazada, que puede crecer.
- **Lenguaje y plataforma:** En lenguajes con recolector de basura (Java, Python) es preferible la implementación con lista enlazada, más simple, pero en lenguajes con gestión manual (C, Pascal) es mejor un array circular, pues evita el engorro de la gestión de memoria.

En resumen: La elección depende del contexto específico: para tamaños conocidos y predecibles, el array circular ofrece mejor rendimiento; para tamaños variables o desconocidos, la lista enlazada proporciona flexibilidad. Ambas garantizan operaciones $O(1)$ para las funcionalidades básicas, cumpliendo con los requisitos de eficiencia de la mayoría de las aplicaciones.

En el próximo capítulo, exploraremos estructuras más complejas como las listas enlazadas, que generalizan tanto pilas como colas, permitiendo operaciones de inserción y eliminación en cualquier posición. Sin embargo, la comprensión sólida de pilas y colas será fundamental para abordar esas estructuras más generales, ya que muchas de sus operaciones se implementan utilizando pilas y colas como componentes auxiliares.

Tema VI – Listas

En el ámbito de las estructuras de datos, las listas representan uno de los pilares fundamentales sobre los que se construyen sistemas de almacenamiento y organización de información más complejos. A diferencia de las estructuras estáticas como los arrays, que requieren dimensionamiento previo y presentan limitaciones en su capacidad de expansión, las listas ofrecen una flexibilidad incomparable al permitir la adición y eliminación dinámica de elementos sin restricciones de tamaño predefinido.

Una lista es una estructura de datos lineal y dinámica que representa una secuencia de elementos, generalmente del mismo tipo. Cada elemento se conecta al siguiente mediante un "enlace" o referencia, formando una cadena de nodos. Esta característica permite tener los elementos en secuencia con la capacidad de llevar a cabo inserciones y eliminaciones eficientes en cualquier posición, a la vez que se hace un uso óptimo de memoria ya que solo se asigna lo necesario.

1. El TAD Lista

El TAD Lista es un ejemplo clásico de abstracción de datos donde se define **qué hace** una lista sin especificar **cómo lo hace**. La flexibilidad de las listas hace que no exista una única definición; más bien, el contrato de operaciones depende del tipo de lista que se desee implementar.

Por su flexibilidad, no existe una única definición del TAD lista: debe determinarse primero el tipo de lista que se desea, lo cual condiciona sus operaciones (su contrato), ya que, por ejemplo, una lista sin restricciones permite operaciones no toleradas en una lista ordenada, tales como `insertar_final`. En todo caso, la especificación en lenguaje natural de una lista general sería:

- **Nombre:** Lista.
- **Valores posibles:** vacía o conteniendo una secuencia de elementos del tipo base.
- **Operaciones:**
 - `insert_at_end(list, elem)`: inserta un elemento al final de la lista
 - `insert_at_begin(list, elem)`: inserta un elemento al principio de la lista
 - `delete_at_end(list)`: elimina el elemento al final de la lista
 - `delete_at_begin(list)`: elimina el elemento al principio de la lista
 - `first(list)`: accede al primer elemento de la lista
 - `last(list)`: accede al último elemento de la lista
 - `is_empty(list)`: comprueba si la lista está vacía
 - `in_list(list, elem)`: determina si un cierto elemento está en la lista
- **Uso de otros tipos:** Depende de los elementos que contenga y de las estructuras auxiliares utilizadas en la implementación. A menudo también, punteros.

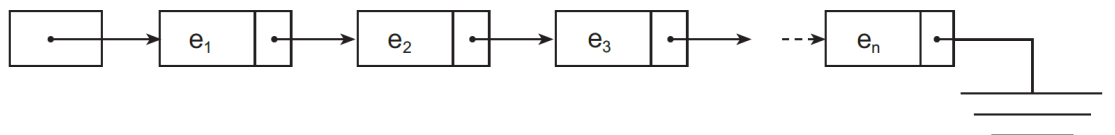


Ilustración 13. Lista enlazada simple

La implementación de listas en estructuras de soporte estáticas como arrays no es habitual, siendo en casi la totalidad de ocasiones preferida una implementación en memoria dinámica. Atendiendo a eso, su implementación puede ser distinta. Así, tendremos listas enlazadas simples, donde en cada nodo hay una referencia o apuntador a otro nodo; listas doblemente enlazadas, donde cada

nodo referencia al anterior y al siguiente; y lista circular, donde el último nodo referencia al primero, etc.

Por otra parte, dependiendo de las restricciones impuestas, podemos también encontrarnos con diferentes tipos de listas:

1. Lista sin restricciones: Permite todas las operaciones básicas
2. Lista ordenada: Mantiene los elementos en un orden específico
3. Lista con acceso restringido

La implementación de listas en memoria dinámica se basa, como ya vimos en las colas y pilas, en el concepto de nodo, una estructura que contiene el dato a almacenar junto con una referencia al siguiente nodo (y al anterior, en listas dobles).

La implementación basada en memoria dinámica permite trabajar solo con el espacio que realmente se necesita, evitando así el desperdicio de tener que asignar un espacio a priori que no se sabe si será adecuado. Además, las operaciones de inserción y eliminación resultan mucho más ágiles, ya que no es necesario desplazar elementos. Otra ventaja importante es que la estructura puede crecer sin límites predeterminados, únicamente condicionada por la memoria disponible en el sistema, a diferencia de lo que ocurre con los arrays estáticos. Esta forma de gestionar los datos aporta una gran flexibilidad, lo que facilita la creación y manejo de estructuras más complejas.

2. Lista enlazada simple

La lista enlazada simple es la forma más básica de lista, donde cada nodo apunta solo al siguiente. En Pascal, su implementación sería la siguiente:

type

```
PNodo = ^tNodo;
```

```
tNodo = record
```

```
  info: integer;      // Dato almacenado
```

```
  sig: PNodo;        // Puntero al siguiente nodo
```

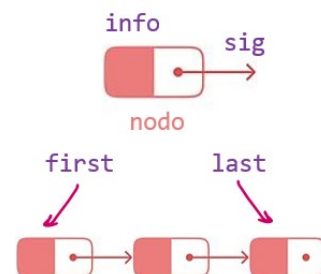
```
end;
```

```
TListaSimple = record
```

```
  first: PNodo;      // Puntero al primer nodo
```

```
  last: PNodo;       // Puntero al último nodo
```

```
end;
```



Como se ve, la lista es en realidad un registro (record) dos punteros a un nodo de tipo lista (first y last), y esto es así para optimizar las operaciones en ambos extremos de la lista.

Inicialización

Operación para establecer un estado inicial consistente antes de cualquier operación sobre la lista. Tiene complejidad $O(1)$ ya que simplemente asigna valores nulos a dos punteros.

```
procedure initialize(var list: TListaSimple);
begin
  list.first := nil;
  list.last := nil;
end;
```

Inserciones

La **inserción al final** tiene complejidad $O(1)$ gracias al puntero last. Sin él, sería necesario recorrer toda la lista para encontrar el último nodo, resultando en $O(n)$.

```

procedure insert_at_end(var list: TListaSimple; x: integer);
var
  newNode: PNode;
begin
  new(newNode);           // 1. Reserva memoria para el nuevo nodo
  newNode^.info := x;     // 2. Asigna el valor
  newNode^.sig := nil;    // 3. El nuevo nodo será el último
  if is_empty(list) then // 4. Caso especial: lista vacía
    list.first := newNode
  else                    // 5. Caso general: lista no vacía
    list.last^.sig := newNode;
  list.last := newNode;   // 6. Actualiza el puntero al último nodo
end;

```

Veamos un ejemplo. Partimos del estado inicial:

```

[10] -> [20] -> nil
first↑   last↑

```

Ejecutamos la instrucción de inserción: `insert_at_end(list, 30)`

Sucede lo siguiente:

1. Crear nuevo nodo: `[30] -> nil`
2. Actualizar enlace: `[20] -> [30]`
3. Actualizar last: last ahora apunta a `[30]`

```

Estado final: [10] -> [20] -> [30] -> nil
               first↑           last↑

```

En cuanto a la **inserción al principio** de la lista, se trata de una operación particularmente eficiente y útil en aplicaciones donde se necesita acceso rápido a los elementos más recientes. Su complejidad es $O(1)$:

```

procedure insert_at_begin(var list: TListaSimple; x: integer);
var
  newNode: PNode;
begin
  new(newNode);
  newNode^.info := x;
  newNode^.sig := list.first; // El sig. al nuevo apunta al antiguo primero
  list.first := newNode;     // Actualizar puntero al primer nodo
  if list.last = nil then    // Si la lista estaba vacía
    list.last := newNode;    // El nuevo nodo también es el último
end;

```

Eliminación de elementos

Para eliminar tenemos ambas opciones, igual que en la inserción. La **inserción al principio de la lista** es una operación eficiente: Complejidad $O(1)$:

```

procedure delete_at_begin(var list: TListaSimple);
var
  temp: PNode;
begin
  if not is_empty(list) then
  begin
    temp := list.first;           // Guardar referencia al nodo a eliminar

```

```

        list.first := list.first^.sig; // Actualizar el primer puntero
        dispose(temp);                // Liberar memoria
        if list.first = nil then      // Si la lista quedó vacía
            list.last := nil;         // Actualizar puntero al último
        end;
    end;
end;

```

La otra operación de **inserción al final** requiere recorrer toda la lista para encontrar el penúltimo nodo. Su ineficiencia relativa justifica el uso de listas doblemente enlazadas cuando se necesitan eliminaciones frecuentes al final:

```

procedure delete_at_end(var list: TListaSimple);
var
    current, prev: PNode;
begin
    if not is_empty(list) then
        begin
            current := list.first;
            prev := nil;
            // Recorrer hasta el último nodo
            while current^.sig <> nil do begin
                prev := current;
                current := current^.sig;
            end;
            if prev = nil then begin // Solo había un elemento
                dispose(list.first);
                list.first := nil;
                list.last := nil;
            end
            else begin // Había más de un elemento
                dispose(current);
                prev^.sig := nil;
                list.last := prev;
            end;
        end;
    end;
end;

```

Pertenencia de un elemento

La función **in** determina si un elemento está o no en la lista. A continuación proponemos dos implementaciones, una iterativa y otra recursiva:

```

function in_list(list: TListaSimple; x: integer): boolean;
var
    current: PNode;
begin
    current := list.first;
    // Recorremos la lista hasta encontrar el elemento o llegar al final
    while (current <> nil) and (current^.info <> x) do
        current := current^.sig;
    end;
    in_list := current <> nil;
end;

function rec_in_list(list: TListaSimple; x: integer): boolean;
var sublist : tListaSimple;
begin
    if list.first = nil then rec_in_list := false // Caso base I: lista vacía
    else if list.first^.info = x then rec_in_list := true // Caso base II: está
    else begin // Llamada recursiva con una sublista sin el primer elemento
        sublist.first := list.first^.sig;
        sublist.last := list.last;
        rec_in_list := rec_in_list(sublist, x);
    end;
end;

```

En cuanto a su complejidad, el mejor escenario ocurre cuando el elemento buscado se encuentra en la primera posición, lo que permite resolver la operación en tiempo constante, $O(1)$. El peor caso aparece cuando el elemento está al final de la lista o no está en ella, situación en la que es necesario recorrerla por completo, alcanzando así un coste computacional de $O(n)$. Si consideramos una distribución uniforme de las posiciones posibles, el caso promedio implica revisar aproximadamente la mitad de la lista, lo que se aproxima a $O(n)$ pese a expresarse como $O(n/2)$.

En cuanto a la implementación recursiva, si bien resulta conceptualmente elegante, presenta algunas desventajas prácticas que conviene tener en cuenta: Cada llamada recursiva ocupa espacio en la pila, de modo que en el peor de los casos el consumo crecería linealmente con el tamaño de la lista. Esto puede convertirse en un problema en entornos donde la memoria de la pila es limitada o está estrictamente controlada, haciendo que este enfoque sea menos adecuado en situaciones de gran profundidad o en sistemas con recursos restringidos.

Otras operaciones

Otras operaciones que podemos incluir en nuestro TAD ListaSimple serían:

```
function to_string(list: TListaSimple): string;
// genera una cadena de texto con el contenido de los elementos de la lista
var
  current: PNode;
  str: string;
begin
  str := '';
  current := list.first;
  while current <> nil do begin
    str := str + IntToStr(current^.info) + ' ';
    current := current^.sig;
  end;
  to_string := str;
end;

function num_elems(list: TListaSimple): integer;
// Cuenta el número de elementos de la lista
var
  current: PNode;
  count: integer;
begin
  count := 0;
  current := list.first;
  while current <> nil do begin
    count := count + 1;
    current := current^.sig;
  end;
  num_elems := count;
end;

procedure copy(list: TListaSimple; var list2: TListaSimple);
// Genera una copia de la lista
var
  current: PNode;
begin
  initialize(list2);
  current := list.first;
  while current <> nil do begin
    insert_at_end(list2, current^.info);
    current := current^.sig;
  end;
end;
```

3. Lista circular

En una lista circular, el último nodo apunta al primero, formando un ciclo. Esto elimina la necesidad de manejar casos especiales para el final de la lista. Se gestiona mediante un único puntero, que apunta al último de la lista, lo que de manera ingeniosa permite acceder también fácilmente al primero de la lista: acceder al primero es tan sencillo como consultar el campo sig del último elemento, lo que simplifica considerablemente su manejo.

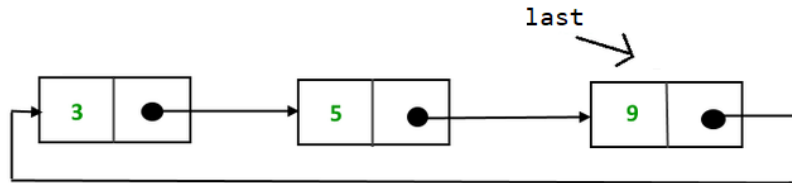


Ilustración 14. Lista circular

En este tipo de estructura no existen valores NIL, ya que todos los nodos tienen siempre un sucesor definido. Esto permite realizar un recorrido continuo, capaz de iterar sin detenerse mientras se siga avanzando por la secuencia. Su implementación en Pascal parte de la siguiente estructura de datos:

```
type
  tListaCircular = record
    last: PNode;
  end;
```

En cuanto a sus operaciones, algunas de ellas no sufren modificaciones con respecto a la lista no circular vista anteriormente, pero la mayoría sí ya que la estructura de manejo es distinta:

```
procedure initialize(var list: tListaCircular);
begin
  list.last := nil;
end;

function is_empty(list: tListaCircular): boolean;
begin
  is_empty := list.last = nil;
end;

function first(list: tListaCircular): integer;
begin
  if not is_empty(list) then first := list.last^.sig^.info;
end;

function in_list(list: tListaCircular; x: integer): boolean;
var
  aux: ^nodo;
  found: boolean;
begin
  found := false;
  if not is_empty(list) then begin
    aux := list.last^.sig;
    while (aux <> list.last) and not found do begin // busca el elemento
      if aux^.info = x then found := true
      else aux := aux^.sig;
    end;
    if aux^.info = x then found := true;
  end;
  in_list := found;
end;
```

```

procedure delete(var list: tListaCircular; x: integer);
var
    current, prev: ^nodo;
    found: boolean;
begin
    // Caso 1: Lista vacía (no hacemos nada)
    if not is_empty(list) then begin
        // Caso 2a: Un único nodo en la lista
        if list.last^.sig = list.last then begin
            if list.last^.info = x then begin
                // Si el único nodo es el que queremos eliminar
                begin
                    dispose(list.last);
                    list.last := nil;
                end;
            end;
        end
        // Caso 2b: Más de un nodo, buscamos el elemento
        found := false;
        prev := list.last; // El anterior al primero es el último
        current := list.last^.sig; // Empezar a buscar por el primero
        repeat
            if current^.info = x then // Encontrado el nodo a eliminar
                found := true
            else begin
                prev := current; // Avanzar puntero al nodo anterior
                current := current^.sig;
            end;
        until found or (current = list.last^.sig);

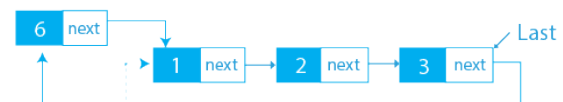
        // Si existe, lo eliminamos
        if found then begin
            // "Punteamos" el nodo actual
            prev^.sig := current^.sig;
            // Si borramos el último nodo, actualizamos last
            if current = list.last then
                list.last := prev;
            dispose(current);
        end;
    end;
end;
end;
end;

```

```

procedure insert_at_begin(var list: tListaCircular; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    if is_empty(list) then begin
        aux^.sig := aux;
        list.last := aux;
    end
    else begin
        aux^.sig := list.last^.sig;
        list.last^.sig := aux;
    end;
end;
end;

```



```

procedure insert_at_end(var lista: tListaCircular; x: integer);
var
  aux: PNode;
begin
  new(aux);
  aux^.info := x;
  if is_empty(lista) then
    aux^.sig := aux
  else begin
    aux^.sig := lista.last^.sig;
    lista.last^.sig := aux;
  end;
  lista.last := aux;
end;

```



```

procedure copy(list: tListaCircular; var list2: tListaCircular);
var
  aux, new_node: ^nodo;
begin
  initialize(list2);
  if not is_empty(list) then begin
    aux := list.last;
    repeat
      insert_at_end(list2, aux^.info);
      aux := aux^.sig;
    until aux = list.last;
  end;
end;

```

La siguiente tabla compara las dos implementaciones anteriores:

	Lista Simple	Lista Circular
Estructura	Tiene un inicio y un final bien definidos	Todos los nodos forman un ciclo cerrado
Punteros nulos	El último nodo tiene sig = nil	No existen punteros nulos
Recorrido completo	Termina cuando current = nil	Termina al volver al nodo inicial
Inserción al principio	O(1)	O(1)
Inserción al final	O(1) si se mantiene un puntero last	O(1)
Eliminación al final	O(n)	O(n)
Aplicaciones típicas	Procesamiento secuencial general	Procesamiento cíclico, buffers circulares, tareas repetitivas

Digresión: Comprendiendo y recordando los métodos

En este punto queremos que reflexiones. Has estudiado ya varias estructuras de datos: pilas, colas, listas, y pronto veremos más. Has visto muchos métodos, implementaciones que te hemos proporcionado y que debes ser capaz de recordar, pero... ¿Es la implementación que te mostramos aquí la única manera de hacer las cosas? La respuesta a esta pregunta es: rotundamente NO.

El código que te mostramos para cada método es una solución correcta y válida, pero no es la única. En programación, y especialmente trabajando con punteros, hay muchas formas de llegar al mismo resultado. Nuestras soluciones suelen ser muy académicas porque primamos la claridad ya que los materiales tienen un objetivo de aprendizaje, sobre todo.

Dicho esto, nos permitimos darte algunos consejos que esperamos te sean útiles:

- No memorices el código. Aprenderse de memoria líneas como esta del método delete: `prev^.sig := current^.sig` es inútil si no visualizas lo que ocurre.
- Entiende la lógica de punteros. Dibuja tu estructura de datos en un papel. Mueve las flechas (punteros). Entiende por qué necesitamos un puntero `prev` o por qué el `last` es especial.
- Haz tu propia implementación. Si entiendes el concepto (buscar → desenlazar → liberar), podrás escribir tu propio código sin necesidad de copiar el nuestro.

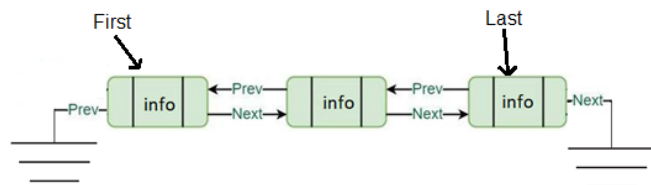
Recuerda: La memoria falla, la lógica permanece.

4. Listas doblemente enlazadas

Las listas doblemente enlazadas pueden entenderse como una evolución de las listas simples, ya que incorporan la posibilidad de desplazarse en ambos sentidos. Aquí, cada nodo incluye dos punteros: uno que apunta al siguiente elemento y otro que referencia al anterior, lo que permite recorrer la estructura de manera bidireccional. Gracias a esta organización, eliminar un nodo concreto resulta muy eficiente, pues basta con tener una referencia directa a él para eliminarlo en tiempo constante. Del mismo modo, insertar nuevos elementos antes o después de un nodo existente no requiere recorrer la lista, lo que hace a estas operaciones especialmente flexibles y rápidas. Su implementación sería:

```
type
  tNodoDoble = record
    info: integer;
    sig: ^tNodoDoble;
    ant: ^tNodoDoble;
  end;

  tListaDoble = record
    first: ^tNodoDoble;
    last: ^tNodoDoble;
  end;
```



Este tipo de estructura se emplea con frecuencia en aplicaciones donde el movimiento hacia adelante y hacia atrás es fundamental. Es el caso de los historiales de navegación en los navegadores web, los sistemas de edición de texto que permiten deshacer y rehacer acciones, o los mecanismos de caché como las implementaciones LRU (*Least Recently Used* o Menos Usada Recientemente). También constituyen la base de estructuras más complejas, como los deque o ciertos tipos de árboles, donde la eficiencia en la manipulación de nodos es especialmente valiosa.

Una implementación en Pascal de sus operaciones básicas sería la siguiente:

```
procedure initialize(var list: tListaDoble);
begin
  list.first := nil;
  list.last := nil;
end;

function is_empty(list: tListaDoble): boolean;
begin
  is_empty := list.first = nil;
end;

function first(list: tListaDoble): integer;
begin
  if not is_empty(list) then first := list.first^.info;
end;

function last(list: tListaDoble): integer;
begin
  if not is_empty(list) then last := list.last^.info;
end;
```

```

procedure insert_at_end(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := nil;
    aux^.ant := list.last;
    if list.last=nil then list.first := aux
    else list.last^.sig := aux;
    list.last := aux;
end;

procedure insert_at_begin(var list: tListaDoble; x: integer);
var
    aux: ^nodo;
begin
    new(aux);
    aux^.info := x;
    aux^.sig := list.first;
    aux^.ant := nil;
    if list.first = nil then list.last := aux
    else list.first^.ant := aux;
    list.first := aux;
end;

procedure delete_at_end(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then begin
        aux := list.last;
        list.last := aux^.ant;
        if list.last <> nil then
            list.last^.sig := nil
        else
            list.first := nil;
        dispose(aux);
    end;
end;

procedure delete_at_begin(var list: tListaDoble);
var
    aux: ^nodo;
begin
    if not is_empty(list) then
        begin
            aux := list.first;
            list.first := aux^.sig;
            if list.first <> nil then
                list.first^.ant := nil
            else
                list.last := nil;
            dispose(aux);
        end;
end;

function in_list(list: tListaDoble; x: integer): boolean;
var
    aux: ^nodo;
    found: boolean;
begin
    aux := list.first;
    found := false;
    while (aux <> nil) and not found do

```

```

if aux^.info = x then found := true;
else aux := aux^.sig;
in_list := found;
end;

```

5. Aplicaciones y estructuras derivadas

La flexibilidad de las listas, tanto simples como doblemente enlazadas, permite utilizarlas como base para construir estructuras de datos más ricas y con comportamientos especializados. Su capacidad para gestionar nodos de manera dinámica, sin necesidad de mover grandes bloques de memoria, facilita adaptar su funcionamiento a distintos tipos de problemas.

Por ejemplo, una lista puede transformarse en una lista ordenada simplemente garantizando que cada elemento nuevo se inserte en la posición adecuada. Esto permite mantener los datos siempre ordenados sin recurrir a procesos de reordenación completos cada vez que se modifica la estructura.

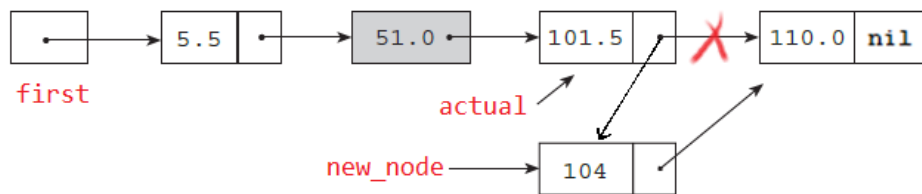


Ilustración 15. Inserción en una lista ordenada

El código de esta operación en Pascal sería:

```

procedure insert(var list: tSortedList; x: integer);
var
  new_node, actual: ^nodo;
begin
  new(new_node);
  new_node^.info := x;
  if is_empty(list) or (x < list.first^.info) then begin
    new_node^.next := list.first; // Insertar el primero
    list.first := new_node;
  end
  else begin // Situar actual en el nodo anterior a la inserción
    actual := list.first;
    while (actual^.next <> nil) and (x > actual^.next^.info) do
      actual := actual^.next;
    end;
    new_node^.next := actual^.next;
    actual^.next := new_node;
  end;
end;

```

Si, por el contrario, añadimos la idea de prioridad, la misma estructura puede servir para implementar colas de prioridades. En ellas, los elementos no se gestionan por orden de llegada, sino según un criterio que determina cuáles deben procesarse antes, algo muy útil en planificación de tareas o en algoritmos de búsqueda informada.

Las listas doblemente enlazadas también facilitan la creación de deque, estructuras en las que se puede insertar y eliminar información tanto por el inicio como por el final con la misma eficiencia. Este comportamiento resulta útil en áreas como la simulación, la gestión de buffers o ciertas estrategias de búsqueda.

Además, las listas pueden participar en la construcción de mapas o diccionarios, especialmente cuando necesitamos agrupar elementos que comparten un mismo valor hash. En estos casos, la lista actúa como contenedor flexible para almacenar pares clave-valor de manera organizada.

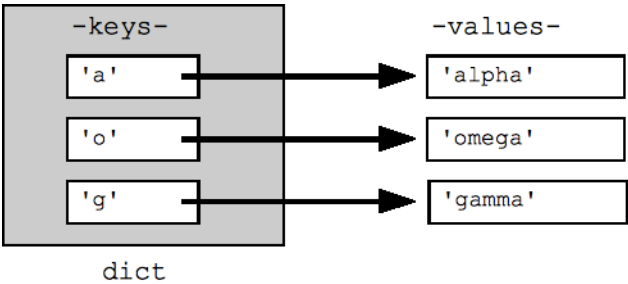


Ilustración 16. Mapa o diccionario

Por último, desempeñan un papel esencial en la representación de grafos. Cada nodo de un grafo puede mantener una lista con sus conexiones o adyacencias, lo que permite manejar estructuras complejas de relaciones tales como redes, rutas o dependencias, de una manera eficiente y relativamente sencilla de implementar.

En definitiva, las listas enlazadas funcionan como bloques fundamentales a partir de los cuales es posible desarrollar estructuras de datos más avanzadas. Comprender su funcionamiento y sus posibilidades abre la puerta a diseñar soluciones eficientes y adaptadas a cada problema.

La elección entre lista simple, circular, doble u ordenada depende de los requisitos específicos de operaciones y del patrón de uso esperado. Ser consciente de la complejidad de las operaciones, ayuda a tomar la decisión en cada caso. La siguiente tabla ofrece un resumen comparativo basado en la complejidad de las operaciones más habituales para cada estructura:

Operación	Lista Simple	Lista Circular	Lista Doble	Lista Ordenada
Insertar inicio	O(1)	O(1)	O(1)	O(n)*
Insertar final	O(1)	O(1)	O(1)	O(n)
Insertar medio	O(n)	O(n)	O(1)**	O(n)
Buscar	O(n)	O(n)	O(n)	O(n)
Eliminar inicio	O(1)	O(1)	O(1)	O(1)
Eliminar final	O(n) †	O(1) ††	O(1)	O(n)
Recorrer	O(n)	O(n)	O(n)	O(n)

* Depende de si la lista mantiene el orden en la inserción.
 ** Requiere tener ya una referencia al nodo donde se inserta.
 † Con puntero “last” pasa a ser O(1).
 ‡ Requiere ajustar correctamente los punteros.

Tema VII - Conjuntos y bolsas

Frecuentemente nos encontramos con la necesidad de agrupar elementos relacionados sin un orden específico ni relaciones de precedencia entre ellos. A diferencia de las estructuras lineales como listas, pilas y colas, que mantienen un orden definido de inserción o acceso, existen situaciones donde lo que importa es simplemente la pertenencia o no de un elemento a un grupo, o la frecuencia con que aparece. Para estas situaciones, las estructuras de datos conocidas como conjuntos (*sets*) y bolsas (*multisets*) ofrecen soluciones elegantes y eficientes.

Un conjunto matemático es una colección bien definida de objetos distintos, considerados como un objeto en sí mismo. Esta abstracción se traduce directamente en estructuras de datos que permiten operaciones como unión, intersección y diferencia, que tienen aplicaciones directas en bases de datos, motores de búsqueda, compiladores y sistemas de recomendación.

Las bolsas, por su parte, extienden este concepto permitiendo elementos duplicados, lo que las hace ideales para aplicaciones donde la multiplicidad importa, como el análisis de frecuencia de palabras en textos, el conteo de votos en sistemas electorales o el seguimiento de inventarios donde un mismo producto puede aparecer múltiples veces.

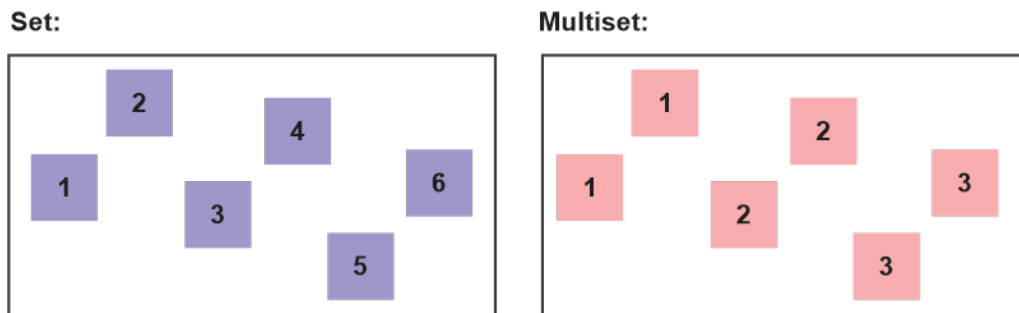


Ilustración 17. Conjuntos y bolsas

1. El TAD Conjunto

Un conjunto es una estructura de datos no lineal que representa una colección no ordenada de elementos en la que no se admiten duplicados. La característica fundamental de un conjunto es que cada elemento puede aparecer como máximo una vez. Esta propiedad de unicidad lo distingue de otras estructuras de colección y tiene importantes implicaciones tanto en su implementación como en su uso. Formalmente, un conjunto S sobre un dominio D puede definirse como $S \subseteq D$, donde para cualquier elemento $x \in D$, x puede estar en S o no, pero no puede estar más de una vez. La relación de pertenencia ($\in$) es la operación fundamental que define un conjunto.

En términos de implementación, esta definición implica que:

- No existe un orden inherente entre los elementos (aunque algunas implementaciones pueden mantener un orden por razones de eficiencia).
- La igualdad entre elementos debe estar bien definida para determinar si un elemento ya está presente.
- Las operaciones de inserción deben verificar la existencia previa del elemento.

Independientemente de su implementación concreta, un conjunto como tipo abstracto de datos proporciona un conjunto mínimo de operaciones que definen su comportamiento. Estas operaciones constituyen el contrato que cualquier implementación debe cumplir:

- Creación/inicialización: `initialize()` - crea un conjunto vacío, preparándolo para operaciones posteriores.
- Adición de elementos: `add(elemento)` - inserta un elemento en el conjunto si no está ya presente. Si el elemento ya existe, la operación no tiene efecto (idempotencia).
- Eliminación de elementos: `remove(elemento)` - Elimina un elemento específico del conjunto. Si el elemento no está en el conjunto, la operación no tiene efecto o puede lanzar una excepción dependiendo del diseño.
- Pertenencia: `in(elemento)` - Devuelve verdadero si el elemento está en el conjunto, falso en caso contrario.
- Tamaño: `size()` - Devuelve el número de elementos distintos en el conjunto.

Además de las operaciones básicas, los conjuntos suelen proporcionar operaciones derivadas que implementan conceptos de la teoría de conjuntos:

- Unión: `union(conj1, conj2)`: devuelve un nuevo conjunto que contiene todos los elementos que están en cualquiera de los dos conjuntos.
- Intersección: `intersection(conj1, conj2)`: Devuelve un nuevo conjunto que contiene solo los elementos que están en ambos conjuntos.
- Diferencia: `difference(conj1, conj2)`: Devuelve un nuevo conjunto con los elementos que están en el primer conjunto pero no en el segundo.
- Subconjunto: `is_subset(conj1, conj2)`: Devuelve verdadero si todos los elementos del primer conjunto están en el segundo.
- Otras: supraconjunto, diferencia simétrica, complementario, etc.

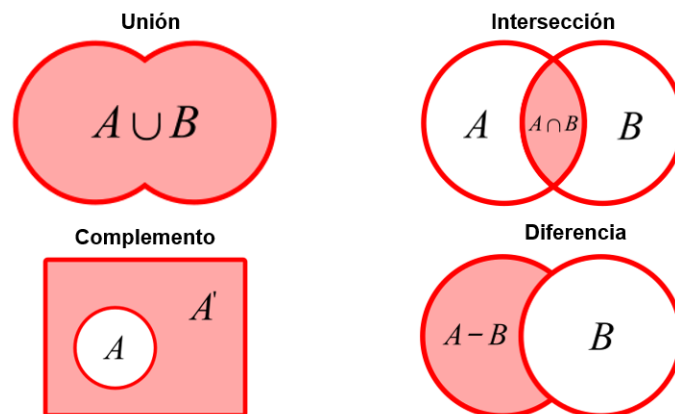


Ilustración 18. Operaciones de conjuntos

En resumen, para el TAD Conjunto, podemos establecer la siguiente especificación formal:

- **Nombre:** Conjunto
- **Valores posibles:** Conjunto vacío ($\emptyset$) o conjuntos con elementos del tipo base.
- **Operaciones principales:**
 - `add(elemento): void`
 - `remove(elemento): void`
 - `in(elemento): boolean`
 - `size(): integer`
- **Operaciones secundarias** (dependiendo de la implementación):
 - `union(Conjunto): Conjunto`
 - `intersection(Conjunto): Conjunto`

- o `difference(Conjunto): Conjunto`
- o `is_subset(Conjunto): boolean`
- **Tipos utilizados:** Tipo base además de la implementación del contenedor mediante arrays, punteros u otras estructuras según cada implementación concreta.

2. El TAD Bolsa

Una bolsa o *multiset*, es un contenedor similar a un conjunto que admite duplicados. Mientras que un conjunto responde a la pregunta "¿Está este elemento en la colección?", una bolsa responde a "¿Cuántas veces aparece este elemento en la colección?". Matemáticamente, una bolsa B sobre un dominio D puede verse como una función $B: D \rightarrow \mathbb{N}_0$ (números naturales incluyendo el cero) que asigna a cada elemento su multiplicidad (número de ocurrencias). Por ejemplo, si $B('a') = 3$, significa que el elemento 'a' aparece tres veces en la bolsa.

La diferencia fundamental entre conjuntos y bolsas se manifiesta en cómo tratan la duplicación:

- En un conjunto: $\{a, b, c\} \cup \{a\} = \{a, b, c\}$ (el elemento 'a' no se duplica)
- En una bolsa: $\{a, b, c\} \cup \{a\} = \{a, a, b, c\}$ (el elemento 'a' aparece dos veces)

La bolsa, al ser una generalización de los conjuntos que admite duplicados, requiere operaciones adicionales / modificadas para manejar la multiplicidad de elementos:

- **Adición de elementos:** `add(elemento)` – Añade una ocurrencia del elemento a la bolsa. A diferencia de los conjuntos, esta operación siempre tiene efecto, incrementando la multiplicidad del elemento si este ya estaba en la bolsa.
- **Eliminación de una ocurrencia:** `remove(elemento)` – Elimina una ocurrencia del elemento de la bolsa. Si el elemento tiene multiplicidad mayor que 1, se reduce en uno. Si la multiplicidad llega a 0, el elemento deja de estar en la bolsa.
- **Eliminación de todas las ocurrencias:** `remove_all(elemento)` – Elimina todas las ocurrencias de un elemento específico.
- **Consulta de multiplicidad:** `multiplicity(elemento)` – Devuelve el número de ocurrencias del elemento en la bolsa, 0 si el elemento no está en la bolsa.
- **Tamaño total:** `size()` – Devuelve el número total de elementos en la bolsa, contando duplicados. Por ejemplo, para la bolsa $\{a, a, b, c\}$, devuelve 4.
- **Número de elementos distintos:** `distinct_size()` – Devuelve el número de elementos distintos en la bolsa. Para la bolsa $\{a, a, b, c\}$, devuelve 3.

Para el TAD Bolsa, establecemos la siguiente especificación:

- **Nombre:** Bolsa
- **Valores posibles:** Bolsa vacía o bolsas con elementos del tipo base, cada uno con una multiplicidad ≥ 1 .
- **Operaciones principales:**
 - o `add(elemento: T): void`
 - o `remove(elemento: T): void`
 - o `remove_all(elemento: T): void`
 - o `multiplicity(elemento: T): integer`
 - o `size(): integer` (total con duplicados)
 - o `distinct_size(): integer` (elementos distintos)
- **Operaciones secundarias** (análogas a conjuntos pero considerando multiplicidades):
 - o `union(otra: Bolsa): Bolsa` (suma de multiplicidades)
 - o `intersection(otra: Bolsa): Bolsa` (mínimo de multiplicidades)
 - o `difference(otra: Bolsa): Bolsa` (resta de multiplicidades, no negativa)
- **Tipos utilizados:** Tipo base más los tipos de las estructuras utilizadas para almacenar pares (elemento, multiplicidad).

3. Implementación

El comportamiento de los conjuntos y las bolsas puede implementarse mediante diversas estructuras de datos subyacentes, cada una con sus ventajas y desventajas. La elección de la implementación depende de factores como el tipo de elementos a almacenar, el número máximo esperado de elementos, la frecuencia de las diferentes operaciones y los requisitos de eficiencia.

Implementación con arrays

Una implementación común de conjuntos utiliza arrays en memoria contigua. En este enfoque, que solo sirve para almacenar elementos ordinales (enteros, booleanos, caracteres, etc.) se define un array cuyos índices representan los posibles valores que pueden incluirse en el conjunto. Las características principales de este enfoque son:

- Acceso eficiente: Acceso $O(1)$ a cualquier elemento mediante indexación directa.
- Limitación de tipos: Solo funciona con tipos base que puedan usarse como índices de array, generalmente tipos ordinales (enteros, caracteres, enumerados).
- Tamaño fijo: Requiere dimensionamiento previo, que puede conllevar desperdicio de memoria si se asigna demasiada o falta de espacio si se subestiman las necesidades.
- Simplicidad: Implementación directa y fácil de entender.

F	F	F	T	T	T
0	1	2	3	4	5

Ilustración 19. Implementación de un conjunto mediante un array de booleanos

El tipo `set` integrado en Pascal sigue precisamente este modelo. Se trata de una implementación de conjuntos muy simple y limitada, pero útil en algunos casos. Pascal incluye un tipo `set` permite conjuntos homogéneos cuyo tipo base es cualquier ordinal (incluidos los enumerados), con la limitación de que todo conjunto está limitado a 256 elementos. Internamente, cada posible elemento se representa con un bit en un array de bits (bit array), donde un valor de 1 en un bit indica que el elemento está presente y 0 que el elemento está ausente. Esta implementación es extremadamente eficiente en cuanto al espacio cuando el universo de posibles elementos es pequeño y conocido de antemano. Un modelo de implementación sería:

```
type
  tipo_base = (value1, value2, ... value_n);
  tSet = set of tipo_base;
var
  my_set: tSet;
begin
  my_set := [value_i, value_j...];
  if some_value in my_set then do_something
end.
```

Un ejemplo práctico:

```
type
  language = (solidity, java, python, C++, ruby, go);
  languages = set of language;
var
  programmer_skills: languages;
begin
  programmer_skills := [java, python];
  if solidity in programmer_skills then
    writeln('The guy is a blockchain developer!');
end.
```

Implementación con listas enlazadas

Otra aproximación utiliza listas enlazadas para almacenar los elementos del conjunto. En esta implementación cada nodo de la lista contiene un elemento del conjunto, de modo que la lista contiene exactamente los elementos presentes (sin espacios para elementos ausentes). Si la implementación es para conjuntos, la lista no debe contener duplicados, si se trata de implementar bolsas, cada nodo necesita un campo adicional para la multiplicidad.

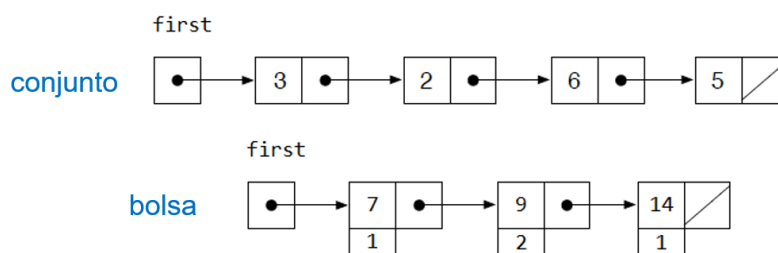


Ilustración 20. Implementación de conjuntos y bolsas con listas

Esta implementación tiene como principales ventajas un uso más eficiente de la memoria, pues solo se reserva memoria para los elementos realmente presentes. Además de ello, aporta mayor flexibilidad de tipos ya que permite almacenar cualquier tipo de dato, no solo tipos ordinales. Por otra parte, el acceso a los elementos no es tan eficiente, ya que encontrar un elemento específico requiere recorrer la lista, resultando en complejidad $O(n)$ en el peor caso. Para mejorar la eficiencia de búsqueda, se suele utilizar una lista ordenada, lo que permite búsquedas más rápidas (aunque aún $O(n)$ en el peor caso, $O(\log n)$ con búsqueda binaria si se implementa con acceso aleatorio).

Implementación con un árbol binario de búsqueda

Los árboles binarios de búsqueda (que veremos con mayor detalle más adelante) ofrecen un compromiso entre la eficiencia de acceso de los arrays y la flexibilidad de las listas. El acceso es más eficiente, pues las operaciones de búsqueda, inserción y eliminación tienen complejidad $O(\log n)$ en media, los elementos se mantienen ordenados automáticamente y aportan la flexibilidad de las listas para almacenar cualquier tipo con una relación de orden definida. Es en definitiva una muy buena opción de implementación que resulta particularmente útil cuando se necesitan operaciones de rango o recorrido ordenado de los elementos.

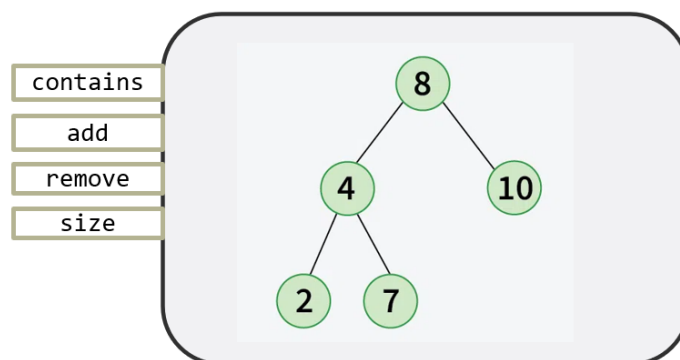


Ilustración 21. La interfaz externa es un conjunto pero el almacenamiento interno usa un BST

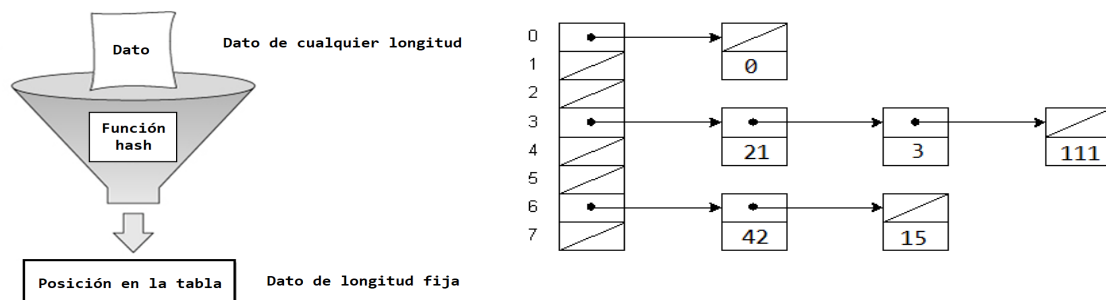
Implementación con Tablas Hash

Las tablas hash son una de las estructuras más eficientes para implementar conjuntos y bolsas, proporcionando en promedio operaciones de tiempo constante. El concepto fundamental es utilizar una función hash que transforma cada elemento en un índice dentro de un array.

Componentes de una Tabla Hash:

- Un array de tamaño fijo en el que cada posición puede contener múltiples elementos.
- Una función hash que transforma la clave de un elemento en índices del array. Se dice que “mapea” los índices de la tabla.
- Mecanismo de resolución de colisiones: estrategia para gestionar aquellos elementos que la función hash mapea a un mismo índice.

La implementación de las funciones esenciales sería en Pascal:



```
unit uHashTable;  
  
interface  
  uses SysUtils, uListaEnlazadaSimple;  
  const HASHTABLESIZE = 10;  
  type  
    tHashTable = array[0..HASHTABLESIZE-1] of tListaSimple;  
  
  procedure initialize(var table: tHashTable);  
  procedure add(var table: tHashTable; key: string);  
  procedure remove(var table: tHashTable; key: string);  
  function contains(table: tHashTable; key: string): boolean;  
  
implementation  
  function hash_function(key: string): integer;  
  // ...
```

En esta implementación, la tabla hash es un array de listas enlazadas, donde cada posición contiene una lista de aquellos elementos con la misma clave hash. Debe tenerse en cuenta que el tamaño de la tabla afecta el rendimiento.

La función hash, interna al TAD y por tanto no expuesta en la interfaz (contrato) del mismo, podría ser algo tan simple como lo siguiente o complicarse según necesidades:

```
function hash_function(key: string): integer;  
var  
  i, hash: integer;  
begin  
  hash := 0;  
  for i := 1 to length(key) do  
    hash := (hash + ord(key[i]));  
  hash_function := hash mod HASHTABLESIZE;  
end;
```

Esta función hash simple suma los valores ASCII de todos los caracteres de la clave para posteriormente aplicar el resto de la división entera con HASHTABLESIZE para obtener un índice válido en la tabla. El objetivo es que sea determinista y lo consigue: la misma clave siempre produce el mismo hash, pero tiene la desventaja de que puede producir muchas colisiones para ciertos patrones de claves (por ejemplo, las claves que compartan unos mismos caracteres ordenados de manera diferente colisionarán). En aplicaciones reales, se usan funciones hash más sofisticadas, como Jenkins, MurmurHash, CityHash, o incluso SHA-1 o MD5 (para aplicaciones criptográficas).

El resto de funciones de la tabla podrían implementarse en Pascal como sigue:

```

procedure initialize(var table: tHashTable);
var
  i: integer;
begin
  for i := 0 to HASHTABLESIZE-1 do
    uListaEnlazadaSimple.initialize(table[i]);
  end;

procedure add(var table: tHashTable; key: string);
var
  index: integer;
begin
  index := hash_function(key);
  if not uListaEnlazadaSimple.in_list(table[index], key) then begin
    uListaEnlazadaSimple.insert_at_end(table[index], key);
  end;
end;

procedure remove(var table: tHashTable; key: string);
var
  index: integer;
begin
  index := hash_function(key);
  uListaEnlazadaSimple.delete(table[index], key);
end;

function contains(table: tHashTable; key: string): Boolean;
var
  index: integer;
begin
  index := hash_function(key);
  contains := uListaEnlazadaSimple.in_list(table[index], key);
end;

```

Esta implementación de conjuntos usando tablas hash sigue una arquitectura en capas, lo que permite tanto la separación de responsabilidades (cada capa tiene responsabilidades claras). Ello facilita la reutilización de la tabla hash, que puede usarse para implementar otras estructuras, y redundante en una mayor facilidad de mantenimiento, pues los cambios en una capa no afectan necesariamente a las otras.

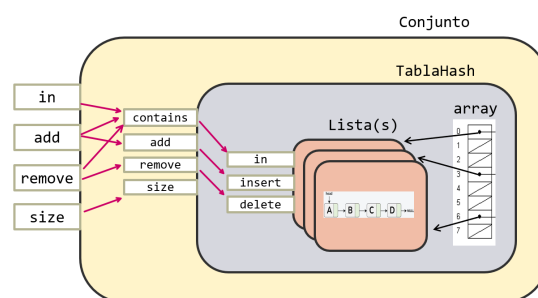


Ilustración 23. Arquitectura por capas de un conjunto implementado como Tabla Hash internamente

```

unit uConjunto;

interface
  uses uHashTable;

  type
    tConjunto = record
      table_hash: tHashTable;
      num_elems: integer;
    end;

  // Operaciones del conjunto
  procedure initialize(var c: tConjunto);
  procedure add(var c: tConjunto; key: string);
  procedure remove(var c: tConjunto; key: string);
  function is_in(c: tConjunto; key: string): boolean;
  function size(c: tConjunto): integer;

implementation

  procedure initialize(var c: tConjunto);
  begin
    uHashTable.initialize(c.table_hash);
    c.num_elems := 0;
  end;

  function is_in(c: tConjunto; key: string): boolean;
  begin
    is_in := uHashTable.contains(c.table_hash, key);
  end;

  function size(c: tConjunto): integer;
  begin
    size := c.num_elems;
  end;

  procedure add(var c: tConjunto; key: string);
  begin
    uHashTable.add(c.table_hash, key);
    s.num_elems := c.num_elems + 1;
  end;

  procedure remove(var c: tConjunto; key: string);
  begin
    uHashTable.remove(c.table_hash, key);
    s.num_elems := c.num_elems - 1;
  end;

```

4. Aplicaciones y uso de conjuntos y bolsas

Los conjuntos se utilizan cuando necesitamos trabajar con elementos sin duplicados y cuando importa saber si algo pertenece o no a un grupo. Son habituales en sistemas de control de acceso, en filtros de contenido y también en bases de datos, donde operaciones como los *JOIN* o las consultas con *IN* funcionan de forma muy parecida a las clásicas operaciones de teoría de conjuntos. También aparecen en grafos, representando vértices y aristas, y en compiladores, por ejemplo, para gestionar tablas de símbolos o analizar qué partes del programa son alcanzables. Incluso los sistemas de recomendación usan conjuntos para comparar intereses entre usuarios.

Por otro lado, las bolsas resultan útiles cuando necesitamos contar cuántas veces aparece algo. Son clave en el análisis de texto para obtener la frecuencia de palabras, en sistemas de votación para contabilizar votos, en inventarios donde importa cuántas unidades hay de cada producto, etc. También son muy comunes en el análisis de transacciones, por ejemplo, para saber qué artículos se compran con mayor frecuencia en un carrito.

Resumen

Los conjuntos y bolsas representan abstracciones fundamentales que modelan colecciones donde el orden no es relevante, pero la pertenencia (y en el caso de las bolsas, la multiplicidad) sí lo es. Aunque estructuras de datos conceptualmente similares, existe entre ellas una diferencia fundamental. En un conjunto, cada elemento puede aparecer como máximo una vez, es decir, no se permiten duplicados. En cambio, una bolsa admite la existencia de múltiples instancias de un mismo elemento, reflejando así su carácter multiconjunto.

En ambas estructuras, los elementos no están ordenados. Un elemento simplemente pertenece o no al conjunto o a la bolsa, sin que exista una relación de precedencia entre ellos, como ocurre en otras estructuras lineales.

Una de las aplicaciones más habituales de los conjuntos y las bolsas es la eliminación de elementos duplicados presentes en otro tipo de almacenamiento, como listas o arrays, lo que resulta especialmente útil en tareas de filtrado y normalización de datos.

El lenguaje Pascal proporciona un soporte directo, aunque muy básico y limitado, para el concepto de conjunto mediante el tipo predefinido `set`. Este mecanismo permite representar colecciones finitas de valores, pero carece de la flexibilidad necesaria para implementar conjuntos o bolsas de uso general. Para aplicaciones más generales, las tablas hash proporcionan tal vez el mejor equilibrio entre flexibilidad y rendimiento promedio.

Desde el punto de vista del diseño de tipos abstractos de datos (TAD), tanto el conjunto como la bolsa pueden entenderse como una interfaz que define un conjunto de operaciones (inserción, eliminación, pertenencia, etc.), mientras que el almacenamiento real de los elementos se delega en un contenedor interno. Dicho contenedor puede ser, según las necesidades, una lista (ordenada o no), un árbol, una tabla hash u otra estructura más especializada.

Las operaciones definidas por la interfaz del conjunto o de la bolsa suelen invocar internamente a las operaciones del contenedor subyacente, sin exponer los detalles de su implementación. Este enfoque constituye un ejemplo claro del principio de encapsulación, uno de los pilares fundamentales del diseño modular y de la programación estructurada y orientada a objetos.

El estudio de conjuntos y bolsas no solo es valioso por su utilidad práctica inmediata, sino también porque introduce conceptos importantes como funciones hash, resolución de colisiones y análisis probabilístico de algoritmos, que serán importantes más adelante en disciplinas más avanzadas.

Tema VIII - Árboles

Un árbol es una estructura de datos abstracta, no lineal, cuyos elementos, llamados nodos, se organizan de manera jerárquica. Es importante destacar su naturaleza recursiva: cualquier región de un árbol es a su vez un árbol, y por ello a menudo denominamos a cualquier región que abarca solo parte de los elementos “subárbol”.

Cada nodo de un árbol tiene un ascendiente directo, llamado padre, y puede tener varios descendientes, llamados hijos. El nodo que se encuentra en la cima de la jerarquía se denomina raíz, y es el único que no tiene un ascendiente. En cuanto a los nodos que no tienen descendientes, se les llama hojas o nodos terminales.

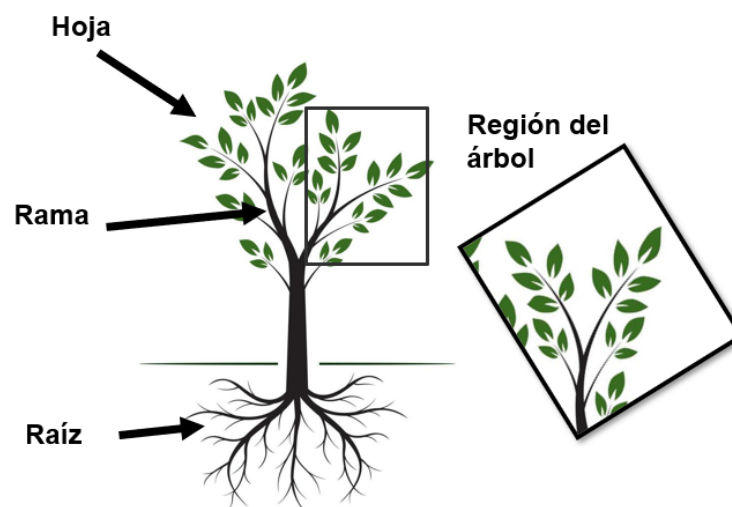


Ilustración 24. Elementos que conforman un árbol

A diferencia de los árboles del mundo real, las estructuras de datos árbol suelen representarse de forma invertida, con la raíz en la parte superior y los descendientes extendiéndose hacia abajo, lo que facilita la comprensión de su estructura jerárquica.

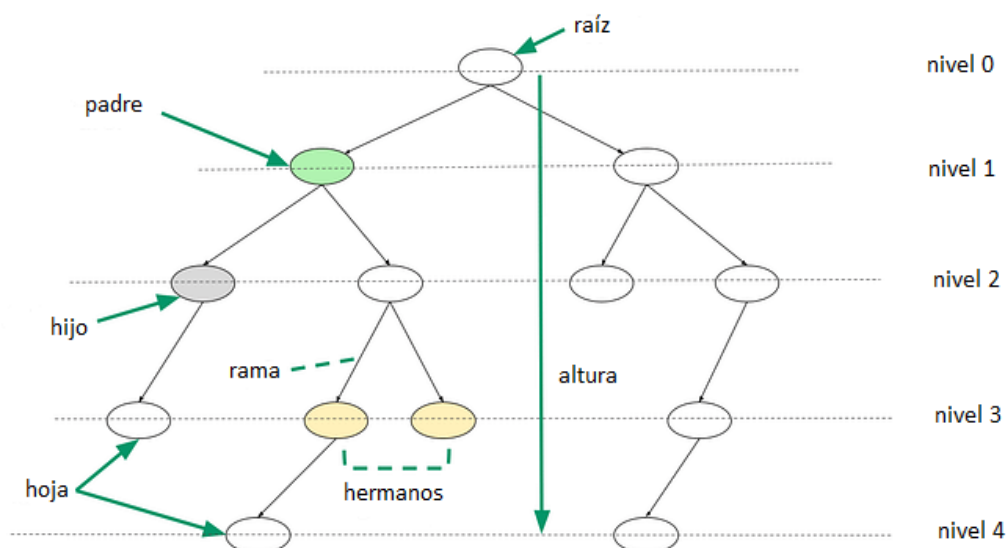


Ilustración 25. Representación de una estructura de datos tipo árbol

El número máximo de descendientes de un nodo puede ser fijo o variable. Un árbol n-ario es aquel en el que cada nodo tiene como máximo n hijos. Cada nodo está conectado a su padre por una única rama. Un camino es la secuencia de nodos entre dos nodos cualesquiera, y su longitud se mide por el número de ramas que lo componen. El nivel de un nodo se define como la distancia desde ese nodo hasta la raíz; la raíz se encuentra en el nivel 0.

Por otra parte, se denomina grado de un nodo al número de hijos que posee. El grado o aridad de un árbol es el máximo grado entre todos sus nodos. Otro concepto asociado es el de altura o profundidad de un árbol: el nivel de la hoja más alejada de la raíz más uno. Así, un árbol vacío tiene una altura de 0.

1. Tipos de árboles

Un **árbol general** es aquel en el que no existe un límite al número de hijos por nodo. Puede estar vacío o consistir en una raíz con múltiples nodos y ramas. Para implementar un árbol general se suele utilizar un manejador, como un puntero a la raíz, y una estructura de nodo que contenga información y enlaces a sus hijos. Existen dos enfoques comunes para gestionar los hijos: mediante una lista de hijos o mediante una representación de hijo izquierdo y hermanos.

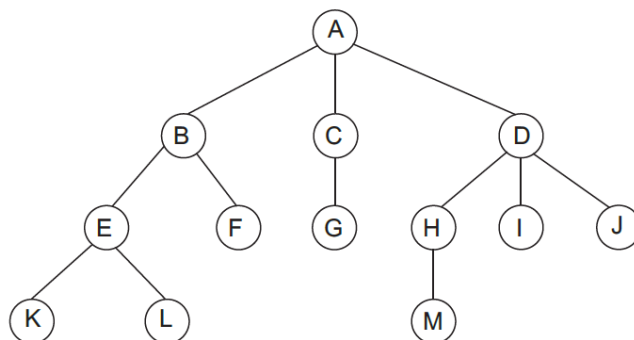


Ilustración 26. Árbol general

La implementación de sus elementos básicos en Pascal sería algo como lo siguiente:

```
tarbolgeneral = ^tnodo;  
  
tnodo = record  
    info: tipo_base;  
    hijos: lista_nodos;  
end;
```

Un **árbol binario** es un tipo especial de árbol en el que cada nodo tiene como máximo dos hijos, denominados hijo izquierdo e hijo derecho. Cada nodo contiene información y referencias a sus hijos. A partir del nodo raíz se puede acceder recursivamente a todos los demás nodos del árbol. Veremos su implementación completa más adelante.

Un **árbol binario de búsqueda (BST)** es un árbol binario con una propiedad adicional de orden: para cada nodo, todos los valores en su subárbol izquierdo son menores que el valor del nodo, y todos los valores en su subárbol derecho son mayores. Esta propiedad permite realizar búsquedas eficientes mediante un algoritmo similar a la búsqueda binaria en arreglos ordenados. Veremos su implementación completa más adelante.

2. El TAD Árbol binario

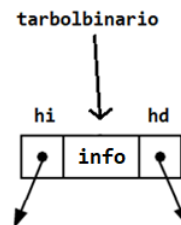
El Tipo Abstracto de Datos (TAD) Árbol Binario –donde cada nodo tiene como máximo dos hijos– se define por sus posibles valores (vacío u otros) y operaciones básicas como add, remove, is_empty,

in e initialize. Su implementación puede utilizar tipos base, arrays o punteros, dependiendo del lenguaje y del diseño, si bien cualquier implementación que no utilice memoria dinámica es poco común. En Pascal lo más habitual es una implementación con punteros tal y como veremos a continuación. Su especificación sería:

- **Nombre:** Árbol binario.
- **Valores posibles:** *vacío, otros.*
- **Operaciones:**
 - add(elemento)
 - remove(elemento)
 - is_empty()
 - in(elemento)
 - initialize()
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

La estructura básica de nodo en este árbol contiene información del tipo base y dos referencias a sus hijos, y su implementación tipo en Pascal sería la siguiente:

```
tnode = RECORD
  info: tipo_base;
  hi, hd: ^tnode;
end;
```



Al tener a lo sumo dos hijos por nodo, el árbol tiene un aspecto como el siguiente:

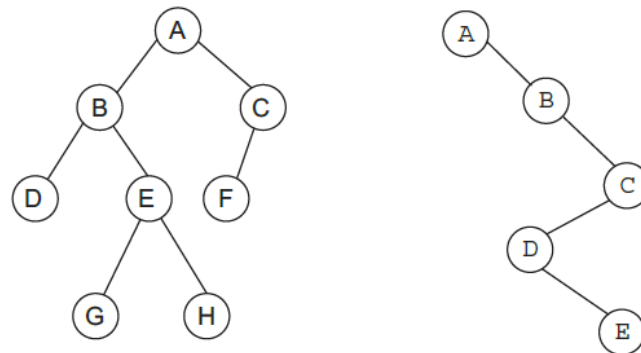


Ilustración 27. Árboles binarios

Como se ve, a partir de su nodo raíz se puede acceder a los demás nodos del árbol: de ahí el manejador al nodo raíz. Los nodos hijo izquierdo y derecho de cada nodo son a su vez la raíz de otros sub-árboles binarios, y así recursivamente hasta llegar a las hojas.

Su implementación completa sería:

```
unit uarbolbinario;

interface
type
  tbinaryTree = ^tnode;
  tnode = RECORD
    info: integer;
    hi, hd : tbinaryTree;
  end;
```

```

procedure initialize(var a:tbinaryTree);
function is_empty(a:tbinaryTree): boolean;
procedure add(var a:tbinaryTree; elem: integer);
function in_tree(a:tbinaryTree; elem: integer): boolean;
procedure remove(var a:tbinaryTree; elem: integer);

implementation

procedure initialize(var a:tbinaryTree);
begin
    a:= nil;
end;

function is_empty(a:tbinaryTree): boolean;
begin
    is_empty:= a=nil;
end;

function in_tree(a: tbinaryTree; elem : integer) : boolean;
begin
    if a = NIL then in_tree := FALSE
    else if a^.info = elem then in_tree := TRUE
    else in_tree := (in_tree(a^.hd, elem)) or (in_tree(a^.hi, elem))
end;

procedure add(var a :tbinaryTree; elem : integer);
begin
    if a = NIL then begin
        new(a);
        a^.info := elem;
        a^.hi := NIL;
        a^.hd := NIL;
    end
    else if a^.info < elem then add(a^.hd, elem)
    else if a^.info > elem then add(a^.hi, elem)
end;

procedure remove(var a:tbinaryTree; elem: integer);
begin
    // Diferentes estrategias, poco utilizado. Ver BST
end;

end.

```

Recorridos

En cualquier árbol binario, bien sea un árbol binario de búsqueda como veremos en la siguiente sección o bien un árbol binario general, es común realizar recorridos para visitar todos los nodos que lo componen. Así, recorrer un árbol binario implica visitar cada nodo exactamente una vez siguiendo una secuencia predeterminada. Existen dos categorías principales de recorridos: en profundidad y en anchura. El recorrido en anchura, el menos común de todos los posibles y también conocido como recorrido por niveles, visita los nodos horizontalmente: primero la raíz, luego todos sus hijos, después los hijos de esos hijos, y así sucesivamente. Puesto que una implementación eficiente de este recorrido se apoya en una estructura cola como auxiliar, su implementación queda fuera del alcance del presente texto y por tanto no la mostramos aquí.

Los recorridos en profundidad, mucho más frecuentes, son 3: inorden, preorden y postorden:

- *Preorden*: Se visita primero el nodo raíz, luego el subárbol izquierdo en preorden y finalmente el subárbol derecho en preorden.
- *Inorden*: Se recorre primero el subárbol izquierdo en inorden, luego se visita el nodo raíz y finalmente el subárbol derecho en inorden.

- **Postorden:** Se recorre el subárbol izquierdo en postorden, luego el subárbol derecho en postorden y finalmente se visita el nodo raíz.

A continuación mostramos el código en Pascal de estos recorridos para un árbol binario general. En estos ejemplos asumimos que la visita simplemente imprime el contenido del nodo pero obviamente dicha visita podría llevar a cabo cualquier otra operación con el contenido del nodo visitado, en función del objetivo que tuviese el recorrer el árbol (por ejemplo sumar todos los elementos del mismo, comprobar alguna condición, contar el número de elementos, etc.):

```

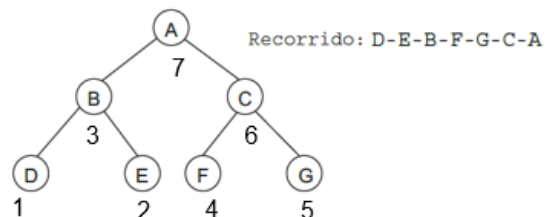
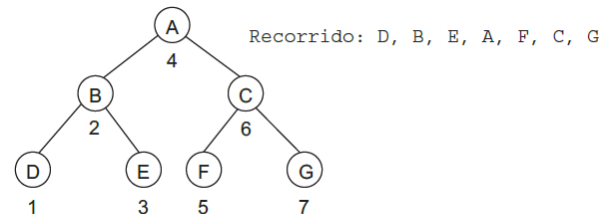
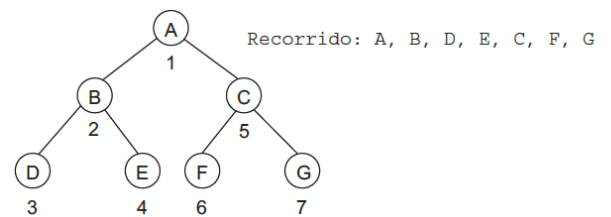
procedure visit(x:integer);
begin
  writeln(x)
end;

procedure preorde r(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    visit(a^.info);
    preorde(a^.hi);
    preorde(a^.hd)
  end
end;

procedure inorder(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    inorder(a^.hi);
    visit(a^.info);
    inorder(a^.hd)
  end
end;

procedure postorder(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    postorder(a^.hi);
    postorder(a^.hd);
    visit(a^.info);
  end
end;

```



3. El TAD Árbol Binario de búsqueda (BST)

Un árbol binario de búsqueda (BST) es un árbol binario con una propiedad adicional de orden: para cada nodo, todos los valores en su subárbol izquierdo son menores que el valor del nodo, y todos los valores en su subárbol derecho son mayores. Esta propiedad, y su estructura recursiva, permite realizar búsquedas eficientes mediante un algoritmo similar al de la búsqueda binaria.

Su especificación sería:

- **Nombre:** Árbol binario de búsqueda (BST, *Binary Search Tree*).
- **Valores posibles:** vacío, otros.
- **Operaciones:**
 - add(elemento)
 - remove(elemento)
 - is_empty()
 - in(elemento)
 - initialize()
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

La inserción en un árbol binario de búsqueda consiste en añadir cada nuevo elemento como hoja: si el árbol está vacío, se convierte en la raíz; si no, se compara con la clave de la raíz y se desciende recursivamente por el subárbol izquierdo si es menor o por el derecho si es mayor, hasta encontrar la posición adecuada. He aquí la principal diferencia con un árbol binario general.

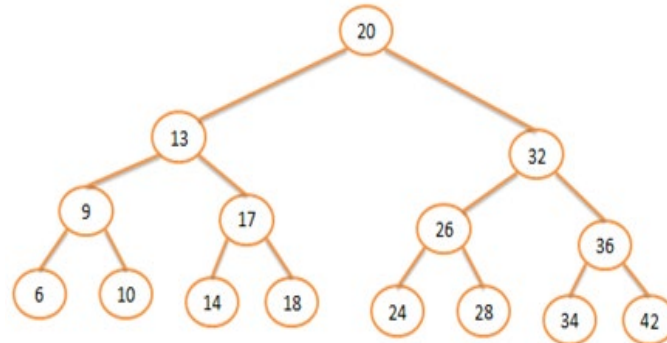


Ilustración 28. Árbol binario de búsqueda

En cuanto a la búsqueda, dado que los elementos están ordenados se sigue una lógica similar a la de inserción: comienza en la raíz y continúa hacia la izquierda o la derecha según la comparación, deteniéndose cuando encuentra la clave o llega a un subárbol vacío.

El borrado es más complicado ya que se debe mantener en todo momento la propiedad de orden, lo que requiere distinguir varios casos: no se hace nada si la clave no está; si el nodo es hoja, simplemente se elimina; si tiene un único hijo, se sustituye por dicho hijo; y si tiene dos hijos, se reemplaza su valor por el de un nodo sustituto (el máximo del subárbol izquierdo o el mínimo del derecho) y luego se elimina ese sustituto. Una variante es el borrado perezoso, que no elimina físicamente el nodo sino que lo marca como borrado, permitiendo posponer la limpieza real para más adelante cuando sea conveniente.

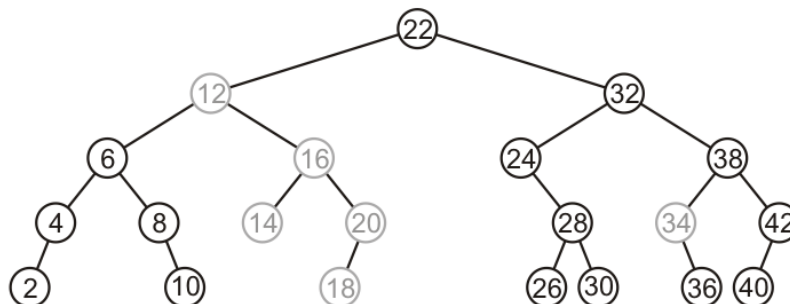


Ilustración 29. Borrado perezoso en BST

A continuación se muestra una implementación básica de un árbol binario de búsqueda en Pascal, que incluye definiciones de tipos, inicialización, inserción, comprobación de si está o no vacío y búsqueda. Obviamente a esta implementación podrían añadirse muchos otros comportamientos.

```

unit uarbolbinariobusqueda;

interface

type
  tBinarySearchTree = ^tnode;
  tnode = RECORD
    info : integer;
    hi, hd : tBinarySearchTree;
  end;

```

```

procedure initialize(var a:tBinarySearchTree);
function is_empty(a:tBinarySearchTree): boolean;
procedure add(var a:tBinarySearchTree; clave: integer);
function in_tree(a: tBinarySearchTree; clave: integer): boolean;
procedure remove(var a:tBinarySearchTree; x: integer);

```

implementation

```

procedure initialize(var a:tBinarySearchTree);
begin
    a:= NIL;
end;

function is_empty(a:tBinarySearchTree): boolean;
begin
    is_empty:= a=NIL;
end;

function in_tree(a: tBinarySearchTree; clave: integer): boolean;
begin
    if a = NIL then in_tree := FALSE
    else if a^.info < clave then in_tree := in_tree(a^.hd, clave)
    else if a^.info > clave then in_tree := in_tree(a^.hi, clave)
    else in_tree := TRUE
end;

procedure add(var a:tBinarySearchTree; clave: integer);
begin
    if a = NIL then begin
        new(a);
        a^.info := clave;
        a^.hi := NIL;
        a^.hd := NIL;
    end
    else if a^.info < clave then add(a^.hd, clave)
    else if a^.info > clave then add(a^.hi, clave)
end;

procedure remove(var a: tBinarySearchTree; x: integer);
var
    aux, ant: tBinarySearchTree;
begin
    if a <> NIL then
        if a^.info < x then remove(a^.hd, x)
        else if a^.info > x then remove(a^.hi, x)
        else
            begin
                aux := a;
                if a^.hi = NIL then a := a^.hd
                else if a^.hd = NIL then a := a^.hi
                else
                    begin
                        aux:= a^.hi;
                        while aux^.hd <> NIL do
                            begin
                                ant:= aux;
                                aux:= aux^.hd;
                            end;
                        if a^.hi = aux then a^.hi:= NIL else ant^.hd:= NIL;
                        a^.info := aux^.info;
                    end;
                dispose(aux)
            end
    end;
end.

```

4. Árboles AVL

La eficiencia de las operaciones en un BST depende en gran medida del equilibrio del árbol. Un árbol perfectamente equilibrado ofrece búsquedas en tiempo logarítmico, mientras que un árbol degenerado puede degradarse a tiempo lineal. Los árboles AVL, nombrados así por sus creadores Adelson-Velsky y Landis, son árboles binarios de búsqueda autobalanceables que mantienen un equilibrio estricto tras cada inserción o eliminación.



Ilustración 30. La búsqueda en un árbol es óptima solo si el árbol está equilibrado

Cada nodo en un árbol AVL almacena un factor de equilibrio, definido como la altura del subárbol derecho menos la altura del subárbol izquierdo. Este factor debe estar entre -1 y 1 para que el árbol se considere equilibrado. La altura de un árbol vacío se define como -1. Tras una inserción o eliminación, se recorre el camino desde el nodo modificado hacia la raíz, actualizando los factores de equilibrio. Si algún nodo presenta un factor de equilibrio de -2 o 2, se detecta un desequilibrio que debe corregirse mediante rotaciones que se llevan a cabo de abajo hacia arriba.

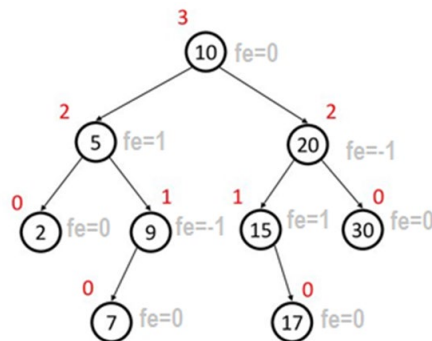


Ilustración 31. AVL: en rojo la altura del nodo, en gris el factor de equilibrio

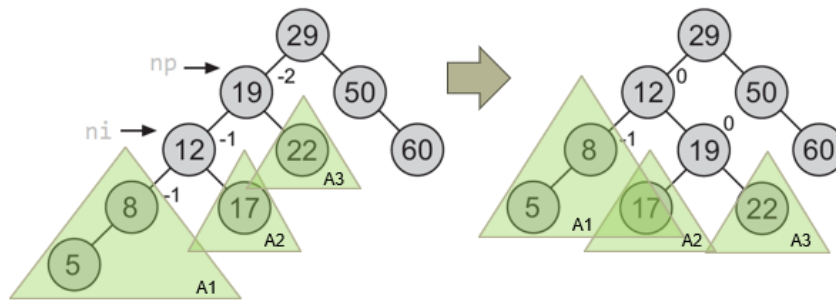
Las rotaciones son operaciones que reestructuran el árbol para restaurar el equilibrio. Existen cuatro casos principales de desequilibrio, identificados según la posición de la inserción respecto al nodo desequilibrado: LL, RR, LR y RL.

- **LL (Rotación simple derecha - RSD):** Ocurre cuando la inserción se realiza en el subárbol izquierdo del hijo izquierdo del nodo desequilibrado. Se soluciona con una rotación simple a la derecha.
- **RR (Rotación simple izquierda - RSI):** Ocurre cuando la inserción se realiza en el subárbol derecho del hijo derecho del nodo desequilibrado. Se soluciona con una rotación simple a la izquierda.
- **LR (Rotación doble izquierda-derecha):** Ocurre cuando la inserción se realiza en el subárbol derecho del hijo izquierdo del nodo desequilibrado. Se soluciona con una rotación simple a la izquierda seguida de una rotación simple a la derecha.
- **RL (Rotación doble derecha-izquierda):** Ocurre cuando la inserción se realiza en el subárbol izquierdo del hijo derecho del nodo desequilibrado. Se soluciona con una rotación simple a la derecha seguida de una rotación simple a la izquierda.

Ejemplo de rotaciones

Supongamos un nodo np desequilibrado con factor -2, y su hijo izquierdo ni. La rotación RSD (rotación simple a la derecha) reasigna los punteros de la siguiente manera:

```
// np es el nodo problema, ni es su hijo izquierdo
ni := np^.hi;
np^.hi := ni^.hd;
ni^.hd := np;
np := ni;
```



si (altura hi - altura hd = 2) entonces
si (altura hi de hi >= altura hd de hi) entonces RSD

```
np^.hi := ni^.hd;
ni^.hd := np;
np := ni;
```

Ilustración 32. Ejemplo de rotación RSD

Para el caso de una rotación doble RL, el movimiento involucra tres nodos: el nodo problema np, su hijo derecho n1 y el hijo izquierdo de n1, n2. La secuencia de operaciones en este caso sería:

```
// Rotación simple derecha sobre n1
n1^.hi := n2^.hd;
n2^.hd := n1;

// Rotación simple izquierda sobre np
np^.hd := n2^.hi;
n2^.hi := np;
np := n2;
```

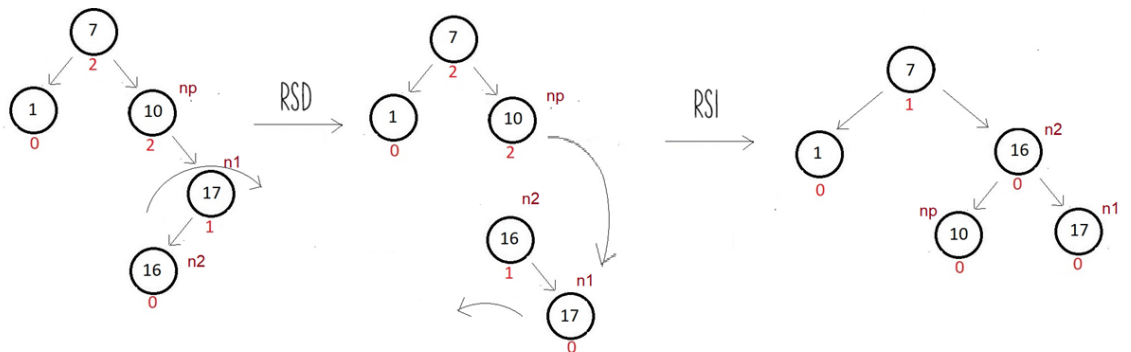


Ilustración 33. Ejemplo de rotación doble RL

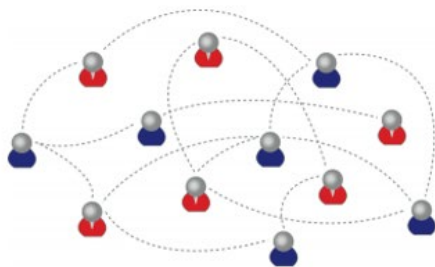
Para mayor detalle sobre los procesos de reequilibrio de árboles AVL, se recomienda consultar el capítulo 17 “Árboles de búsqueda equilibrados” del libro “Estructuras de datos en C++” de Luis Joyanes, Lucas Sánchez e Ignacio Zahonero.

Tema IX. Grafos

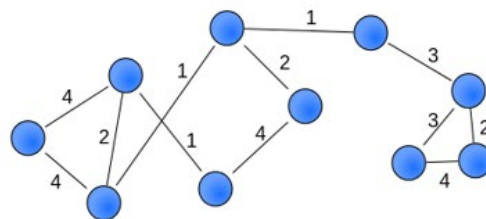
Un grafo es una estructura de datos abstracta, no lineal, compuesta por un conjunto de vértices o nodos y un conjunto de aristas o arcos que los conectan. Formalmente, un grafo G se define como un par $G = (V, A)$ donde V es el conjunto de vértices y A el conjunto de arcos. Un arco es un par ordenado o no ordenado (v_1, v_2) de vértices de V . Los grafos permiten modelar relaciones en dominios diversos, como mapas, redes de transporte, redes sociales, redes biológicas y de comunicación.

Existen múltiples clasificaciones de grafos según sus características:

- **Grafos dirigidos vs. no dirigidos:** en los dirigidos las aristas tienen orientación ($u \rightarrow v$), mientras que en los no dirigidos las relaciones son bidireccionales y sin sentido definido.
- **Grafos binarios vs. ponderados:** los binarios solo indican si existe una arista, mientras que los ponderados asignan un peso numérico a cada conexión.

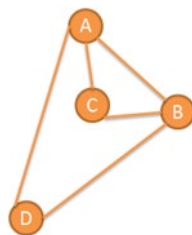


Grafo binario

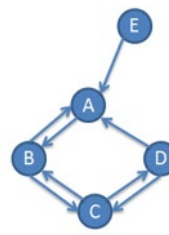


Grafo ponderado

- **Grafos conexos vs. no conexos:** un grafo conexo permite llegar entre cualquier par de vértices, mientras que en uno no conexo existen componentes aislados.
- **Grafos unicapa vs. multicapa:** los unicapa representan un único tipo de relación, mientras que los multicapa incorporan varias relaciones distribuidas en distintas capas.
- **Grafos unipartitos vs. bipartitos:** los unipartitos no necesitan dividir los vértices, mientras que los bipartitos separan los nodos en dos conjuntos disjuntos conectados entre sí.



no dirigido

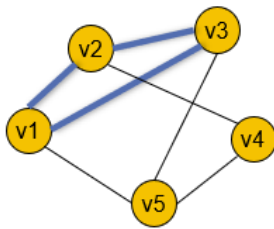


dirigido

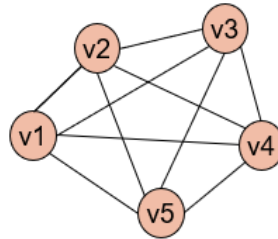
- **Grafos homogéneos vs. heterogéneos:** los homogéneos contienen nodos y aristas del mismo tipo, mientras que los heterogéneos combinan tipos distintos.
- **Grafos estáticos vs. dinámicos:** los estáticos mantienen su estructura en el tiempo, mientras que los dinámicos cambian añadiendo o quitando nodos y aristas.
- **Grafos simples vs. multigrafos:** los simples no permiten aristas paralelas ni bucles, mientras que los multigrafos sí admiten múltiples aristas entre el mismo par de nodos.
- Etc.

Antes de profundizar en el estudio e implementación de los grafos, es necesario conocer algunos conceptos básicos:

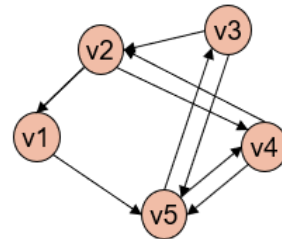
- **Grado de un vértice:** Número de aristas incidentes en él. En grafos dirigidos, se distingue entre grado saliente (out-degree) y grado entrante (in-degree).
- **Camino:** Secuencia de vértices conectados por aristas desde un origen hasta un destino.
- **Camino simple:** Camino donde todos los nodos son distintos.
- **Longitud de un camino:** Número de aristas que lo componen.
- **Camino cerrado:** Camino donde el primer y último vértice coinciden (también llamado ciclo o bucle).
- **Camino mínimo:** El camino más corto entre dos vértices, en términos de número de aristas o suma de pesos.



Bucle de longitud 3



Grafo completo



Grafo fuertemente conexo

- **Adyacencia:** Dos vértices son adyacentes si están conectados por una arista.
- **Grafo completo:** Contiene todas las posibles aristas entre sus vértices.
- **Grafo fuertemente conexo:** En un digrafo, existe un camino dirigido desde cualquier nodo a cualquier otro.

1. El TAD Grafo

El TAD Grafo se especifica con operaciones básicas como inserción y eliminación de nodos y aristas, verificación de pertenencia, cálculo de grados y detección de adyacencia. Dependiendo del tipo de grafo (dirigido, valorado, etc.), las operaciones pueden variar.

- **Nombre:** Grafo dirigido / valorado / etc.
- **Valores posibles:** *vacío, otros (según elementos).*
- **Operaciones:**
 - `insert_node()`
 - `insert_edge()`
 - `delete_node()`
 - `delete_edge()`
 - `is_empty(graph)`
 - `in_graph(node) / in_graph(arc)`
 - `degree/indegree/outdegree(node)`
 - `adjacent(node, node)`
- **Uso de otros tipos:** tipo base, punteros, arrays

Además de las que aquí aparecen, en aplicaciones como análisis de redes sociales, es común calcular métricas avanzadas como diámetro, coeficiente de clustering, centralidad (betweenness, closeness, eigenvector), densidad, k-core, componentes conexos, o extraer subgrafos.

En cuanto a su representación, los grafos pueden representarse mediante estructuras de datos estáticas o dinámicas, cada una con ventajas y desventajas según el contexto.

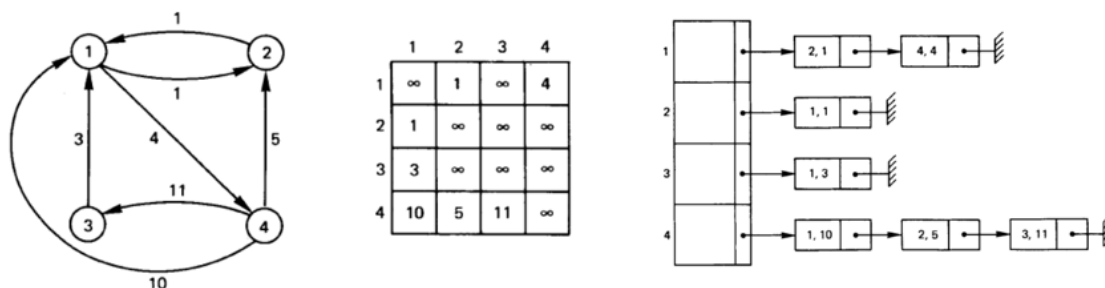


Ilustración 34. Dos representaciones del grafo de la izquierda: matriz de adyacencia vs. lista de adyacencia

Dentro de las representaciones estáticas, la más común es la que cnlleva una matriz de adyacencia + vector nodos, mientras que en las dinámicas, es muy común la lista de adyacencia, pero también la lista de arcos y otras, dependiendo de la aplicación del grafo en sí.

Representación mediante matriz de adyacencia

Una matriz de adyacencia es una tabla cuadrada de tamaño $|V| \times |V|$, donde la celda (i, j) indica la relación entre el nodo i y el nodo j . En grafos binarios, el valor es 1 si existe una arista y 0 en caso contrario. En grafos ponderados, la celda contiene el peso de la arista. La diagonal representa autoenlaces (bucles). En grafos no dirigidos, la matriz es simétrica.

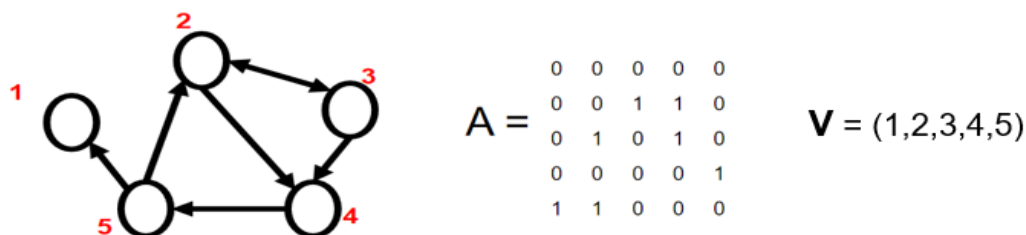


Ilustración 35. Representación mediante matriz de adyacencia

Sus ventajas principales es el acceso rápido $O(1)$ para verificar si existe una arista y la fácil implementación que supone especialmente para grafos densos. En cuanto a sus desventajas, debe considerarse el consumo de memoria que resulta ineficiente para grafos dispersos, así como el hecho de que la inserción o eliminación de nodos está limitado pues no es posible redimensionar la matriz.

Representación dinámica

La representación dinámica más común es una lista de adyacencia, donde se asocia a cada nodo una lista de sus vecinos. En grafos dirigidos, se almacenan solo los sucesores; en no dirigidos, todos los adyacentes. Para grafos ponderados, cada entrada en la lista incluye el nodo destino y el peso.

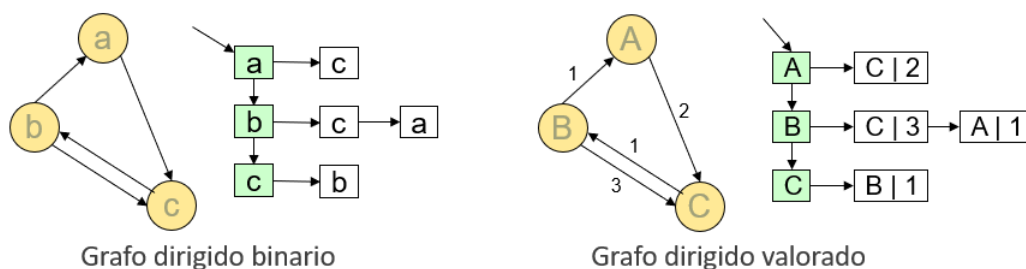
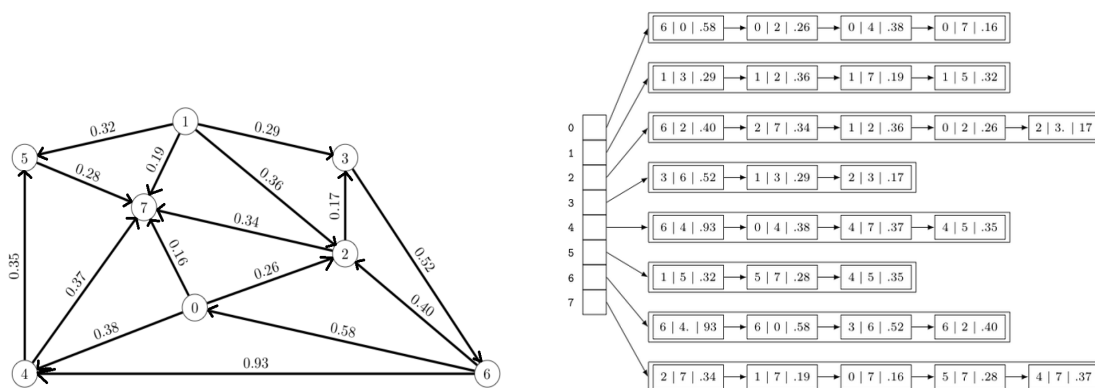


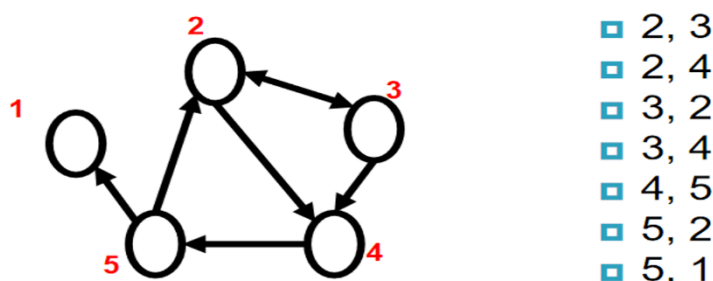
Ilustración 36. Representación de un grafo mediante lista de adyacencia

Las ventajas de esta representación con, por una parte, que resulta una gestión de memoria mucho más eficiente, ideal para grafos dispersos. Además, las listas facilitan iterar sobre los vecinos de un nodo. En contraprestación, verificar la existencia de una arista puede tomar un tiempo mayor $O(\text{grado}(v))$ y por supuesto se trata de una implementación más compleja que la matriz.

Aunque la implementación anterior es la más habitual, existen variantes de la lista de adyacencia. Por ejemplo, puede utilizarse una lista de adyacencia con arcos asociados en la que, en lugar de vincular a cada nodo únicamente los vértices a los que apunta, se le asocian todos los vértices que tienen aristas de entrada o de salida respecto a él. En este tipo de representación se asumen redundancias, lo cual puede ser útil dependiendo de la aplicación concreta.



Finalmente la lista de arcos es otra representación dinámica, donde se emplea una lista simple de todas las aristas, representadas como pares (origen, destino) o tripletas (origen, destino, peso) en grafos ponderados. Es una representación minimalista pero poco eficiente para consultas de adyacencia, por ejemplo.



En grafos poco densos, es decir, aquellos que contienen un gran número de nodos pero solo unas pocas conexiones entre ellos, la lista de adyacencia suele ser la opción más adecuada para su representación. Este tipo de estructura resulta especialmente eficiente porque solo almacena las aristas realmente existentes, lo que reduce significativamente el consumo de memoria. La mayoría de los grafos derivados de interacciones sociales —como redes de amistad, colaboración o comunicación— presentan justamente este patrón de baja densidad, y por eso las listas de adyacencia son la representación habitual en estos contextos. Además, este formato facilita operaciones como la determinación de caminos, la exploración de vecinos o la implementación de algoritmos de búsqueda como BFS o DFS, que funcionan de manera muy natural sobre estas listas.

Por el contrario, cuando se trabaja con grafos densos que contienen pocos nodos pero un número elevado de conexiones entre ellos, la representación mediante matrices de adyacencia suele ser la alternativa más práctica. En estos casos, el coste de almacenar una matriz completa es aceptable

y se obtiene a cambio un acceso inmediato para comprobar la existencia de una arista entre dos nodos concretos. Este acceso constante resulta especialmente útil cuando se necesita consultar rápidamente los vecinos de un nodo o cuando se aplican algoritmos que requieren realizar muchas comprobaciones de adyacencia de forma repetida. Por ello, en grafos pequeños y altamente conectados, la matriz de adyacencia es una solución eficiente y conceptualmente sencilla.

La siguiente tabla muestra una comparación de eficiencia entre representaciones:

Representación	espacio	añadir arco / nodo	¿arco entre n y m?	Iterar el vecindario de un nodo
Lista de arcos	A	1	A	A
Matriz de adyacencia	V ²	1	1	V
Lista de adyacencia	V+A	1	A	A

2. Implementación de un digrafo valorado con matriz de adyacencia

Supongamos un grafo dirigido (digrafo) y ponderado con nodos etiquetados {A,B,C,D}. La matriz de adyacencia se implementa como una matriz bidimensional, donde matrix[i][j] almacena el peso de la arista del nodo i al nodo j, o un valor especial (como ∞) si no existe una arista entre ellos.

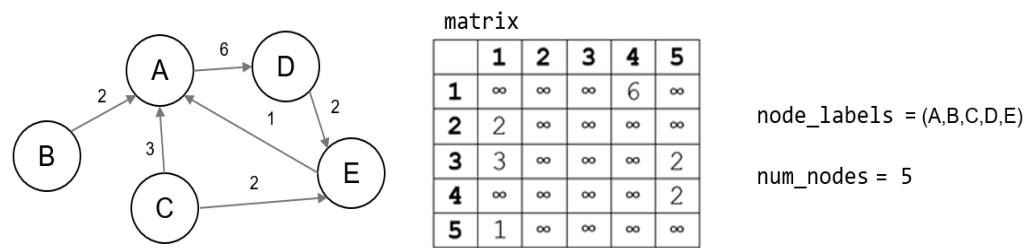


Ilustración 39. Ejemplo de grafo dirigido (digrafo) valorado

Una implementación de una unidad Pascal que diera soporte a este modelo sería la siguiente:

```

unit uMatrixEdgeWeightedDigraph;

interface
const
    MAX_NODES = 100;

type
    tadjacency_matrix = array [1..MAX_NODES,1..MAX_NODES] of string;
    edgeWeightedDigraph = record
        matrix :      tadjacency_matrix;
        node_labels : array[1..MAX_NODES] of string;
        num_nodes:    integer;
    end;

    procedure initialize(var g: edgeWeightedDigraph);
    procedure add_node(var g: edgeWeightedDigraph; node:string);
    procedure add_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
    procedure delete_node(var g: edgeWeightedDigraph; node:string);
    procedure delete_edge(var g: edgeWeightedDigraph; n1,n2:string);
    function is_empty(g: edgeWeightedDigraph): boolean;
    function node_in_graph(g: edgeWeightedDigraph; node: string): boolean;
    function edge_in_graph(g: edgeWeightedDigraph; n1,n2:string): boolean;
    function edge_weight(g: edgeWeightedDigraph; n1,n2:string): string;

```

```
function adjacent(g: edgeWeightedDigraph; n1,n2:string): boolean;
function has_self_loops(g: edgeWeightedDigraph): boolean;
```

implementation

```
procedure initialize(var g: edgeWeightedDigraph);
var i,j:integer;
begin
    g.num_nodes:=0;
    for i:= 1 to MAX_NODES do begin
        g.node_labels[i]:= '';
        for j:= 1 to MAX_NODES do g.matrix[i,j]:= 'inf';
    end
end;

function get_index(g:edgeWeightedDigraph; node:string): integer;
{ retorna el índice en el array de un nodo por su etiqueta }
var i:integer;
begin
    get_index:= -1;
    i:=1;
    while (get_index=-1) and (i<=g.num_nodes) do begin
        if g.node_labels[i] = node then get_index := i;
        i:= i+1;
    end
end;

procedure add_node(var g: edgeWeightedDigraph; node:string);
begin
    if (g.num_nodes<MAX_NODES) and (not node_in_graph(g,node)) then begin
        g.num_nodes:= g.num_nodes + 1;
        g.node_labels[g.num_nodes]:= node;
    end;
end;

procedure add_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
begin
    if node_in_graph(g,n1) and node_in_graph(g,n2) then
        g.matrix[get_index(g,n1),get_index(g,n2)]:= weight;
end;

procedure delete_node(var g: edgeWeightedDigraph; node:string);
var i,k:integer;
begin
    if node_in_graph(g,node) then begin
        k:= get_index(g,node);
        if k=g.num_nodes then // caso especial: eliminar el último nodo
            for i:=1 to g.num_nodes-1 do begin
                g.matrix[i,g.num_nodes]:= 'inf';
                g.matrix[g.num_nodes,i]:= 'inf';
            end
        else begin // eliminar sobrescribiendo su información con la del último
            for i:=1 to g.num_nodes-1 do begin
                g.matrix[i,k]:=g.matrix[i,g.num_nodes];
                g.matrix[i,g.num_nodes]:= 'inf';
                g.matrix[k,i]:=g.matrix[g.num_nodes,i];
                g.matrix[g.num_nodes,i]:= 'inf';
            end;
            g.matrix[k,k]:=g.matrix[g.num_nodes,g.num_nodes];
            g.node_labels[k]:= g.node_labels[g.num_nodes];
        end;
        g.matrix[g.num_nodes,g.num_nodes]:= 'inf';
        g.num_nodes:=g.num_nodes-1;
    end;
end;
```

```

procedure delete_edge(var g: edgeWeightedDigraph; n1,n2:string);
begin
  if node_in_graph(g,n1) and node_in_graph(g,n2) then
    g.matrix[get_index(g,n1),get_index(g,n2)]:= 'inf';
  end;

function is_empty(g: edgeWeightedDigraph): boolean;
begin
  is_empty:= g.num_nodes=0
end;

function node_in_graph(g: edgeWeightedDigraph; node: string): boolean;
begin
  node_in_graph:= get_index(g,node)<>-1;
end;

function edge_in_graph(g: edgeWeightedDigraph; n1,n2:string): boolean;
var node1_index,node2_index: integer;
begin
  node1_index:= get_index(g,n1);
  node2_index:= get_index(g,n2);
  edge_in_graph:= (node1_index <> -1) and
    (node2_index <> -1) and
    (g.matrix[node1_index,node2_index]<>'inf');
end;

function edge_weight(g: edgeWeightedDigraph; n1,n2:string): string;
// PRE: Edge exists in graph
begin
  edge_weight:= g.matrix[get_index(g,n1),get_index(g,n2)];
end;

function adjacent(g: edgeWeightedDigraph; n1,n2:string): boolean;
begin
  adjacent:= edge_in_graph(g,n1,n2);
end;
end.

```

3. Implementación de un grafo valorado con listas de adyacencia

Para implementar un grafo dirigido (digrafo) ponderado, como el de la implementación anterior, mediante listas de adyacencia, necesitaríamos dos tipos de nodo: el nodo que almacena la información (node) y el nodo que da soporte a los nodos adyacentes a uno dado (adjacency_node). La siguiente figura muestra esquemáticamente la estructura que da soporte a esta implementación.

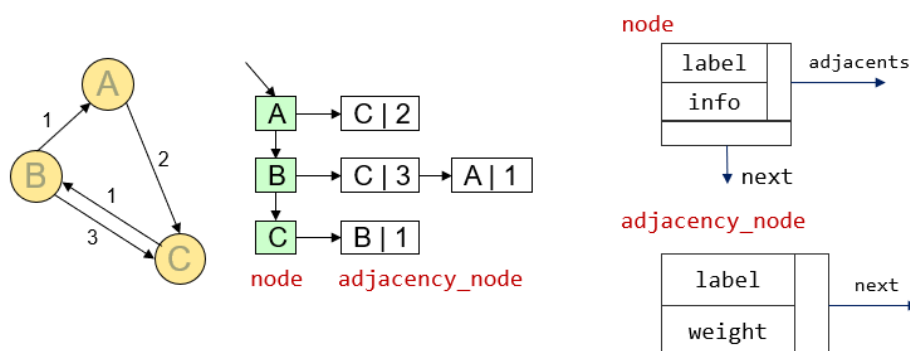


Ilustración 40. Implementación de un digrafo valorado con listas de adyacencia

La implementación con listas de adyacencia podemos incluirla en un nuevo TAD que en Pascal vendría representado por una unidad como la siguiente:

```

unit uAdjacencyListEdgeWeightedDigraph;

interface

type
  tAdjacencyNode = record
    nodeTo_label, weight: string;
    next: ^tadjacencyNode;
  end;
  tNode = record
    node_label: string;
    info: integer;
    adjacents: ^tAdjacencyNode;
    next: ^tNode;
  end;
  edgeWeightedDigraph = ^tNode;

procedure initialize(var g: edgeWeightedDigraph);
function is_empty(g: edgeWeightedDigraph): boolean;
procedure add_node(var g: edgeWeightedDigraph; label:string; info: integer);
procedure add_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
function get_node(g: edgeWeightedDigraph; n:string): edgeWeightedDigraph;
procedure delete_node(var g: edgeWeightedDigraph; label:string);
procedure delete_edge(var g: edgeWeightedDigraph; n1_label,n2_label:string);
function node_in_graph(g: edgeWeightedDigraph; label: string): boolean;
function edge_in_graph(g: edgeWeightedDigraph; n1, n2:string): boolean;

implementation

procedure initialize(var g: edgeWeightedDigraph);
begin
  g:= NIL
end;

function is_empty(g: edgeWeightedDigraph): boolean;
begin
  is_empty:= g=NIL
end;

procedure add_node(var g: edgeWeightedDigraph; label:string; info: integer);
var new_node:^tNode;
begin
  if not node_in_graph(g,label) then begin
    new(new_node);
    new_node^.node_label:= label;
    new_node^.info:= info;
    new_node^.adjacents:=NIL;
    new_node^.next:=g;
    g:= new_node;
  end;
end;

function get_node(g: edgeWeightedDigraph; n:string): edgeWeightedDigraph;
begin
  get_node:= g;
  while (get_node<>NIL) and (get_node^.node_label<>n) do
    get_node:= get_node^.next;
  end;
end;

function node_in_graph(g: edgeWeightedDigraph; label: string): boolean;
begin
  node_in_graph:= get_node(g,label)<>NIL
end;

```

```

procedure add_edge(var g: edgeWeightedDigraph; n1,n2,weight: string);
var
    origen, destino: ^tNode;
    adj_node: ^tAdjacencyNode;
begin
    origen:= get_node(g,n1);
    destino:= get_node(g,n2);
    if (origen <> NIL) and (destino <> NIL) then begin
        new(adj_node);
        adj_node^.nodeTo_label:= n2;
        adj_node^.weight:= weight;
        adj_node^.next:= origen^.adjacents;
        origen^.adjacents:= adj_node;
    end;
end;

function edge_in_graph(g: edgeWeightedDigraph; n1,n2: string): boolean;
var origen: ^tNode;
    destino: ^tAdjacencyNode;
begin
    origen:= get_node(g,n1);
    if origen=NIL then edge_in_graph:= FALSE
    else begin
        destino:= origen^.adjacents;
        while (destino<>NIL) and (destino^.nodeTo_label<>n2) do
            destino:= destino^.next;
        edge_in_graph:= destino<>NIL;
    end
end;

procedure delete_adjacent_list(var g: edgeWeightedDigraph;
origen:edgeWeightedDigraph);
// Procedure interno para eliminar la lista de adyacentes de un nodo
var aux,ant: ^tAdjacencyNode;
begin
    if origen<>NIL then begin
        aux:= origen^.adjacents;
        while aux<>NIL do begin
            ant:= aux;
            aux:= aux^.next;
            dispose(ant);
        end;
        origen^.adjacents:= NIL;
    end;
end;

procedure delete_adjacent_node(var g: edgeWeightedDigraph;
origen:edgeWeightedDigraph; toRemoveNode:string);
// Procedure interno para eliminar un adyacente en la lista de un nodo
var aux,ant: ^tAdjacencyNode;
begin
    if origen<>NIL then begin
        aux:= origen^.adjacents;
        while (aux<>NIL) and (aux^.nodeTo_label <> toRemoveNode) do begin
            ant:= aux;
            aux:= aux^.next;
        end;
        if aux<> NIL then begin // Si había arcos a toRemoveNode eliminarlos
            if aux=origen^.adjacents then // primero en la lista de adyacentes
                origen^.adjacents:= aux^.next
            else
                ant^.next:= aux^.next;
            dispose(aux);
        end;
    end;
end;
end;

```

```

procedure delete_node(var g: edgeWeightedDigraph; node_label:string);
var nodo_a_borrar, aux: ^tNode;
begin
    nodo_a_borrar:= get_node(g,node_label);
    if (nodo_a_borrar <> NIL) then begin
        // borrar toda su lista de adyacencia
        delete_adjacent_list(g,nodo_a_borrar);
        // borrar el nodo
        if g=nodo_a_borrar then // borrar el primer nodo de la lista
            g:= nodo_a_borrar^.next
        else begin // el nodo no es el primero de la lista de adyacencia
            aux:= g;
            while aux^.next<>nodo_a_borrar do
                aux:= aux^.next;
            aux^.next:= nodo_a_borrar^.next
        end;
        dispose(nodo_a_borrar);
        // borrar arcos con este nodo en otras listas de adyacencia
        aux:= g;
        while aux<>NIL do begin
            delete_adjacent_node(g,aux,node_label);
            aux:= aux^.next;
        end;
    end;
end;

procedure delete_edge(var g: edgeWeightedDigraph; n1_label,n2_label:string);
var origen: ^tNode;
    aux,ant: ^tAdjacencyNode;
begin
    if edge_in_graph(g,n1_label,n2_label) then begin
        origen:= get_node(g,n1_label);
        aux:= origen^.adjacents;
        while aux^.nodeTo_label<>n2_label do begin
            ant:=aux;
            aux:=aux^.next;
        end;
        if aux=ant then origen^.adjacents:= aux^.next
        else ant^.next:= aux^.next;
        dispose(aux);
    end
end;

end.

```

Como se ve, la implementación es bastante más compleja que la implementación con matrices de adyacencia, e incluye algunos procedimientos internos a la unidad Pascal que no se exportan, tales como `delete_adjacent_list` o `delete_adjacent_node` que dan soporte a operaciones que sí se ofrecen al usuario del TAD.

4. Recorridos

Recorrer un grafo implica visitar todos los nodos alcanzables desde un nodo inicial, sin repeticiones. Los algoritmos principales son el recorrido en anchura (BFS) y el recorrido en profundidad (DFS). Ambos requieren mantener un registro de nodos visitados para evitar ciclos.

Recorrido en anchura (BFS)

El *Breadth-First Search* (BFS) o Búsqueda en Anchura, es un algoritmo clásico de recorrido en grafos y árboles que explora el espacio disponible por niveles. Parte de un nodo inicial (llamado fuente) y

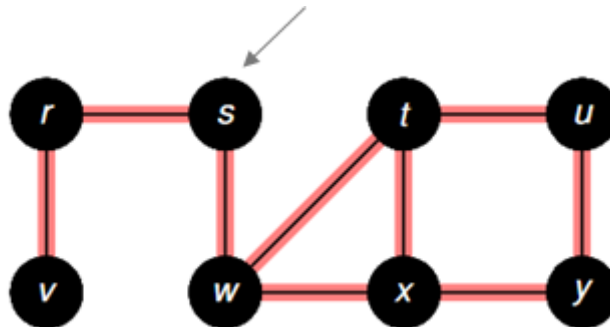
visita primero todos los nodos a distancia 1, luego todos los nodos a distancia 2, después los de distancia 3, y así sucesivamente. Este algoritmo explora por tanto el grafo nivel por nivel: primero visita todos los vecinos directos del nodo inicial, luego los vecinos de esos vecinos, y así sucesivamente, marcando los nodos ya visitados para evitar bucles y repeticiones no deseadas. Utiliza una cola como estructura auxiliar.

Imagina un edificio cuyas habitaciones debes visitar. Empiezas por el vestíbulo. Antes de aventurarte por pasillos, decides asomarte primero a todas las habitaciones que están justo accesibles desde tu posición actual. Una vez vistas, avanzas un paso más, explorando las habitaciones conectadas a cada una de ellas, siempre sin repetir ninguna para no perderte ni dar vueltas en círculo. Así continúas recorriendo el edificio por “capas”: primero lo cercano, luego lo que está un poco más lejos, y después lo siguiente. Ese avance ordenado, casi como si llevaras una cola de estancias pendientes de visitar, es exactamente lo que hace BFS cuando recorre un grafo.

El algoritmo en pseudocódigo sería:

```
inicio
  inicializar(cola)
  marcar(nodo_inicial)
  encolar(cola, nodo_inicial)
  mientras no vacía(cola) hacer
    nodo_actual := frente(cola)
    desencolar(cola)
    procesar(nodo_actual)
    para todo nodo_aux adyacente a nodo_actual hacer
      si no marcado(nodo_aux) entonces
        marcar(nodo_aux)
        encolar(cola, nodo_aux)
    fin_si
  fin_para
fin_mientras
fin BFS
```

Si tomamos el siguiente grafo como ejemplo y hacemos el recorrido paso a paso desde “s”:



Se elige el origen (por ejemplo, nodo S). Se visitan sus vecinos directos (nivel 1). Luego se visitan los vecinos de nivel 2, y así sucesivamente hasta cubrir todos los nodos alcanzables. El orden de visita en el ejemplo sería: S, R, W, V, T, X, U, Y.

Recorrido en profundidad (DFS)

Imagina un explorador frente a un laberinto. Ante múltiples caminos posibles, el explorador elegiría un túnel y lo seguiría hasta su final, sin importar lo lejos que se adentre, registrando y recordando cada bifurcación que encuentre pero posponiendo su exploración hasta haber agotado completamente el camino actual. Solo cuando alcanza un punto sin salida, retorna al último cruce donde había alternativas pendientes y repite el proceso. Esta metáfora captura perfectamente el

espíritu del algoritmo DFS (*Depth First Search* o Búsqueda en profundidad primero): una inmersión vertical antes que una expansión horizontal.

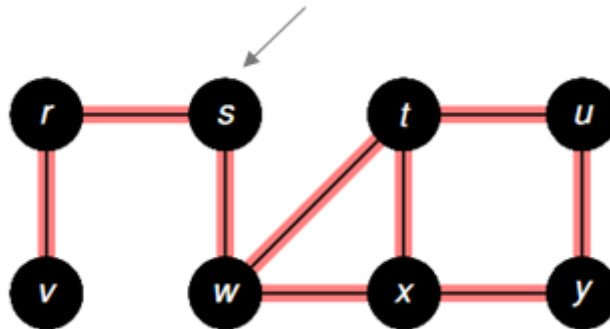
DFS es el contraparte natural de la Búsqueda en Anchura o BFS (*Breadth First Search*), que opera con la filosofía opuesta: mientras DFS desciende como una raíz en busca de agua, BFS se expande como las ondas en un estanque. Esta dualidad no es meramente técnica sino conceptual, reflejando dos modos complementarios de abordar problemas de exploración.

El algoritmo DFS por tanto explora tan lejos como sea posible a lo largo de cada rama antes de retroceder y para ello utiliza una pila auxiliar para gestionar el recorrido.

En pseudocódigo, el algoritmo DFS iterativo con una pila auxiliar sería:

```
inicio
  inicializar(pila)
  apilar(pila, nodo_inicial)
  mientras no vacía(pila) hacer
    nodo_actual := tope(pila)
    si no marcado(nodo_actual) entonces
      procesar(nodo_actual)
      marcar(nodo_actual)
    fin_si
    si existe nodo_adyacente no marcado a nodo_actual entonces
      apilar(pila, nodo_adyacente)
    si_no
      desapilar(pila)
    fin_si
  fin_mientras
fin DFS
```

Si tomamos el grafo anterior como ejemplo y hacemos el recorrido paso a paso desde “s”, tendríamos lo siguiente. Se empieza en el nodo origen (S). Se avanza a un vecino no visitado (por ejemplo, R), luego a un vecino de R (V), y así sucesivamente hasta llegar a un nodo sin vecinos no visitados. Se retrocede (desapila) y se exploran caminos alternativos.



El orden de visita en el ejemplo sería: S, R, V, W, X, Y, U, T.